

CHAPTER

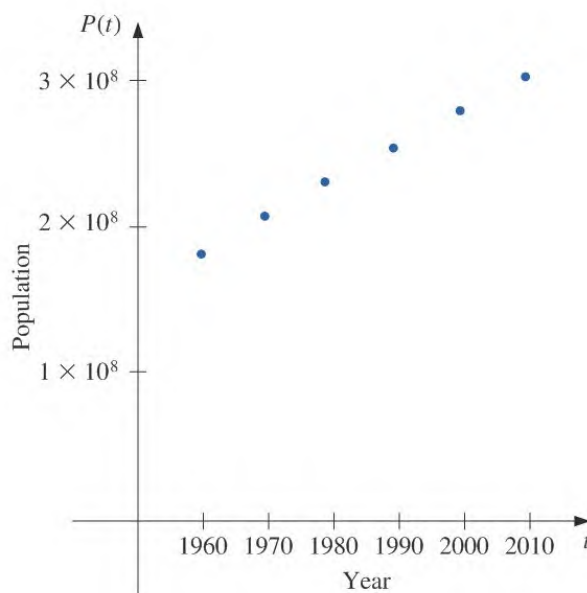
3

Interpolation and Polynomial Approximation

Introduction

A census of the population of the United States is taken every 10 years. The following table lists the population, in thousands of people, from 1960 to 2010, and the data are also represented in the figure.

Year	1960	1970	1980	1990	2000	2010
Population (in thousands)	179,323	203,302	226,542	249,633	281,422	308,746



In reviewing these data, we might ask whether they could be used to provide a reasonable estimate of the population, say, in 1975 or even in the year 2020. Predictions of this type can be obtained by using a function that fits the given data. This process is called *interpolation* and is the subject of this chapter. This population problem is considered throughout the chapter and in Exercises 19 of Section 3.1, 17 of Section 3.3, and 24 of Section 3.5

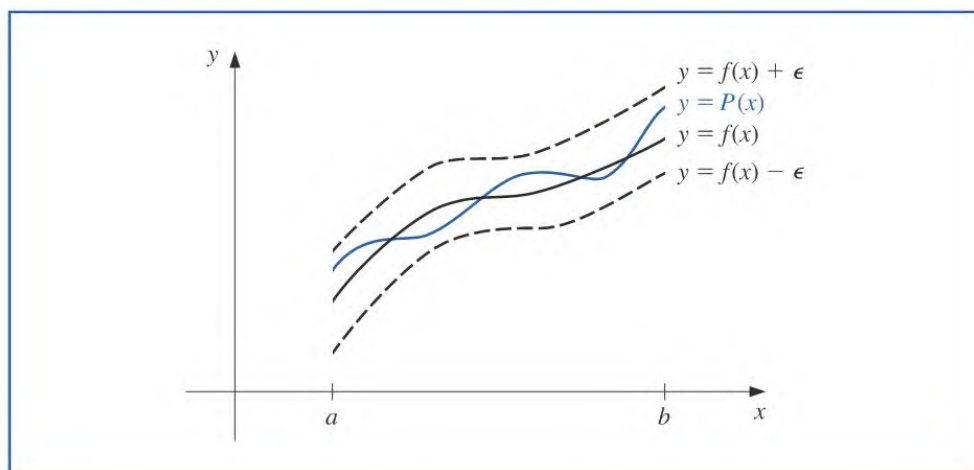
3.1 Interpolation and the Lagrange Polynomial

One of the most useful and well-known classes of functions mapping the set of real numbers into itself is the *algebraic polynomials*, the set of functions of the form

$$P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0,$$

where n is a nonnegative integer and $a_0, \dots, a_n$ are real constants. One reason for their importance is that they uniformly approximate continuous functions. By this we mean that given any function, defined and continuous on a closed and bounded interval, there exists a polynomial that is as “close” to the given function as desired. This result is expressed precisely in the Weierstrass Approximation Theorem. (See Figure 3.1.)

Figure 3.1



Theorem 3.1 (Weierstrass Approximation Theorem)

Suppose f is defined and continuous on $[a, b]$. For each $\epsilon > 0$, there exists a polynomial $P(x)$, with the property that

$$|f(x) - P(x)| < \epsilon, \quad \text{for all } x \text{ in } [a, b].$$

Karl Weierstrass (1815–1897) is often referred to as the father of modern analysis because of his insistence on rigor in the demonstration of mathematical results. He was instrumental in developing tests for convergence of series and determining ways to rigorously define irrational numbers. He was the first to demonstrate that a function could be everywhere continuous but nowhere differentiable, a result that shocked some of his contemporaries.

The proof of this theorem can be found in most elementary texts on real analysis (see, for example, [Bart], pp. 165–172).

Another important reason for considering the class of polynomials in the approximation of functions is that the derivative and indefinite integral of a polynomial are easy to determine and are also polynomials. For these reasons, polynomials are often used for approximating continuous functions.

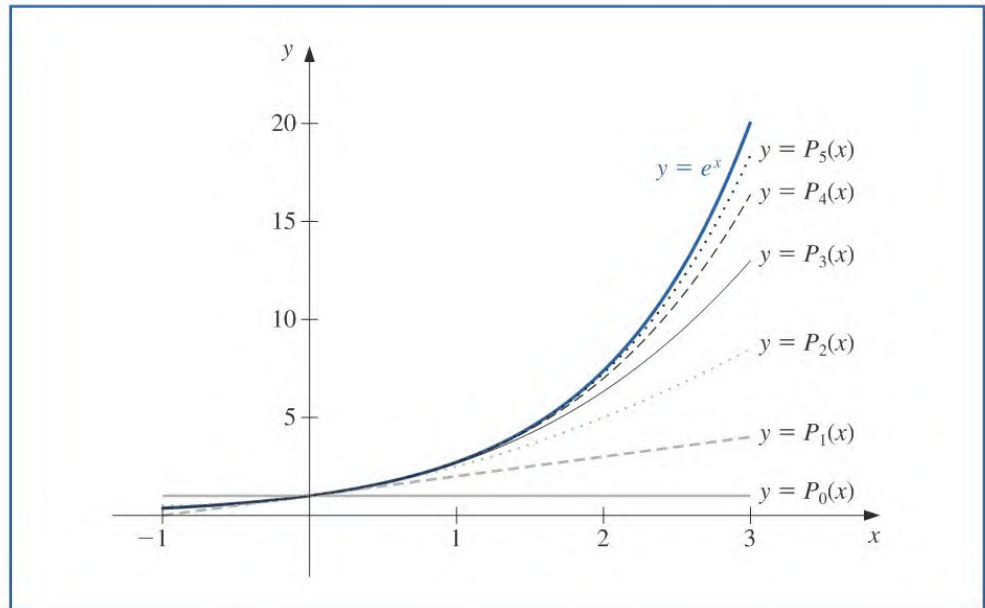
The Taylor polynomials were introduced in Section 1.1, where they were described as one of the fundamental building blocks of numerical analysis. Given this prominence, you might expect that polynomial approximation would make heavy use of these functions. However, this is not the case. The Taylor polynomials agree as closely as possible with a given function at a specific point, but they concentrate their accuracy near that point. A good approximating polynomial needs to provide a relative accuracy over an entire interval, and Taylor polynomials do not generally do this. For example, suppose we calculate the first

six Taylor polynomials about $x_0 = 0$ for $f(x) = e^x$. Since the derivatives of $f(x)$ are all e^x , which evaluated at $x_0 = 0$ gives 1, the Taylor polynomials are

$$\begin{aligned} P_0(x) &= 1, & P_1(x) &= 1 + x, & P_2(x) &= 1 + x + \frac{x^2}{2}, & P_3(x) &= 1 + x + \frac{x^2}{2} + \frac{x^3}{6}, \\ P_4(x) &= 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24}, & \text{and} & & P_5(x) &= 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24} + \frac{x^5}{120}. \end{aligned}$$

The graphs of the polynomials are shown in Figure 3.2. (Notice that even for the higher-degree polynomials, the error becomes progressively worse as we move away from zero.)

Figure 3.2



Although better approximations are obtained for $f(x) = e^x$ if higher-degree Taylor polynomials are used, this is not true for all functions. Consider, as an extreme example, using Taylor polynomials of various degrees for $f(x) = 1/x$ expanded about $x_0 = 1$ to approximate $f(3) = 1/3$. Since

$$f(x) = x^{-1}, \quad f'(x) = -x^{-2}, \quad f''(x) = (-1)^2 2 \cdot x^{-3},$$

and, in general,

$$f^{(k)}(x) = (-1)^k k! x^{-k-1},$$

the Taylor polynomials are

$$P_n(x) = \sum_{k=0}^n \frac{f^{(k)}(1)}{k!} (x-1)^k = \sum_{k=0}^n (-1)^k (x-1)^k.$$

To approximate $f(3) = 1/3$ by $P_n(3)$ for increasing values of n , we obtain the values in Table 3.1—rather a dramatic failure! When we approximate $f(3) = 1/3$ by $P_n(3)$ for larger values of n , the approximations become increasingly inaccurate.

Table 3.1

n	0	1	2	3	4	5	6	7
$P_n(3)$	1	-1	3	-5	11	-21	43	-85

For the Taylor polynomials, all the information used in the approximation is concentrated at the single number x_0 , so these polynomials will generally give inaccurate approximations as we move away from x_0 . This limits Taylor polynomial approximation to the situation in which approximations are needed only at numbers close to x_0 . For ordinary computational purposes, it is more efficient to use methods that include information at various points. We consider this in the remainder of the chapter. The primary use of Taylor polynomials in numerical analysis is not for approximation purposes but rather for the derivation of numerical techniques and error estimation.

Lagrange Interpolating Polynomials

The problem of determining a polynomial of degree one that passes through the distinct points (x_0, y_0) and (x_1, y_1) is the same as approximating a function f for which $f(x_0) = y_0$ and $f(x_1) = y_1$ by means of a first-degree polynomial **interpolating**, or agreeing with, the values of f at the given points. Using this polynomial for approximation within the interval given by the endpoints is called polynomial **interpolation**.

Define the functions

$$L_0(x) = \frac{x - x_1}{x_0 - x_1} \quad \text{and} \quad L_1(x) = \frac{x - x_0}{x_1 - x_0}.$$

The linear **Lagrange interpolating polynomial** through (x_0, y_0) and (x_1, y_1) is

$$P(x) = L_0(x)f(x_0) + L_1(x)f(x_1) = \frac{x - x_1}{x_0 - x_1}f(x_0) + \frac{x - x_0}{x_1 - x_0}f(x_1).$$

Note that

$$L_0(x_0) = 1, \quad L_0(x_1) = 0, \quad L_1(x_0) = 0, \quad \text{and} \quad L_1(x_1) = 1,$$

which implies that

$$P(x_0) = 1 \cdot f(x_0) + 0 \cdot f(x_1) = f(x_0) = y_0$$

and

$$P(x_1) = 0 \cdot f(x_0) + 1 \cdot f(x_1) = f(x_1) = y_1.$$

So, P is the unique polynomial of degree at most one that passes through (x_0, y_0) and (x_1, y_1) .

Example 1 Determine the linear Lagrange interpolating polynomial that passes through the points $(2, 4)$ and $(5, 1)$.

Solution In this case, we have

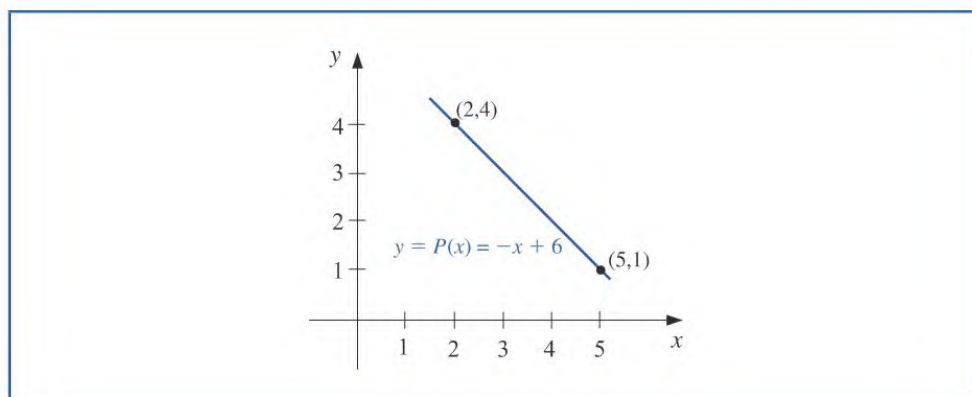
$$L_0(x) = \frac{x - 5}{2 - 5} = -\frac{1}{3}(x - 5) \quad \text{and} \quad L_1(x) = \frac{x - 2}{5 - 2} = \frac{1}{3}(x - 2),$$

so

$$P(x) = -\frac{1}{3}(x - 5) \cdot 4 + \frac{1}{3}(x - 2) \cdot 1 = -\frac{4}{3}x + \frac{20}{3} + \frac{1}{3}x - \frac{2}{3} = -x + 6.$$

The graph of $y = P(x)$ is shown in Figure 3.3. ■

Figure 3.3

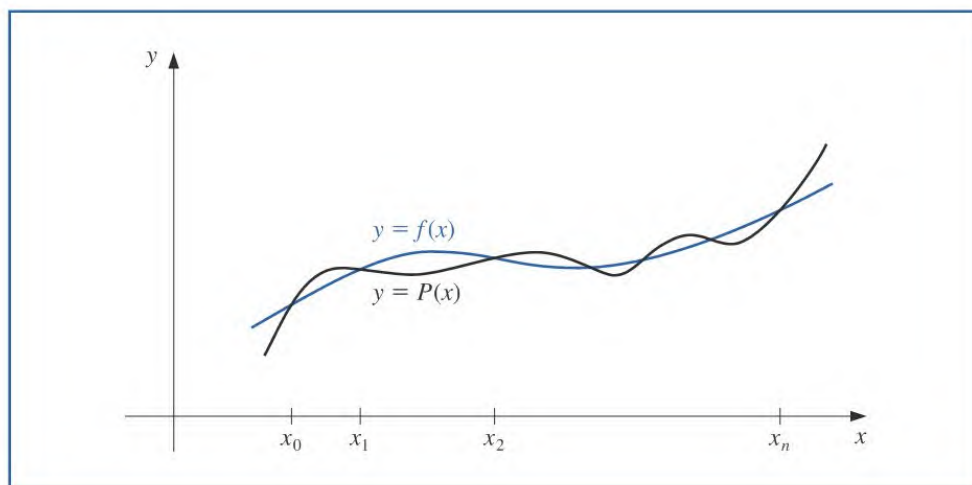


To generalize the concept of linear interpolation, consider the construction of a polynomial of degree at most n that passes through the $n + 1$ points

$$(x_0, f(x_0)), (x_1, f(x_1)), \dots, (x_n, f(x_n)).$$

(See Figure 3.4.)

Figure 3.4



In this case, we first construct, for each $k = 0, 1, \dots, n$, a function $L_{n,k}(x)$ with the property that $L_{n,k}(x_i) = 0$ when $i \neq k$ and $L_{n,k}(x_k) = 1$. To satisfy $L_{n,k}(x_i) = 0$ for each $i \neq k$ requires that the numerator of $L_{n,k}(x)$ contain the term

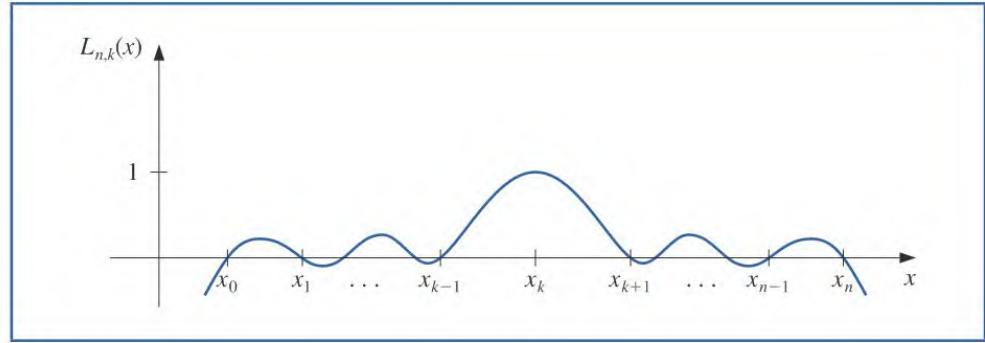
$$(x - x_0)(x - x_1) \cdots (x - x_{k-1})(x - x_{k+1}) \cdots (x - x_n).$$

To satisfy $L_{n,k}(x_k) = 1$, the denominator of $L_{n,k}(x)$ must be this same term but evaluated at $x = x_k$. Thus,

$$L_{n,k}(x) = \frac{(x - x_0) \cdots (x - x_{k-1})(x - x_{k+1}) \cdots (x - x_n)}{(x_k - x_0) \cdots (x_k - x_{k-1})(x_k - x_{k+1}) \cdots (x_k - x_n)}.$$

A sketch of the graph of a typical $L_{n,k}$ (when n is even) is shown in Figure 3.5.

Figure 3.5



The interpolating polynomial is easily described once the form of $L_{n,k}$ is known. This polynomial, called the **n th Lagrange interpolating polynomial**, is defined in the following theorem.

Theorem 3.2

If $x_0, x_1, \dots, x_n$ are $n + 1$ distinct numbers and f is a function whose values are given at these numbers, then a unique polynomial $P(x)$ of degree at most n exists with

$$f(x_k) = P(x_k), \quad \text{for each } k = 0, 1, \dots, n.$$

This polynomial is given by

$$P(x) = f(x_0)L_{n,0}(x) + \cdots + f(x_n)L_{n,n}(x) = \sum_{k=0}^n f(x_k)L_{n,k}(x), \quad (3.1)$$

where, for each $k = 0, 1, \dots, n$,

$$\begin{aligned} L_{n,k}(x) &= \frac{(x - x_0)(x - x_1) \cdots (x - x_{k-1})(x - x_{k+1}) \cdots (x - x_n)}{(x_k - x_0)(x_k - x_1) \cdots (x_k - x_{k-1})(x_k - x_{k+1}) \cdots (x_k - x_n)} \\ &= \prod_{\substack{i=0 \\ i \neq k}}^n \frac{(x - x_i)}{(x_k - x_i)}. \end{aligned} \quad (3.2)$$

We will write $L_{n,k}(x)$ simply as $L_k(x)$ when there is no confusion as to its degree.

The interpolation formula named for Joseph Louis Lagrange (1736–1813) was likely known by Isaac Newton around 1675, but it appears to first have been published in 1779 by Edward Waring (1736–1798). Lagrange wrote extensively on the subject of interpolation, and his work had significant influence on later mathematicians. He published this result in 1795.

The symbol $\prod$ is used to write products compactly and parallels the symbol $\sum$, which is used for writing sums. For example, $\prod_{i=0}^3 a_i = a_1 * a_2 * a_3$.

Example 2

(a) Use the numbers (called *nodes*)

$x_0 = 2$, $x_1 = 2.75$, and $x_2 = 4$ to find the second Lagrange interpolating polynomial for $f(x) = 1/x$.

(b) Use this polynomial to approximate $f(3) = 1/3$.

Solution (a) We first determine the coefficient polynomials $L_0(x)$, $L_1(x)$, and $L_2(x)$. In nested form, they are

$$L_0(x) = \frac{(x - 2.75)(x - 4)}{(2 - 2.75)(2 - 4)} = \frac{2}{3}(x - 2.75)(x - 4),$$

$$L_1(x) = \frac{(x - 2)(x - 4)}{(2.75 - 2)(2.75 - 4)} = -\frac{16}{15}(x - 2)(x - 4),$$

and

$$L_2(x) = \frac{(x - 2)(x - 2.75)}{(4 - 2)(4 - 2.75)} = \frac{2}{5}(x - 2)(x - 2.75).$$

Also, $f(x_0) = f(2) = 1/2$, $f(x_1) = f(2.75) = 4/11$, and $f(x_2) = f(4) = 1/4$, so

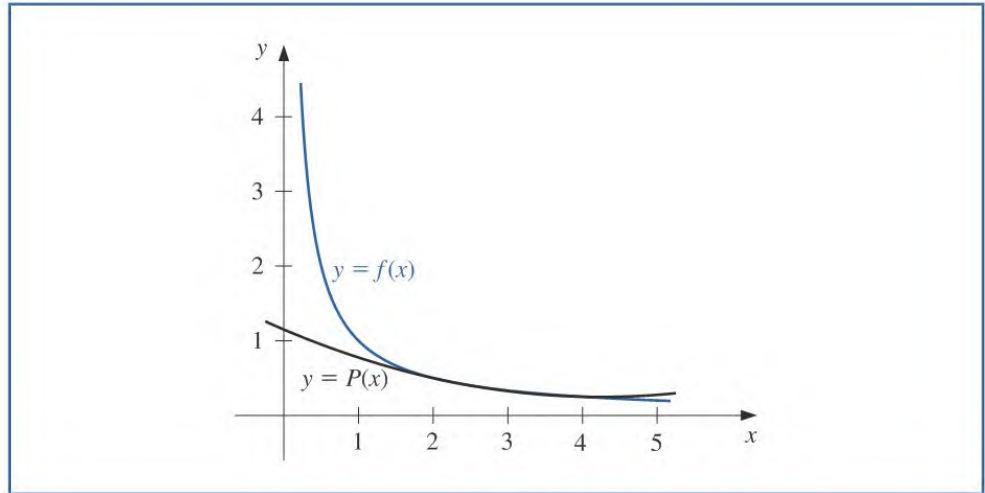
$$\begin{aligned} P(x) &= \sum_{k=0}^2 f(x_k) L_k(x) \\ &= \frac{1}{3}(x - 2.75)(x - 4) - \frac{64}{165}(x - 2)(x - 4) + \frac{1}{10}(x - 2)(x - 2.75) \\ &= \frac{1}{22}x^2 - \frac{35}{88}x + \frac{49}{44}. \end{aligned}$$

(b) An approximation to $f(3) = 1/3$ (see Figure 3.6) is

$$f(3) \approx P(3) = \frac{9}{22} - \frac{105}{88} + \frac{49}{44} = \frac{29}{88} \approx 0.32955.$$

Recall that in the opening section of this chapter (see Table 3.1), we found that no Taylor polynomial expanded about $x_0 = 1$ could be used to reasonably approximate $f(x) = 1/x$ at $x = 3$. ■

Figure 3.6



The next step is to calculate a remainder term or bound for the error involved in approximating a function by an interpolating polynomial.

Theorem 3.3

Suppose $x_0, x_1, \dots, x_n$ are distinct numbers in the interval $[a, b]$ and $f \in C^{n+1}[a, b]$. Then, for each x in $[a, b]$, a number $\xi(x)$ (generally unknown) between $\min\{x_0, x_1, \dots, x_n\}$, and the $\max\{x_0, x_1, \dots, x_n\}$ and hence in (a, b) , exists with

$$f(x) = P(x) + \frac{f^{(n+1)}(\xi(x))}{(n+1)!} (x - x_0)(x - x_1) \cdots (x - x_n), \quad (3.3)$$

where $P(x)$ is the interpolating polynomial given in Eq. (3.1).

Proof Note first that if $x = x_k$, for any $k = 0, 1, \dots, n$, then $f(x_k) = P(x_k)$, and choosing $\xi(x_k)$ arbitrarily in (a, b) yields Eq. (3.3).

There are other ways that the error term for the Lagrange polynomial can be expressed, but this is the most useful form and the one that most closely agrees with the standard Taylor polynomial error form.

If $x \neq x_k$, for all $k = 0, 1, \dots, n$, define the function g for t in $[a, b]$ by

$$\begin{aligned} g(t) &= f(t) - P(t) - [f(x) - P(x)] \frac{(t - x_0)(t - x_1) \cdots (t - x_n)}{(x - x_0)(x - x_1) \cdots (x - x_n)} \\ &= f(t) - P(t) - [f(x) - P(x)] \prod_{i=0}^n \frac{(t - x_i)}{(x - x_i)}. \end{aligned}$$

Since $f \in C^{n+1}[a, b]$, and $P \in C^\infty[a, b]$, it follows that $g \in C^{n+1}[a, b]$. For $t = x_k$, we have

$$g(x_k) = f(x_k) - P(x_k) - [f(x) - P(x)] \prod_{i=0}^n \frac{(x_k - x_i)}{(x - x_i)} = 0 - [f(x) - P(x)] \cdot 0 = 0.$$

Moreover,

$$g(x) = f(x) - P(x) - [f(x) - P(x)] \prod_{i=0}^n \frac{(x - x_i)}{(x - x_i)} = f(x) - P(x) - [f(x) - P(x)] = 0.$$

Thus, $g \in C^{n+1}[a, b]$, and g is zero at the $n + 2$ distinct numbers $x, x_0, x_1, \dots, x_n$. By Generalized Rolle's Theorem 1.10, there exists a number ξ in (a, b) for which $g^{(n+1)}(\xi) = 0$. So,

$$0 = g^{(n+1)}(\xi) = f^{(n+1)}(\xi) - P^{(n+1)}(\xi) - [f(x) - P(x)] \frac{d^{n+1}}{dt^{n+1}} \left[\prod_{i=0}^n \frac{(t - x_i)}{(x - x_i)} \right]_{t=\xi}. \quad (3.4)$$

However, $P(x)$ is a polynomial of degree at most n , so the $(n+1)$ st derivative, $P^{(n+1)}(x)$, is identically zero. Also, $\prod_{i=0}^n [(t - x_i)/(x - x_i)]$ is a polynomial of degree $(n + 1)$, so

$$\prod_{i=0}^n \frac{(t - x_i)}{(x - x_i)} = \left[\frac{1}{\prod_{i=0}^n (x - x_i)} \right] t^{n+1} + (\text{lower-degree terms in } t),$$

and

$$\frac{d^{n+1}}{dt^{n+1}} \prod_{i=0}^n \frac{(t - x_i)}{(x - x_i)} = \frac{(n+1)!}{\prod_{i=0}^n (x - x_i)}.$$

Equation (3.4) now becomes

$$0 = f^{(n+1)}(\xi) - 0 - [f(x) - P(x)] \frac{(n+1)!}{\prod_{i=0}^n (x - x_i)},$$

and, upon solving for $f(x)$, we have

$$f(x) = P(x) + \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i). \quad \blacksquare$$

The error formula in Theorem 3.3 is an important theoretical result because Lagrange polynomials are used extensively for deriving numerical differentiation and integration methods. Error bounds for these techniques are obtained from the Lagrange error formula.

Note that the error form for the Lagrange polynomial is quite similar to that for the Taylor polynomial. The n th Taylor polynomial about x_0 concentrates all the known information at x_0 and has an error term of the form

$$\frac{f^{(n+1)}(\xi(x))}{(n+1)!} (x - x_0)^{n+1}.$$

The Lagrange polynomial of degree n uses information at the distinct numbers $x_0, x_1, \dots, x_n$, and, in place of $(x - x_0)^n$, its error formula uses a product of the $n + 1$ terms $(x - x_0), (x - x_1), \dots, (x - x_n)$:

$$\frac{f^{(n+1)}(\xi(x))}{(n+1)!}(x - x_0)(x - x_1) \cdots (x - x_n).$$

Example 3 In Example 2, we found the second Lagrange polynomial for $f(x) = 1/x$ on $[2, 4]$ using the nodes $x_0 = 2, x_1 = 2.75$, and $x_2 = 4$. Determine the error form for this polynomial and the maximum error when the polynomial is used to approximate $f(x)$ for $x \in [2, 4]$.

Solution Because $f(x) = x^{-1}$, we have

$$f'(x) = -x^{-2}, \quad f''(x) = 2x^{-3}, \quad \text{and} \quad f'''(x) = -6x^{-4}.$$

As a consequence, the second Lagrange polynomial has the error form

$$\frac{f'''(\xi(x))}{3!}(x - x_0)(x - x_1)(x - x_2) = -(\xi(x))^{-4}(x - 2)(x - 2.75)(x - 4), \quad \text{for } \xi(x) \text{ in } (2, 4).$$

The maximum value of $(\xi(x))^{-4}$ on the interval is $2^{-4} = 1/16$. We now need to determine the maximum value on this interval of the absolute value of the polynomial

$$g(x) = (x - 2)(x - 2.75)(x - 4) = x^3 - \frac{35}{4}x^2 + \frac{49}{2}x - 22.$$

Because

$$D_x \left(x^3 - \frac{35}{4}x^2 + \frac{49}{2}x - 22 \right) = 3x^2 - \frac{35}{2}x + \frac{49}{2} = \frac{1}{2}(3x - 7)(2x - 7),$$

the critical points occur at

$$x = \frac{7}{3}, \quad \text{with } g\left(\frac{7}{3}\right) = \frac{25}{108}, \quad \text{and} \quad x = \frac{7}{2}, \quad \text{with } g\left(\frac{7}{2}\right) = -\frac{9}{16}.$$

Hence, the maximum error is

$$\frac{f'''(\xi(x))}{3!}|(x - x_0)(x - x_1)(x - x_2)| \leq \frac{1}{16} \left| -\frac{9}{16} \right| = \frac{9}{256} \approx 0.03515625. \quad \blacksquare$$

The next example illustrates how the error formula can be used to prepare a table of data that will ensure a specified interpolation error within a specified bound.

Example 4 Suppose a table is to be prepared for the function $f(x) = e^x$, for x in $[0, 1]$. Assume the number of decimal places to be given per entry is $d \geq 8$ and that the difference between adjacent x -values, the step size, is h . What step size h will ensure that linear interpolation gives an absolute error of at most 10^{-6} for all x in $[0, 1]$?

Solution Let $x_0, x_1, \dots$ be the numbers at which f is evaluated and x be in $[0, 1]$ and suppose j satisfies $x_j \leq x \leq x_{j+1}$. Eq. (3.3) implies that the error in linear interpolation is

$$|f(x) - P(x)| = \left| \frac{f^{(2)}(\xi)}{2!}(x - x_j)(x - x_{j+1}) \right| = \frac{|f^{(2)}(\xi)|}{2}|(x - x_j)||x - x_{j+1}|.$$

The step size is h , so $x_j = jh, x_{j+1} = (j + 1)h$, and

$$|f(x) - P(x)| \leq \frac{|f^{(2)}(\xi)|}{2}|(x - jh)(x - (j + 1)h)|.$$

Hence,

$$\begin{aligned}|f(x) - P(x)| &\leq \frac{\max_{\xi \in [0,1]} e^{\xi}}{2} \max_{x_j \leq x \leq x_{j+1}} |(x - jh)(x - (j+1)h)| \\ &\leq \frac{e}{2} \max_{x_j \leq x \leq x_{j+1}} |(x - jh)(x - (j+1)h)|.\end{aligned}$$

Consider the function $g(x) = (x - jh)(x - (j+1)h)$, for $jh \leq x \leq (j+1)h$. Because

$$g'(x) = (x - (j+1)h) + (x - jh) = 2\left(x - jh - \frac{h}{2}\right),$$

the only critical point for g is at $x = jh + h/2$, with $g(jh + h/2) = (h/2)^2 = h^2/4$.

Since $g(jh) = 0$ and $g((j+1)h) = 0$, the maximum value of $|g'(x)|$ in $[jh, (j+1)h]$ must occur at the critical point, which implies that (See Exercise 21)

$$|f(x) - P(x)| \leq \frac{e}{2} \max_{x_j \leq x \leq x_{j+1}} |g(x)| \leq \frac{e}{2} \cdot \frac{h^2}{4} = \frac{eh^2}{8}.$$

Consequently, to ensure that the error in linear interpolation is bounded by 10^{-6} , it is sufficient for h to be chosen so that

$$\frac{eh^2}{8} \leq 10^{-6}. \quad \text{This implies that } h < 1.72 \times 10^{-3}.$$

Because $n = (1 - 0)/h$ must be an integer, a reasonable choice for the step size is $h = 0.001$. ■

EXERCISE SET 3.1

- For the given functions $f(x)$, let $x_0 = 0$, $x_1 = 0.6$, and $x_2 = 0.9$. Construct interpolation polynomials of degree at most one and at most two to approximate $f(0.45)$ and find the absolute error.
 - $f(x) = \cos x$
 - $f(x) = \sqrt{1+x}$
 - $f(x) = \ln(x+1)$
 - $f(x) = \tan x$
- For the given functions $f(x)$, let $x_0 = 1$, $x_1 = 1.25$, and $x_2 = 1.6$. Construct interpolation polynomials of degree at most one and at most two to approximate $f(1.4)$ and find the absolute error.
 - $f(x) = \sin \pi x$
 - $f(x) = \sqrt[3]{x-1}$
 - $f(x) = \log_{10}(3x-1)$
 - $f(x) = e^{2x} - x$
- Use Theorem 3.3 to find an error bound for the approximations in Exercise 1.
- Use Theorem 3.3 to find an error bound for the approximations in Exercise 2.
- Use appropriate Lagrange interpolating polynomials of degrees one, two, and three to approximate each of the following:
 - $f(8.4)$ if $f(8.1) = 16.94410$, $f(8.3) = 17.56492$, $f(8.6) = 18.50515$, $f(8.7) = 18.82091$
 - $f(-\frac{1}{3})$ if $f(-0.75) = -0.07181250$, $f(-0.5) = -0.02475000$, $f(-0.25) = 0.33493750$, $f(0) = 1.10100000$
 - $f(0.25)$ if $f(0.1) = 0.62049958$, $f(0.2) = -0.28398668$, $f(0.3) = 0.00660095$, $f(0.4) = 0.24842440$
 - $f(0.9)$ if $f(0.6) = -0.17694460$, $f(0.7) = 0.01375227$, $f(0.8) = 0.22363362$, $f(1.0) = 0.65809197$
- Use appropriate Lagrange interpolating polynomials of degrees one, two, and three to approximate each of the following:
 - $f(0.43)$ if $f(0) = 1$, $f(0.25) = 1.64872$, $f(0.5) = 2.71828$, $f(0.75) = 4.48169$
 - $f(0)$ if $f(-0.5) = 1.93750$, $f(-0.25) = 1.33203$, $f(0.25) = 0.800781$, $f(0.5) = 0.687500$

- c. $f(0.18)$ if $f(0.1) = -0.29004986$, $f(0.2) = -0.56079734$, $f(0.3) = -0.81401972$, $f(0.4) = -1.0526302$
- d. $f(0.25)$ if $f(-1) = 0.86199480$, $f(-0.5) = 0.95802009$, $f(0) = 1.0986123$, $f(0.5) = 1.2943767$
7. The data for Exercise 5 were generated using the following functions. Use the error formula to find a bound for the error and compare the bound to the actual error for the cases $n = 1$ and $n = 2$.
- $f(x) = x \ln x$
 - $f(x) = x^3 + 4.001x^2 + 4.002x + 1.101$
 - $f(x) = x \cos x - 2x^2 + 3x - 1$
 - $f(x) = \sin(e^x - 2)$
8. The data for Exercise 6 were generated using the following functions. Use the error formula to find a bound for the error and compare the bound to the actual error for the cases $n = 1$ and $n = 2$.
- $f(x) = e^{2x}$
 - $f(x) = x^4 - x^3 + x^2 - x + 1$
 - $f(x) = x^2 \cos x - 3x$
 - $f(x) = \ln(e^x + 2)$
9. Let $P_3(x)$ be the interpolating polynomial for the data $(0, 0)$, $(0.5, y)$, $(1, 3)$, and $(2, 2)$. The coefficient of x^3 in $P_3(x)$ is 6. Find y .
10. Let $f(x) = \sqrt{x - x^2}$ and $P_2(x)$ be the interpolation polynomial on $x_0 = 0$, x_1 and $x_2 = 1$. Find the largest value of x_1 in $(0, 1)$ for which $f(0.5) - P_2(0.5) = -0.25$.
11. Use the following values and four-digit rounding arithmetic to construct a third Lagrange polynomial approximation to $f(1.09)$. The function being approximated is $f(x) = \log_{10}(\tan x)$. Use this knowledge to find a bound for the error in the approximation.

$$f(1.00) = 0.1924 \quad f(1.05) = 0.2414 \quad f(1.10) = 0.2933 \quad f(1.15) = 0.3492$$

12. Use the Lagrange interpolating polynomial of degree three or less and four-digit chopping arithmetic to approximate $\cos 0.750$ using the following values. Find an error bound for the approximation.

$$\cos 0.698 = 0.7661 \quad \cos 0.733 = 0.7432 \quad \cos 0.768 = 0.7193 \quad \cos 0.803 = 0.6946$$

The actual value of $\cos 0.750$ is 0.7317 (to four decimal places). Explain the discrepancy between the actual error and the error bound.

13. Construct the Lagrange interpolating polynomials for the following functions and find a bound for the absolute error on the interval $[x_0, x_n]$.
- $f(x) = e^{2x} \cos 3x$, $x_0 = 0$, $x_1 = 0.3$, $x_2 = 0.6$, $n = 2$
 - $f(x) = \sin(\ln x)$, $x_0 = 2.0$, $x_1 = 2.4$, $x_2 = 2.6$, $n = 2$
 - $f(x) = \ln x$, $x_0 = 1$, $x_1 = 1.1$, $x_2 = 1.3$, $x_3 = 1.4$, $n = 3$
 - $f(x) = \cos x + \sin x$, $x_0 = 0$, $x_1 = 0.25$, $x_2 = 0.5$, $x_3 = 1.0$, $n = 3$
14. Construct the Lagrange interpolating polynomials for the following functions, and find a bound for the absolute error on the interval $[x_0, x_n]$.
- $f(x) = e^{-2x} \sin 3x$, $x_0 = 0$, $x_1 = \frac{\pi}{6}$, $x_2 = \frac{\pi}{4}$, $n = 2$
 - $f(x) = \log_{10} x$, $x_0 = 3.0$, $x_1 = 3.2$, $x_2 = 3.5$, $n = 2$
 - $f(x) = e^x + e^{-x}$, $x_0 = -0.3$, $x_1 = 0$, $x_2 = 0.3$, $n = 2$
 - $f(x) = \cos(2 \ln(3x))$, $x_0 = 0$, $x_1 = 0.3$, $x_2 = 0.5$, $x_3 = 1.0$, $n = 3$
15. Let $f(x) = e^x$, for $0 \leq x \leq 2$.
- Approximate $f(0.25)$ using linear interpolation with $x_0 = 0$ and $x_1 = 0.5$.
 - Approximate $f(0.75)$ using linear interpolation with $x_0 = 0.5$ and $x_1 = 1$.
 - Approximate $f(0.25)$ and $f(0.75)$ using the second interpolating polynomial with $x_0 = 0$, $x_1 = 1$, and $x_2 = 2$.
 - Which approximations are better, and why?

16. Let $f(x) = e^{-x} \cos x$, for $0 \leq x \leq 1$.
- Approximate $f(0.25)$ using linear interpolation with $x_0 = 0$ and $x_1 = 0.5$.
 - Approximate $f(0.75)$ using linear interpolation with $x_0 = 0.5$ and $x_1 = 1$.
 - Approximate $f(0.25)$ and $f(0.75)$ using the second interpolating polynomial with $x_0 = 0$, $x_1 = 0.5$, and $x_2 = 1.0$.
 - Which approximations are better, and why?
17. Suppose you need to construct eight-decimal-place tables for the common, or base-10, logarithm function from $x = 1$ to $x = 10$ in such a way that linear interpolation is accurate to within 10^{-6} . Determine a bound for the step size for this table. What choice of step size would you make to ensure that $x = 10$ is included in the table?
18. In Exercise 24 of Section 1.1, a Maclaurin series was integrated to approximate $\text{erf}(1)$, where $\text{erf}(x)$ is the normal distribution error function defined by

$$\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt.$$

- Use the Maclaurin series to construct a table for $\text{erf}(x)$ that is accurate to within 10^{-4} for $\text{erf}(x_i)$, where $x_i = 0.2i$, for $i = 0, 1, \dots, 5$.
- Use both linear interpolation and quadratic interpolation to obtain an approximation to $\text{erf}(\frac{1}{3})$. Which approach seems most feasible?

APPLIED EXERCISES

19.
 - The introduction to this chapter included a table listing the population of the United States from 1960 to 2010. Use Lagrange interpolation to approximate the population in the years 1950, 1975, 2014, and 2020.
 - The population in 1950 was approximately 150,697,360, and in 2014 the population was estimated to be 317,298,000. How accurate do you think your 1975 and 2020 figures are?
20. It is suspected that the high amounts of tannin in mature oak leaves inhibit the growth of the winter moth (*Operophtera bromata* L., *Geometridae*) larvae that extensively damage these trees in certain years. The following table lists the average weight of two samples of larvae at times in the first 28 days after birth. The first sample was reared on young oak leaves, whereas the second sample was reared on mature leaves from the same tree.
- Use Lagrange interpolation to approximate the average weight curve for each sample.
 - Find an approximate maximum average weight for each sample by determining the maximum of the interpolating polynomial.

Day	0	6	10	13	17	20	28
Sample 1 average weight (mg)	6.67	17.33	42.67	37.33	30.10	29.31	28.74
Sample 2 average weight (mg)	6.67	16.11	18.89	15.00	10.56	9.44	8.89

THEORETICAL EXERCISES

21. Show that $\max_{x_j \leq x \leq x_{j+1}} |g(x)| = h^2/4$, where $g(x) = (x - jh)(x - (j + 1)h)$.
22. Prove Taylor's Theorem 1.14 by following the procedure in the proof of Theorem 3.3. [Hint: Let

$$g(t) = f(t) - P(t) - [f(x) - P(x)] \cdot \frac{(t - x_0)^{n+1}}{(x - x_0)^{n+1}},$$

where P is the n th Taylor polynomial, and use the Generalized Rolle's Theorem 1.10.]

23. The Bernstein polynomial of degree n for $f \in C[0, 1]$ is given by

$$B_n(x) = \sum_{k=0}^n \binom{n}{k} f\left(\frac{k}{n}\right) x^k (1-x)^{n-k},$$

where $\binom{n}{k}$ denotes $n!/k!(n-k)!$. These polynomials can be used in a constructive proof of the Weierstrass Approximation Theorem 3.1 (see [Bart]) because $\lim_{n \rightarrow \infty} B_n(x) = f(x)$, for each $x \in [0, 1]$.

a. Find $B_3(x)$ for the functions

i. $f(x) = x$

ii. $f(x) = 1$

b. Show that for each $k \leq n$,

$$\binom{n-1}{k-1} = \binom{k}{n} \binom{n}{k}.$$

c. Use part (b) and the fact, from (ii) in part (a), that

$$1 = \sum_{k=0}^n \binom{n}{k} x^k (1-x)^{n-k}, \quad \text{for each } n,$$

to show that, for $f(x) = x^2$,

$$B_n(x) = \left(\frac{n-1}{n}\right)x^2 + \frac{1}{n}x.$$

d. Use part (c) to estimate the value of n necessary for $|B_n(x) - x^2| \leq 10^{-6}$ to hold for all x in $[0, 1]$.

DISCUSSION QUESTIONS

- Suppose that we use the Lagrange polynomial to fit two given data sets that match exactly except for a small perturbation in one of the data points due to measurement error. Although the perturbation is small, the change in the Lagrange polynomial is large. Explain why this discrepancy occurs.
- If we decide to increase the degree of the interpolating polynomial by adding nodes, is there an easy way to use a previous interpolating polynomial to obtain a higher-degree interpolating polynomial, or do we need to start over?

3.2 Data Approximation and Neville's Method

In the previous section, we found an explicit representation for Lagrange polynomials and their error when approximating a function on an interval. A frequent use of these polynomials involves the interpolation of tabulated data. In this case, an explicit representation of the polynomial might not be needed, only the values of the polynomial at specified points. In this situation, the function underlying the data might not be known, so the explicit form of the error cannot be used. We will now illustrate a practical application of interpolation in such a situation.

Illustration

Table 3.2 lists values of a function f at various points. The approximations to $f(1.5)$ obtained by various Lagrange polynomials that use these data will be compared to try and determine the accuracy of the approximation.

Table 3.2

x	$f(x)$
1.0	0.7651977
1.3	0.6200860
1.6	0.4554022
1.9	0.2818186
2.2	0.1103623

The most appropriate linear polynomial uses $x_0 = 1.3$ and $x_1 = 1.6$ because 1.5 is between 1.3 and 1.6. The value of the interpolating polynomial at 1.5 is

$$\begin{aligned} P_1(1.5) &= \frac{(1.5 - 1.6)}{(1.3 - 1.6)} f(1.3) + \frac{(1.5 - 1.3)}{(1.6 - 1.3)} f(1.6) \\ &= \frac{(1.5 - 1.6)}{(1.3 - 1.6)} (0.6200860) + \frac{(1.5 - 1.3)}{(1.6 - 1.3)} (0.4554022) = 0.5102968. \end{aligned}$$

Two polynomials of degree two can reasonably be used, one with $x_0 = 1.3$, $x_1 = 1.6$, and $x_2 = 1.9$, which gives

$$\begin{aligned} P_2(1.5) = & \frac{(1.5 - 1.6)(1.5 - 1.9)}{(1.3 - 1.6)(1.3 - 1.9)}(0.6200860) + \frac{(1.5 - 1.3)(1.5 - 1.9)}{(1.6 - 1.3)(1.6 - 1.9)}(0.4554022) \\ & + \frac{(1.5 - 1.3)(1.5 - 1.6)}{(1.9 - 1.3)(1.9 - 1.6)}(0.2818186) = 0.5112857, \end{aligned}$$

and one with $x_0 = 1.0$, $x_1 = 1.3$, and $x_2 = 1.6$, which gives $\hat{P}_2(1.5) = 0.5124715$.

In the third-degree case, there are also two reasonable choices for the polynomial, one with $x_0 = 1.3$, $x_1 = 1.6$, $x_2 = 1.9$, and $x_3 = 2.2$, which gives $P_3(1.5) = 0.5118302$. The second third-degree approximation is obtained with $x_0 = 1.0$, $x_1 = 1.3$, $x_2 = 1.6$, and $x_3 = 1.9$, which gives $\hat{P}_3(1.5) = 0.5118127$.

The fourth-degree Lagrange polynomial uses all the entries in the table. With $x_0 = 1.0$, $x_1 = 1.3$, $x_2 = 1.6$, $x_3 = 1.9$, and $x_4 = 2.2$, the approximation is $P_4(1.5) = 0.5118200$.

Because $P_3(1.5)$, $\hat{P}_3(1.5)$, and $P_4(1.5)$ all agree to within 2×10^{-5} units, we expect this degree of accuracy for these approximations. We also expect $P_4(1.5)$ to be the most accurate approximation since it uses more of the given data.

The function we are approximating is actually the Bessel function of the first kind of order zero, whose value at 1.5 is known to be 0.5118277. Therefore, the true accuracies of the approximations are as follows:

$$\begin{aligned} |P_1(1.5) - f(1.5)| &\approx 1.53 \times 10^{-3}, \\ |P_2(1.5) - f(1.5)| &\approx 5.42 \times 10^{-4}, \\ |\hat{P}_2(1.5) - f(1.5)| &\approx 6.44 \times 10^{-4}, \\ |P_3(1.5) - f(1.5)| &\approx 2.5 \times 10^{-6}, \\ |\hat{P}_3(1.5) - f(1.5)| &\approx 1.50 \times 10^{-5}, \\ |P_4(1.5) - f(1.5)| &\approx 7.7 \times 10^{-6}. \end{aligned}$$

Although $P_3(1.5)$ is the most accurate approximation, if we had no knowledge of the actual value of $f(1.5)$, we would accept $P_4(1.5)$ as the best approximation since it includes the most data about the function. The Lagrange error term derived in Theorem 3.3 cannot be applied here because we have no knowledge of the fourth derivative of f . Unfortunately, this is generally the case. ■

Neville's Method

A practical difficulty with Lagrange interpolation is that the error term is difficult to apply, so the degree of the polynomial needed for the desired accuracy is generally not known until computations have been performed. A common practice is to compute the results given from various polynomials until appropriate agreement is obtained, as was done in the previous illustration. However, the work done in calculating the approximation by the second polynomial does not lessen the work needed to calculate the third approximation, nor is the fourth approximation easier to obtain once the third approximation is known and so on. We will now derive these approximating polynomials in a manner that uses the previous calculations to greater advantage.

Definition 3.4 Let f be a function defined at $x_0, x_1, x_2, \dots, x_n$ and suppose that $m_1, m_2, \dots, m_k$ are k distinct integers, with $0 \leq m_i \leq n$ for each i . The Lagrange polynomial that agrees with $f(x)$ at the k points $x_{m_1}, x_{m_2}, \dots, x_{m_k}$ is denoted $P_{m_1, m_2, \dots, m_k}(x)$. ■

Example 1 Suppose that $x_0 = 1$, $x_1 = 2$, $x_2 = 3$, $x_3 = 4$, $x_4 = 6$, and $f(x) = e^x$. Determine the interpolating polynomial denoted $P_{1,2,4}(x)$ and use this polynomial to approximate $f(5)$.

Solution This is the Lagrange polynomial that agrees with $f(x)$ at $x_1 = 2$, $x_2 = 3$, and $x_4 = 6$. Hence,

$$P_{1,2,4}(x) = \frac{(x-3)(x-6)}{(2-3)(2-6)}e^2 + \frac{(x-2)(x-6)}{(3-2)(3-6)}e^3 + \frac{(x-2)(x-3)}{(6-2)(6-3)}e^6.$$

So,

$$\begin{aligned} f(5) \approx P(5) &= \frac{(5-3)(5-6)}{(2-3)(2-6)}e^2 + \frac{(5-2)(5-6)}{(3-2)(3-6)}e^3 + \frac{(5-2)(5-3)}{(6-2)(6-3)}e^6 \\ &= -\frac{1}{2}e^2 + e^3 + \frac{1}{2}e^6 \approx 218.105. \end{aligned}$$

The next result describes a method for recursively generating Lagrange polynomial approximations.

Theorem 3.5 Let f be defined at $x_0, x_1, \dots, x_k$ and let x_j and x_i be two distinct numbers in this set. Then

$$P(x) = \frac{(x-x_j)P_{0,1,\dots,j-1,j+1,\dots,k}(x) - (x-x_i)P_{0,1,\dots,i-1,i+1,\dots,k}(x)}{(x_i-x_j)}$$

is the k th Lagrange polynomial that interpolates f at the $k+1$ points $x_0, x_1, \dots, x_k$.

Proof For ease of notation, let $Q \equiv P_{0,1,\dots,i-1,i+1,\dots,k}$ and $\hat{Q} \equiv P_{0,1,\dots,j-1,j+1,\dots,k}$. Since $Q(x)$ and $\hat{Q}(x)$ are polynomials of degree $k-1$ or less, $P(x)$ is of degree at most k .

First, note that $\hat{Q}(x_i) = f(x_i)$ implies that

$$P(x_i) = \frac{(x_i-x_j)\hat{Q}(x_i) - (x_i-x_i)Q(x_i)}{x_i-x_j} = \frac{(x_i-x_j)}{(x_i-x_j)}f(x_i) = f(x_i).$$

Similarly, since $Q(x_j) = f(x_j)$, we have $P(x_j) = f(x_j)$.

In addition, if $0 \leq r \leq k$ and r is neither i nor j , then $Q(x_r) = \hat{Q}(x_r) = f(x_r)$. So,

$$P(x_r) = \frac{(x_r-x_j)\hat{Q}(x_r) - (x_r-x_i)Q(x_r)}{x_i-x_j} = \frac{(x_i-x_j)}{(x_i-x_j)}f(x_r) = f(x_r).$$

But, by definition, $P_{0,1,\dots,k}(x)$ is the unique polynomial of degree at most k that agrees with f at $x_0, x_1, \dots, x_k$. Thus, $P \equiv P_{0,1,\dots,k}$.

Theorem 3.5 implies that the interpolating polynomials can be generated recursively. For example, we have

$$\begin{aligned} P_{0,1} &= \frac{1}{x_1-x_0}[(x-x_0)P_1 + (x-x_1)P_0], & P_{1,2} &= \frac{1}{x_2-x_1}[(x-x_1)P_2 + (x-x_2)P_1], \\ P_{0,1,2} &= \frac{1}{x_2-x_0}[(x-x_0)P_{1,2} + (x-x_2)P_{0,1}], \end{aligned}$$

and so on. They are generated in the manner shown in Table 3.3, where each row is completed before the succeeding rows are begun.

Table 3.3

x_0	P_0				
x_1	P_1	$P_{0,1}$			
x_2	P_2	$P_{1,2}$	$P_{0,1,2}$		
x_3	P_3	$P_{2,3}$	$P_{1,2,3}$	$P_{0,1,2,3}$	
x_4	P_4	$P_{3,4}$	$P_{2,3,4}$	$P_{1,2,3,4}$	$P_{0,1,2,3,4}$

The procedure that uses the result of Theorem 3.5 to recursively generate interpolating polynomial approximations is called **Neville's method**. The P notation used in Table 3.3 is cumbersome because of the number of subscripts used to represent the entries. Note, however, that as an array is being constructed, only two subscripts are needed. Proceeding down the table corresponds to using consecutive points x_i with larger i , and proceeding to the right corresponds to increasing the degree of the interpolating polynomial. Since the points appear consecutively in each entry, we need to describe only a starting point and the number of additional points used in constructing the approximation.

To avoid the multiple subscripts, we let $Q_{i,j}(x)$, for $0 \leq j \leq i$, denote the interpolating polynomial of degree j on the $(j+1)$ numbers $x_{i-j}, x_{i-j+1}, \dots, x_{i-1}, x_i$; that is,

$$Q_{i,j} = P_{i-j, i-j+1, \dots, i-1, i}.$$

Using this notation provides the Q notation array in Table 3.4.

Table 3.4

x_0	$P_0 = Q_{0,0}$				
x_1	$P_1 = Q_{1,0}$	$P_{0,1} = Q_{1,1}$			
x_2	$P_2 = Q_{2,0}$	$P_{1,2} = Q_{2,1}$	$P_{0,1,2} = Q_{2,2}$		
x_3	$P_3 = Q_{3,0}$	$P_{2,3} = Q_{3,1}$	$P_{1,2,3} = Q_{3,2}$	$P_{0,1,2,3} = Q_{3,3}$	
x_4	$P_4 = Q_{4,0}$	$P_{3,4} = Q_{4,1}$	$P_{2,3,4} = Q_{4,2}$	$P_{1,2,3,4} = Q_{4,3}$	$P_{0,1,2,3,4} = Q_{4,4}$

Example 2 Values of various interpolating polynomials at $x = 1.5$ were obtained in the illustration at the beginning of this section using the data shown in Table 3.5. Apply Neville's method to the data by constructing a recursive table of the form shown in Table 3.4.

Table 3.5

x	$f(x)$
1.0	0.7651977
1.3	0.6200860
1.6	0.4554022
1.9	0.2818186
2.2	0.1103623

Solution Let $x_0 = 1.0$, $x_1 = 1.3$, $x_2 = 1.6$, $x_3 = 1.9$, and $x_4 = 2.2$, then $Q_{0,0} = f(1.0)$, $Q_{1,0} = f(1.3)$, $Q_{2,0} = f(1.6)$, $Q_{3,0} = f(1.9)$, and $Q_{4,0} = f(2.2)$. These are the five polynomials of degree zero (constants) that approximate $f(1.5)$ and are the same as data given in Table 3.5.

Calculating the first-degree approximation $Q_{1,1}(1.5)$ gives

$$\begin{aligned}
 Q_{1,1}(1.5) &= \frac{(x - x_0)Q_{1,0} - (x - x_1)Q_{0,0}}{x_1 - x_0} \\
 &= \frac{(1.5 - 1.0)Q_{1,0} - (1.5 - 1.3)Q_{0,0}}{1.3 - 1.0} \\
 &= \frac{0.5(0.6200860) - 0.2(0.7651977)}{0.3} = 0.5233449.
 \end{aligned}$$

Similarly,

$$\begin{aligned}
 Q_{2,1}(1.5) &= \frac{(1.5 - 1.3)(0.4554022) - (1.5 - 1.6)(0.6200860)}{1.6 - 1.3} = 0.5102968, \\
 Q_{3,1}(1.5) &= 0.5132634, \quad \text{and} \quad Q_{4,1}(1.5) = 0.5104270.
 \end{aligned}$$

Eric Harold Neville (1889–1961) gave this modification of the Lagrange formula in a paper published in 1932. [N]

The best linear approximation is expected to be $Q_{2,1}$ because 1.5 is between $x_1 = 1.3$ and $x_2 = 1.6$.

In a similar manner, approximations using higher-degree polynomials are given by

$$Q_{2,2}(1.5) = \frac{(1.5 - 1.0)(0.5102968) - (1.5 - 1.6)(0.5233449)}{1.6 - 1.0} = 0.5124715,$$

$$Q_{3,2}(1.5) = 0.5112857, \quad \text{and} \quad Q_{4,2}(1.5) = 0.5137361.$$

The higher-degree approximations are generated in a similar manner and are shown in Table 3.6. ■

Table 3.6

1.0	0.7651977				
1.3	0.6200860	0.5233449			
1.6	0.4554022	0.5102968	0.5124715		
1.9	0.2818186	0.5132634	0.5112857	0.5118127	
2.2	0.1103623	0.5104270	0.5137361	0.5118302	0.5118200

If the latest approximation, $Q_{4,4}$, was not sufficiently accurate, another node, x_5 , could be selected and another row added to the table:

$$x_5 \quad Q_{5,0} \quad Q_{5,1} \quad Q_{5,2} \quad Q_{5,3} \quad Q_{5,4} \quad Q_{5,5}.$$

Then $Q_{4,4}$, $Q_{5,4}$, and $Q_{5,5}$ could be compared to determine further accuracy.

The function in Example 2 is the Bessel function of the first kind of order zero, whose value at 2.5 is -0.0483838 , and the next row of approximations to $f(1.5)$ is

$$2.5 \quad -0.0483838 \quad 0.4807699 \quad 0.5301984 \quad 0.5119070 \quad 0.5118430 \quad 0.5118277.$$

The final new entry, 0.5118277, is correct to all seven decimal places.

Example 3

Table 3.7 lists the values of $f(x) = \ln x$ accurate to the places given. Use Neville's method and four-digit rounding arithmetic to approximate $f(2.1) = \ln 2.1$ by completing the Neville table.

Table 3.7

i	x_i	$\ln x_i$
0	2.0	0.6931
1	2.2	0.7885
2	2.3	0.8329

Solution Because $x - x_0 = 0.1$, $x - x_1 = -0.1$, and $x - x_2 = -0.2$ and we are given $Q_{0,0} = 0.6931$, $Q_{1,0} = 0.7885$, and $Q_{2,0} = 0.8329$, we have

$$Q_{1,1} = \frac{1}{0.2} [(0.1)0.7885 - (-0.1)0.6931] = \frac{0.1482}{0.2} = 0.7410$$

and

$$Q_{2,1} = \frac{1}{0.1} [(-0.1)0.8329 - (-0.2)0.7885] = \frac{0.07441}{0.1} = 0.7441.$$

The final approximation we can obtain from this data is

$$Q_{2,1} = \frac{1}{0.3} [(0.1)0.7441 - (-0.2)0.7410] = \frac{0.2276}{0.3} = 0.7420.$$

These values are shown in Table 3.8. ■

Table 3.8

i	x_i	$x - x_i$	Q_{i0}	Q_{i1}	Q_{i2}
0	2.0	0.1	0.6931		
1	2.2	-0.1	0.7885	0.7410	
2	2.3	-0.2	0.8329	0.7441	0.7420

In the preceding example, we have $f(2.1) = \ln 2.1 = 0.7419$ to four decimal places, so the absolute error is

$$|f(2.1) - P_2(2.1)| = |0.7419 - 0.7420| = 10^{-4}.$$

However, $f'(x) = 1/x$, $f''(x) = -1/x^2$, and $f'''(x) = 2/x^3$, so the Lagrange error formula (3.3) in Theorem 3.3 gives the error bound

$$\begin{aligned} |f(2.1) - P_2(2.1)| &= \left| \frac{f'''(\xi(2.1))}{3!} (x - x_0)(x - x_1)(x - x_2) \right| \\ &= \left| \frac{1}{3(\xi(2.1))^3} (0.1)(-0.1)(-0.2) \right| \leq \frac{0.002}{3(2)^3} = 8.\bar{3} \times 10^{-5}. \end{aligned}$$

Notice that the actual error, 10^{-4} , exceeds the error bound, $8.\bar{3} \times 10^{-5}$. This apparent contradiction is a consequence of finite-digit computations. We used four-digit rounding arithmetic, and the Lagrange error formula (3.3) assumes infinite-digit arithmetic. This caused our actual errors to exceed the theoretical error estimate.

- Remember: You cannot expect more accuracy than the arithmetic provides.

Algorithm 3.1 constructs the entries in Neville's method by rows.



Neville's Iterated Interpolation

To evaluate the interpolating polynomial P on the $n + 1$ distinct numbers $x_0, \dots, x_n$ at the number x for the function f :

INPUT numbers $x, x_0, x_1, \dots, x_n$; values $f(x_0), f(x_1), \dots, f(x_n)$ as the first column $Q_{0,0}, Q_{1,0}, \dots, Q_{n,0}$ of Q .

OUTPUT the table Q with $P(x) = Q_{n,n}$.

Step 1 For $i = 1, 2, \dots, n$
for $j = 1, 2, \dots, i$

$$\text{set } Q_{i,j} = \frac{(x - x_{i-j})Q_{i,j-1} - (x - x_i)Q_{i-1,j-1}}{x_i - x_{i-j}}.$$

Step 2 OUTPUT (Q);
STOP.

EXERCISE SET 3.2

- Use Neville's method to obtain the approximations for Lagrange interpolating polynomials of degrees one, two, and three to approximate each of the following:
 - $f(8.4)$ if $f(8.1) = 16.94410$, $f(8.3) = 17.56492$, $f(8.6) = 18.50515$, $f(8.7) = 18.82091$
 - $f(-\frac{1}{3})$ if $f(-0.75) = -0.07181250$, $f(-0.5) = -0.02475000$, $f(-0.25) = 0.33493750$, $f(0) = 1.10100000$
 - $f(0.25)$ if $f(0.1) = 0.62049958$, $f(0.2) = -0.28398668$, $f(0.3) = 0.00660095$, $f(0.4) = 0.24842440$
 - $f(0.9)$ if $f(0.6) = -0.17694460$, $f(0.7) = 0.01375227$, $f(0.8) = 0.22363362$, $f(1.0) = 0.65809197$

2. Use Neville's method to obtain the approximations for Lagrange interpolating polynomials of degrees one, two, and three to approximate each of the following:
 - a. $f(0.43)$ if $f(0) = 1$, $f(0.25) = 1.64872$, $f(0.5) = 2.71828$, $f(0.75) = 4.48169$
 - b. $f(0)$ if $f(-0.5) = 1.93750$, $f(-0.25) = 1.33203$, $f(0.25) = 0.800781$, $f(0.5) = 0.687500$
 - c. $f(0.18)$ if $f(0.1) = -0.29004986$, $f(0.2) = -0.56079734$, $f(0.3) = -0.81401972$, $f(0.4) = -1.0526302$
 - d. $f(0.25)$ if $f(-1) = 0.86199480$, $f(-0.5) = 0.95802009$, $f(0) = 1.0986123$, $f(0.5) = 1.2943767$
3. Use Neville's method to approximate $\sqrt{3}$ with the following functions and values.
 - a. $f(x) = 3^x$ and the values $x_0 = -2$, $x_1 = -1$, $x_2 = 0$, $x_3 = 1$, and $x_4 = 2$.
 - b. $f(x) = \sqrt{x}$ and the values $x_0 = 0$, $x_1 = 1$, $x_2 = 2$, $x_3 = 4$, and $x_4 = 5$.
 - c. Compare the accuracy of the approximation in parts (a) and (b).
4. Let $P_3(x)$ be the interpolating polynomial for the data $(0, 0)$, $(0.5, y)$, $(1, 3)$, and $(2, 2)$. Find y if the coefficient of x^3 in $P_3(x)$ is 6.
5. Neville's method is used to approximate $f(0.4)$, giving the following table.

$x_0 = 0$	$P_0 = 1$				
$x_1 = 0.25$	$P_1 = 2$	$P_{0,1} = 2.6$			
$x_2 = 0.5$	P_2	$P_{1,2}$	$P_{0,1,2}$		
$x_3 = 0.75$	$P_3 = 8$	$P_{2,3} = 2.4$	$P_{1,2,3} = 2.96$	$P_{0,1,2,3} = 3.016$	

Determine $P_2 = f(0.5)$.

6. Neville's method is used to approximate $f(0.5)$, giving the following table.

$x_0 = 0$	$P_0 = 0$			
$x_1 = 0.4$	$P_1 = 2.8$	$P_{0,1} = 3.5$		
$x_2 = 0.7$	P_2	$P_{1,2}$	$P_{0,1,2} = \frac{27}{7}$	

Determine $P_2 = f(0.7)$.

7. Suppose $x_j = j$, for $j = 0, 1, 2, 3$, and it is known that

$$P_{0,1}(x) = 2x + 1, \quad P_{0,2}(x) = x + 1, \quad \text{and} \quad P_{1,2,3}(2.5) = 3.$$

Find $P_{0,1,2,3}(2.5)$.

8. Suppose $x_j = j$, for $j = 0, 1, 2, 3$, and it is known that

$$P_{0,1}(x) = x + 1, \quad P_{1,2}(x) = 3x - 1, \quad \text{and} \quad P_{1,2,3}(1.5) = 4.$$

Find $P_{0,1,2,3}(1.5)$.

9. Neville's Algorithm is used to approximate $f(0)$ using $f(-2)$, $f(-1)$, $f(1)$, and $f(2)$. Suppose $f(-1)$ was understated by 2 and $f(1)$ was overstated by 3. Determine the error in the original calculation of the value of the interpolating polynomial to approximate $f(0)$.
10. Neville's Algorithm is used to approximate $f(0)$ using $f(-2)$, $f(-1)$, $f(1)$, and $f(2)$. Suppose $f(-1)$ was overstated by 2 and $f(1)$ was understated by 3. Determine the error in the original calculation of the value of the interpolating polynomial to approximate $f(0)$.

THEORETICAL EXERCISES

11. Construct a sequence of interpolating values y_n to $f(1 + \sqrt{10})$, where $f(x) = (1 + x^2)^{-1}$ for $-5 \leq x \leq 5$, as follows: For each $n = 1, 2, \dots, 10$, let $h = 10/n$ and $y_n = P_n(1 + \sqrt{10})$, where $P_n(x)$ is the interpolating polynomial for $f(x)$ at the nodes $x_0^{(n)}, x_1^{(n)}, \dots, x_n^{(n)}$ and $x_j^{(n)} = -5 + jh$, for each $j = 0, 1, 2, \dots, n$. Does the sequence $\{y_n\}$ appear to converge to $f(1 + \sqrt{10})$?

Inverse Interpolation Suppose $f \in C^1[a, b]$, $f'(x) \neq 0$ on $[a, b]$ and f has one zero p in $[a, b]$. Let $x_0, \dots, x_n$ be $n + 1$ distinct numbers in $[a, b]$ with $f(x_k) = y_k$, for each $k = 0, 1, \dots, n$. To approximate p , construct the interpolating polynomial of degree n on the nodes $y_0, \dots, y_n$ for f^{-1} . Since $y_k = f(x_k)$ and $0 = f(p)$, it follows that $f^{-1}(y_k) = x_k$ and $p = f^{-1}(0)$. Using iterated interpolation to approximate $f^{-1}(0)$ is called *iterated inverse interpolation*.

12. Use iterated inverse interpolation to find an approximation to the solution of $x - e^{-x} = 0$, using the data

x	0.3	0.4	0.5	0.6
e^{-x}	0.740818	0.670320	0.606531	0.548812

13. Construct an algorithm that can be used for inverse interpolation.

DISCUSSION QUESTIONS

1. Reliability: What is it, and how is it measured? Read the paper found at <http://www.slideshare.net/analisedecurvas/reliability-what-is-it-and-how-is-it-measured>. Summarize your findings and describe how Neville's method is applied to measure the error.
2. Can Neville's method be used to obtain the interpolation polynomial at a general point as opposed to a specific point?

3.3 Divided Differences

Iterated interpolation was used in the previous section to generate successively higher-degree polynomial approximations at a specific point. Divided-difference methods introduced in this section are used to successively generate the polynomials themselves.

Divided Differences

Suppose that $P_n(x)$ is the n th interpolating polynomial that agrees with the function f at the distinct numbers $x_0, x_1, \dots, x_n$. Although this polynomial is unique, there are alternate algebraic representations that are useful in certain situations. The divided differences of f with respect to $x_0, x_1, \dots, x_n$ are used to express $P_n(x)$ in the form

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0) \cdots (x - x_{n-1}), \quad (3.5)$$

for appropriate constants $a_0, a_1, \dots, a_n$. To determine the first of these constants, a_0 , note that if $P_n(x)$ is written in the form of Eq. (3.5), then evaluating $P_n(x)$ at x_0 leaves only the constant term a_0 ; that is,

$$a_0 = P_n(x_0) = f(x_0).$$

Similarly, when $P(x)$ is evaluated at x_1 , the only nonzero terms in the evaluation of $P_n(x_1)$ are the constant and linear terms,

$$f(x_0) + a_1(x_1 - x_0) = P_n(x_1) = f(x_1);$$

so,

$$a_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}. \quad (3.6)$$

We now introduce the divided-difference notation, which is related to Aitken's Δ^2 notation used in Section 2.5. The *zeroth divided difference* of the function f with respect

As in so many areas, Isaac Newton is prominent in the study of difference equations. He developed interpolation formulas as early as 1675, using his Δ notation in tables of differences. He took a very general approach to the difference formulas, so explicit examples that he produced, including Lagrange's formulas, are often known by other names.

to x_i , denoted $f[x_i]$, is simply the value of f at x_i :

$$f[x_i] = f(x_i). \quad (3.7)$$

The remaining divided differences are defined recursively; the *first divided difference* of f with respect to x_i and x_{i+1} is denoted $f[x_i, x_{i+1}]$ and defined as

$$f[x_i, x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}. \quad (3.8)$$

The *second divided difference*, $f[x_i, x_{i+1}, x_{i+2}]$, is defined as

$$f[x_i, x_{i+1}, x_{i+2}] = \frac{f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]}{x_{i+2} - x_i}.$$

Similarly, after the $(k - 1)$ st divided differences,

$$f[x_i, x_{i+1}, x_{i+2}, \dots, x_{i+k-1}] \quad \text{and} \quad f[x_{i+1}, x_{i+2}, \dots, x_{i+k-1}, x_{i+k}],$$

have been determined, the **k th divided difference** relative to $x_i, x_{i+1}, x_{i+2}, \dots, x_{i+k}$ is

$$f[x_i, x_{i+1}, \dots, x_{i+k-1}, x_{i+k}] = \frac{f[x_{i+1}, x_{i+2}, \dots, x_{i+k}] - f[x_i, x_{i+1}, \dots, x_{i+k-1}]}{x_{i+k} - x_i}. \quad (3.9)$$

The process ends with the single *n th divided difference*,

$$f[x_0, x_1, \dots, x_n] = \frac{f[x_1, x_2, \dots, x_n] - f[x_0, x_1, \dots, x_{n-1}]}{x_n - x_0}.$$

Because of Eq. (3.6), we can write $a_1 = f[x_0, x_1]$, just as a_0 can be expressed as $a_0 = f(x_0) = f[x_0]$. Hence, the interpolating polynomial in Eq. (3.5) is

$$\begin{aligned} P_n(x) = & f[x_0] + f[x_0, x_1](x - x_0) + a_2(x - x_0)(x - x_1) \\ & + \dots + a_n(x - x_0)(x - x_1) \dots (x - x_{n-1}). \end{aligned}$$

As might be expected from the evaluation of a_0 and a_1 , the required constants are

$$a_k = f[x_0, x_1, x_2, \dots, x_k],$$

for each $k = 0, 1, \dots, n$. So, $P_n(x)$ can be rewritten in a form called Newton's Divided-Difference:

$$P_n(x) = f[x_0] + \sum_{k=1}^n f[x_0, x_1, \dots, x_k](x - x_0) \dots (x - x_{k-1}). \quad (3.10)$$

The value of $f[x_0, x_1, \dots, x_k]$ is independent of the order of the numbers $x_0, x_1, \dots, x_k$, as is shown in Exercise 23.

The generation of the divided differences is outlined in Table 3.9. Two fourth and one fifth difference can also be determined from these data.

Table 3.9

x	$f(x)$	First divided differences	Second divided differences	Third divided differences
x_0	$f[x_0]$			
		$f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0}$		
x_1	$f[x_1]$		$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$	
		$f[x_1, x_2] = \frac{f[x_2] - f[x_1]}{x_2 - x_1}$		$f[x_0, x_1, x_2, x_3] = \frac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0}$
x_2	$f[x_2]$		$f[x_1, x_2, x_3] = \frac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1}$	
		$f[x_2, x_3] = \frac{f[x_3] - f[x_2]}{x_3 - x_2}$		$f[x_1, x_2, x_3, x_4] = \frac{f[x_2, x_3, x_4] - f[x_1, x_2, x_3]}{x_4 - x_1}$
x_3	$f[x_3]$		$f[x_2, x_3, x_4] = \frac{f[x_3, x_4] - f[x_2, x_3]}{x_4 - x_2}$	
		$f[x_3, x_4] = \frac{f[x_4] - f[x_3]}{x_4 - x_3}$		$f[x_2, x_3, x_4, x_5] = \frac{f[x_3, x_4, x_5] - f[x_2, x_3, x_4]}{x_5 - x_2}$
x_4	$f[x_4]$		$f[x_3, x_4, x_5] = \frac{f[x_4, x_5] - f[x_3, x_4]}{x_5 - x_3}$	
		$f[x_4, x_5] = \frac{f[x_5] - f[x_4]}{x_5 - x_4}$		
x_5	$f[x_5]$			

ALGORITHM
3.2

Newton's Divided-Difference Formula

To obtain the divided-difference coefficients of the interpolatory polynomial P on the $(n+1)$ distinct numbers $x_0, x_1, \dots, x_n$ for the function f :

INPUT numbers $x_0, x_1, \dots, x_n$; values $f(x_0), f(x_1), \dots, f(x_n)$ as $F_{0,0}, F_{1,0}, \dots, F_{n,0}$.

OUTPUT the numbers $F_{0,0}, F_{1,1}, \dots, F_{n,n}$ where

$$P_n(x) = F_{0,0} + \sum_{i=1}^n F_{i,i} \prod_{j=0}^{i-1} (x - x_j). \quad (F_{i,i} \text{ is } f[x_0, x_1, \dots, x_i].)$$

Step 1 For $i = 1, 2, \dots, n$

For $j = 1, 2, \dots, i$

$$\text{set } F_{i,j} = \frac{F_{i,j-1} - F_{i-1,j-1}}{x_i - x_{i-j}}. \quad (F_{i,j} = f[x_{i-j}, \dots, x_i].)$$

Step 2 OUTPUT $(F_{0,0}, F_{1,1}, \dots, F_{n,n})$;

STOP.

The form of the output in Algorithm 3.2 can be modified to produce all the divided differences, as shown in Example 1.

Example 1

Complete the divided difference table for the data used in Example 1 of Section 3.2, and reproduced in Table 3.10 and construct the interpolating polynomial that uses all these data.

Table 3.10

x	$f(x)$
1.0	0.7651977
1.3	0.6200860
1.6	0.4554022
1.9	0.2818186
2.2	0.1103623

Solution The first divided difference involving x_0 and x_1 is

$$f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0} = \frac{0.6200860 - 0.7651977}{1.3 - 1.0} = -0.4837057.$$

The remaining first divided differences are found in a similar manner and are shown in the fourth column in Table 3.11.

Table 3.11

i	x_i	$f[x_i]$	$f[x_{i-1}, x_i]$	$f[x_{i-2}, x_{i-1}, x_i]$	$f[x_{i-3}, \dots, x_i]$	$f[x_{i-4}, \dots, x_i]$
0	1.0	0.7651977				
1	1.3	0.6200860	-0.4837057			
2	1.6	0.4554022	-0.5489460	-0.1087339	0.0658784	
3	1.9	0.2818186	-0.5786120	-0.0494433	0.0680685	0.0018251
4	2.2	0.1103623	-0.5715210	0.0118183		

The second divided difference involving x_0, x_1 , and x_2 is

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = \frac{-0.5489460 - (-0.4837057)}{1.6 - 1.0} = -0.1087339.$$

The remaining second divided differences are shown in the fifth column of Table 3.11. The third divided difference involving x_0, x_1, x_2 , and x_3 and the fourth divided difference involving all the data points are, respectively,

$$\begin{aligned} f[x_0, x_1, x_2, x_3] &= \frac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0} = \frac{-0.0494433 - (-0.1087339)}{1.9 - 1.0} \\ &= 0.0658784, \end{aligned}$$

and

$$\begin{aligned} f[x_0, x_1, x_2, x_3, x_4] &= \frac{f[x_1, x_2, x_3, x_4] - f[x_0, x_1, x_2, x_3]}{x_4 - x_0} = \frac{0.0680685 - 0.0658784}{2.2 - 1.0} \\ &= 0.0018251. \end{aligned}$$

All the entries are given in Table 3.11.

The coefficients of the Newton forward-divided-difference form of the interpolating polynomial are along the diagonal in the table. This polynomial is

$$\begin{aligned} P_4(x) &= 0.7651977 - 0.4837057(x - 1.0) - 0.1087339(x - 1.0)(x - 1.3) \\ &\quad + 0.0658784(x - 1.0)(x - 1.3)(x - 1.6) \\ &\quad + 0.0018251(x - 1.0)(x - 1.3)(x - 1.6)(x - 1.9). \end{aligned}$$

Notice that the value $P_4(1.5) = 0.5118200$ agrees with the result in Table 3.6 for Example 2 of Section 3.2, as it must because the polynomials are the same. ■

The Mean Value Theorem 1.8 applied to Eq. (3.8) when $i = 0$,

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0},$$

implies that when f' exists, $f[x_0, x_1] = f'(\xi)$ for some number ξ between x_0 and x_1 . The following theorem generalizes this result.

Theorem 3.6 Suppose that $f \in C^n[a, b]$ and $x_0, x_1, \dots, x_n$ are distinct numbers in $[a, b]$. Then a number ξ exists in (a, b) with

$$f[x_0, x_1, \dots, x_n] = \frac{f^{(n)}(\xi)}{n!}.$$

Proof Let

$$g(x) = f(x) - P_n(x).$$

Since $f(x_i) = P_n(x_i)$ for each $i = 0, 1, \dots, n$, the function g has $n + 1$ distinct zeros in $[a, b]$. Generalized Rolle's Theorem 1.10 implies that a number ξ in (a, b) exists with $g^{(n)}(\xi) = 0$, so

$$0 = f^{(n)}(\xi) - P_n^{(n)}(\xi).$$

Since $P_n(x)$ is a polynomial of degree n whose leading coefficient is $f[x_0, x_1, \dots, x_n]$,

$$P_n^{(n)}(x) = n!f[x_0, x_1, \dots, x_n],$$

for all values of x . As a consequence,

$$f[x_0, x_1, \dots, x_n] = \frac{f^{(n)}(\xi)}{n!}. \quad \blacksquare$$

Newton's divided-difference formula can be expressed in a simplified form when the nodes are arranged consecutively with equal spacing. In this case, we introduce the notation $h = x_{i+1} - x_i$, for each $i = 0, 1, \dots, n - 1$ and let $x = x_0 + sh$. Then the difference $x - x_i$ is $x - x_i = (s - i)h$. So, Eq. (3.10) becomes

$$\begin{aligned} P_n(x) &= P_n(x_0 + sh) = f[x_0] + shf[x_0, x_1] + s(s-1)h^2f[x_0, x_1, x_2] \\ &\quad + \dots + s(s-1)\dots(s-n+1)h^nf[x_0, x_1, \dots, x_n] \\ &= f[x_0] + \sum_{k=1}^n s(s-1)\dots(s-k+1)h^kf[x_0, x_1, \dots, x_k]. \end{aligned}$$

Using binomial coefficient notation,

$$\binom{s}{k} = \frac{s(s-1)\dots(s-k+1)}{k!},$$

we can express $P_n(x)$ compactly as

$$P_n(x) = P_n(x_0 + sh) = f[x_0] + \sum_{k=1}^n \binom{s}{k} k!h^k f[x_0, x_1, \dots, x_k]. \quad (3.11)$$

Forward Differences

The **Newton forward-difference formula** is constructed by making use of the forward-difference notation Δ introduced in Aitken's Δ^2 method. With this notation,

$$\begin{aligned} f[x_0, x_1] &= \frac{f(x_1) - f(x_0)}{x_1 - x_0} = \frac{1}{h}(f(x_1) - f(x_0)) = \frac{1}{h}\Delta f(x_0) \\ f[x_0, x_1, x_2] &= \frac{1}{2h} \left[\frac{\Delta f(x_1) - \Delta f(x_0)}{h} \right] = \frac{1}{2h^2}\Delta^2 f(x_0), \end{aligned}$$

and, in general,

$$f[x_0, x_1, \dots, x_k] = \frac{1}{k!h^k}\Delta^k f(x_0).$$

Since $f[x_0] = f(x_0)$, Eq. (3.11) has the following form.

Newton Forward-Difference Formula

$$P_n(x) = f(x_0) + \sum_{k=1}^n \binom{s}{k} \Delta^k f(x_0) \quad (3.12)$$

Backward Differences

If the interpolating nodes are reordered from last to first as $x_n, x_{n-1}, \dots, x_0$, we can write the interpolatory formula as

$$P_n(x) = f[x_n] + f[x_n, x_{n-1}](x - x_n) + f[x_n, x_{n-1}, x_{n-2}](x - x_n)(x - x_{n-1}) \\ + \dots + f[x_n, \dots, x_0](x - x_n)(x - x_{n-1}) \dots (x - x_1).$$

If, in addition, the nodes are equally spaced with $x = x_n + sh$ and $x = x_i + (s + n - i)h$, then

$$P_n(x) = P_n(x_n + sh) \\ = f[x_n] + shf[x_n, x_{n-1}] + s(s+1)h^2 f[x_n, x_{n-1}, x_{n-2}] + \dots \\ + s(s+1) \dots (s+n-1)h^n f[x_n, \dots, x_0].$$

This is used to derive a commonly applied formula known as the **Newton backward-difference formula**. To discuss this formula, we need the following definition.

Definition 3.7 Given the sequence $\{p_n\}_{n=0}^\infty$, define the backward-difference ∇p_n (read *nabla p_n*) by

$$\nabla p_n = p_n - p_{n-1}, \quad \text{for } n \geq 1.$$

Higher powers are defined recursively by

$$\nabla^k p_n = \nabla(\nabla^{k-1} p_n), \quad \text{for } k \geq 2. \quad \blacksquare$$

Definition 3.7 implies that

$$f[x_n, x_{n-1}] = \frac{1}{h} \nabla f(x_n), \quad f[x_n, x_{n-1}, x_{n-2}] = \frac{1}{2h^2} \nabla^2 f(x_n),$$

and, in general,

$$f[x_n, x_{n-1}, \dots, x_{n-k}] = \frac{1}{k!h^k} \nabla^k f(x_n).$$

Consequently,

$$P_n(x) = f[x_n] + s \nabla f(x_n) + \frac{s(s+1)}{2} \nabla^2 f(x_n) + \dots + \frac{s(s+1) \dots (s+n-1)}{n!} \nabla^n f(x_n).$$

If we extend the binomial coefficient notation to include all real values of s by letting

$$\binom{-s}{k} = \frac{-s(-s-1) \dots (-s-k+1)}{k!} = (-1)^k \frac{s(s+1) \dots (s+k-1)}{k!},$$

then

$$P_n(x) = f[x_n] + (-1)^1 \binom{-s}{1} \nabla f(x_n) + (-1)^2 \binom{-s}{2} \nabla^2 f(x_n) + \dots + (-1)^n \binom{-s}{n} \nabla^n f(x_n).$$

This gives the following result.

Newton Backward-Difference Formula

$$P_n(x) = f[x_n] + \sum_{k=1}^n (-1)^k \binom{-s}{k} \nabla^k f(x_n) \quad (3.13)$$

Illustration The divided-difference Table 3.12 corresponds to the data in Example 1.

Table 3.12

		First divided differences	Second divided differences	Third divided differences	Fourth divided differences
1.0	<u>0.7651977</u>				
		<u>-0.4837057</u>			
1.3	0.6200860		<u>-0.1087339</u>		
		-0.5489460		<u>0.0658784</u>	
1.6	0.4554022		-0.0494433		<u>0.0018251</u>
		-0.5786120		<u>0.0680685</u>	
1.9	0.2818186		<u>0.0118183</u>		
		<u>-0.5715210</u>			
2.2	<u>0.1103623</u>				

Only one interpolating polynomial of degree at most four uses these five data points, but we will organize the data points to obtain the best interpolation approximations of degrees one, two, and three. This will give us a sense of accuracy of the fourth-degree approximation for the given value of x .

If an approximation to $f(1.1)$ is required, the reasonable choice for the nodes would be $x_0 = 1.0$, $x_1 = 1.3$, $x_2 = 1.6$, $x_3 = 1.9$, and $x_4 = 2.2$, since this choice makes the earliest possible use of the data points closest to $x = 1.1$ and also makes use of the fourth divided difference. This implies that $h = 0.3$ and $s = \frac{1}{3}$, so the Newton forward-divided-difference formula is used with the divided differences that have a *solid* underline (—) in Table 3.12:

$$\begin{aligned}
 P_4(1.1) &= P_4(1.0 + \frac{1}{3}(0.3)) \\
 &= 0.7651977 + \frac{1}{3}(0.3)(-0.4837057) + \frac{1}{3}\left(-\frac{2}{3}\right)(0.3)^2(-0.1087339) \\
 &\quad + \frac{1}{3}\left(-\frac{2}{3}\right)\left(-\frac{5}{3}\right)(0.3)^3(0.0658784) \\
 &\quad + \frac{1}{3}\left(-\frac{2}{3}\right)\left(-\frac{5}{3}\right)\left(-\frac{8}{3}\right)(0.3)^4(0.0018251) \\
 &= 0.7196460.
 \end{aligned}$$

To approximate a value when x is close to the end of the tabulated values, say, $x = 2.0$, we would again like to make the earliest use of the data points closest to x . This requires using the Newton backward-divided-difference formula with $s = -\frac{2}{3}$ and the divided differences in Table 3.12 that have a *wavy* underline (~~~~). Notice that the fourth divided difference

is used in both formulas:

$$\begin{aligned}
 P_4(2.0) &= P_4\left(2.2 - \frac{2}{3}(0.3)\right) \\
 &= 0.1103623 - \frac{2}{3}(0.3)(-0.5715210) - \frac{2}{3}\left(\frac{1}{3}\right)(0.3)^2(0.0118183) \\
 &\quad - \frac{2}{3}\left(\frac{1}{3}\right)\left(\frac{4}{3}\right)(0.3)^3(0.0680685) - \frac{2}{3}\left(\frac{1}{3}\right)\left(\frac{4}{3}\right)\left(\frac{7}{3}\right)(0.3)^4(0.0018251) \\
 &= 0.2238754.
 \end{aligned}$$

Centered Differences

The Newton forward- and backward-difference formulas are not appropriate for approximating $f(x)$ when x lies near the center of the table because neither will permit the highest-order difference to have x_0 close to x . A number of divided-difference formulas are available for this case, each of which has situations when it can be used to maximum advantage. These methods are known as **centered-difference formulas**. We will consider only one centered-difference formula, Stirling's method.

For the centered-difference formulas, we choose x_0 near the point being approximated and label the nodes directly below x_0 as $x_1, x_2, \dots$ and those directly above as $x_{-1}, x_{-2}, \dots$. With this convention, **Stirling's formula** is given by

$$\begin{aligned}
 P_n(x) = P_{2m+1}(x) &= f[x_0] + \frac{sh}{2}(f[x_{-1}, x_0] + f[x_0, x_1]) + s^2 h^2 f[x_{-1}, x_0, x_1] \quad (3.14) \\
 &\quad + \frac{s(s^2 - 1)h^3}{2} f[x_{-2}, x_{-1}, x_0, x_1] + f[x_{-1}, x_0, x_1, x_2]) \\
 &\quad + \cdots + s^2(s^2 - 1)(s^2 - 4) \cdots (s^2 - (m - 1)^2)h^{2m} f[x_{-m}, \dots, x_m] \\
 &\quad + \frac{s(s^2 - 1) \cdots (s^2 - m^2)h^{2m+1}}{2} (f[x_{-m-1}, \dots, x_m] + f[x_{-m}, \dots, x_{m+1}]),
 \end{aligned}$$

if $n = 2m + 1$ is odd. If $n = 2m$ is even, we use the same formula but delete the last line. The entries used for this formula are underlined in Table 3.13.

Table 3.13

x	$f(x)$	First divided differences	Second divided differences	Third divided differences	Fourth divided differences
x_{-2}	$f[x_{-2}]$				
		$f[x_{-2}, x_{-1}]$			
x_{-1}	$f[x_{-1}]$		$f[x_{-2}, x_{-1}, x_0]$		
		<u>$f[x_{-1}, x_0]$</u>		<u>$f[x_{-2}, x_{-1}, x_0, x_1]$</u>	
x_0	<u>$f[x_0]$</u>		<u>$f[x_{-1}, x_0, x_1]$</u>		<u>$f[x_{-2}, x_{-1}, x_0, x_1, x_2]$</u>
		<u>$f[x_0, x_1]$</u>		<u>$f[x_{-1}, x_0, x_1, x_2]$</u>	
x_1	$f[x_1]$		$f[x_0, x_1, x_2]$		
		$f[x_1, x_2]$			
x_2	$f[x_2]$				

Example 2 Consider the table of data given in the previous examples. Use Stirling's formula to approximate $f(1.5)$ with $x_0 = 1.6$.

Solution To apply Stirling's formula, we use the *underlined* entries in the difference Table 3.14.

James Stirling (1692–1770) published this and numerous other formulas in *Methodus Differentialis* in 1720. Techniques for accelerating the convergence of various series are included in this work.

Table 3.14

x	$f(x)$	First divided differences	Second divided differences	Third divided differences	Fourth divided differences
1.0	0.7651977				
		-0.4837057			
1.3	0.6200860		-0.1087339		
		-0.5489460		0.0658784	
1.6	0.4554022		-0.0494433		0.0018251
		-0.5786120		0.0680685	
1.9	0.2818186		0.0118183		
		-0.5715210			
2.2	0.1103623				

The formula, with $h = 0.3$, $x_0 = 1.6$, and $s = -\frac{1}{3}$, becomes

$$\begin{aligned}
 f(1.5) &\approx P_4 \left(1.6 + \left(-\frac{1}{3} \right) (0.3) \right) \\
 &= 0.4554022 + \left(-\frac{1}{3} \right) \left(\frac{0.3}{2} \right) ((-0.5489460) + (-0.5786120)) \\
 &\quad + \left(-\frac{1}{3} \right)^2 (0.3)^2 (-0.0494433) \\
 &\quad + \frac{1}{2} \left(-\frac{1}{3} \right) \left(\left(-\frac{1}{3} \right)^2 - 1 \right) (0.3)^3 (0.0658784 + 0.0680685) \\
 &\quad + \left(-\frac{1}{3} \right)^2 \left(\left(-\frac{1}{3} \right)^2 - 1 \right) (0.3)^4 (0.0018251) = 0.5118200. \quad \blacksquare
 \end{aligned}$$

Most texts on numerical analysis written before the widespread use of computers have extensive treatments of divided-difference methods. If a more comprehensive treatment of this subject is needed, the book by Hildebrand [Hild] is a particularly good reference.

EXERCISE SET 3.3

- Use Eq. (3.10) or Algorithm 3.2 to construct interpolating polynomials of degree one, two, and three for the following data. Approximate the specified value using each of the polynomials.
 - $f(8.4)$ if $f(8.1) = 16.94410$, $f(8.3) = 17.56492$, $f(8.6) = 18.50515$, $f(8.7) = 18.82091$
 - $f(0.9)$ if $f(0.6) = -0.17694460$, $f(0.7) = 0.01375227$, $f(0.8) = 0.22363362$, $f(1.0) = 0.65809197$
- Use Eq. (3.10) or Algorithm 3.2 to construct interpolating polynomials of degree one, two, and three for the following data. Approximate the specified value using each of the polynomials.
 - $f(0.43)$ if $f(0) = 1$, $f(0.25) = 1.64872$, $f(0.5) = 2.71828$, $f(0.75) = 4.48169$
 - $f(0)$ if $f(-0.5) = 1.93750$, $f(-0.25) = 1.33203$, $f(0.25) = 0.800781$, $f(0.5) = 0.687500$
- Use the Newton forward-difference formula to construct interpolating polynomials of degree one, two, and three for the following data. Approximate the specified value using each of the polynomials.
 - $f(-\frac{1}{3})$ if $f(-0.75) = -0.07181250$, $f(-0.5) = -0.02475000$, $f(-0.25) = 0.33493750$, $f(0) = 1.10100000$
 - $f(0.25)$ if $f(0.1) = -0.62049958$, $f(0.2) = -0.28398668$, $f(0.3) = 0.00660095$, $f(0.4) = 0.24842440$

4. Use the Newton forward-difference formula to construct interpolating polynomials of degree one, two, and three for the following data. Approximate the specified value using each of the polynomials.
 - a. $f(0.43)$ if $f(0) = 1$, $f(0.25) = 1.64872$, $f(0.5) = 2.71828$, $f(0.75) = 4.48169$
 - b. $f(0.18)$ if $f(0.1) = -0.29004986$, $f(0.2) = -0.56079734$, $f(0.3) = -0.81401972$, $f(0.4) = -1.0526302$
5. Use the Newton backward-difference formula to construct interpolating polynomials of degree one, two, and three for the following data. Approximate the specified value using each of the polynomials.
 - a. $f(-1/3)$ if $f(-0.75) = -0.07181250$, $f(-0.5) = -0.02475000$, $f(-0.25) = 0.33493750$, $f(0) = 1.10100000$
 - b. $f(0.25)$ if $f(0.1) = -0.62049958$, $f(0.2) = -0.28398668$, $f(0.3) = 0.00660095$, $f(0.4) = 0.24842440$
6. Use the Newton backward-difference formula to construct interpolating polynomials of degree one, two, and three for the following data. Approximate the specified value using each of the polynomials.
 - a. $f(0.43)$ if $f(0) = 1$, $f(0.25) = 1.64872$, $f(0.5) = 2.71828$, $f(0.75) = 4.48169$
 - b. $f(0.25)$ if $f(-1) = 0.86199480$, $f(-0.5) = 0.95802009$, $f(0) = 1.0986123$, $f(0.5) = 1.2943767$
7. a. Use Algorithm 3.2 to construct the interpolating polynomial of degree three for the unequally spaced points given in the following table:

x	$f(x)$
-0.1	5.30000
0.0	2.00000
0.2	3.19000
0.3	1.00000

- b. Add $f(0.35) = 0.97260$ to the table and construct the interpolating polynomial of degree four.
8. a. Use Algorithm 3.2 to construct the interpolating polynomial of degree four for the unequally spaced points given in the following table:

x	$f(x)$
0.0	-6.00000
0.1	-5.89483
0.3	-5.65014
0.6	-5.17788
1.0	-4.28172

- b. Add $f(1.1) = -3.99583$ to the table and construct the interpolating polynomial of degree five.
9. a. Approximate $f(0.05)$ using the following data and the Newton forward-difference formula:

x	0.0	0.2	0.4	0.6	0.8
$f(x)$	1.00000	1.22140	1.49182	1.82212	2.22554

- b. Use the Newton backward-difference formula to approximate $f(0.65)$.
- c. Use Stirling's formula to approximate $f(0.43)$.
10. a. Approximate $f(-0.05)$ using the following data and the Newton forward-difference formula:

x	-1.2	-0.9	-0.6	-0.3	0.0
$f(x)$	0.18232	-0.105083	-0.51036	-1.20397	-3.12145

- b. Use the Newton backward-difference formula to approximate $f(-0.2)$.
- c. Use Stirling's formula to approximate $f(-0.43)$.

11. The following data are given for a polynomial $P(x)$ of unknown degree.

x	0	1	2
$P(x)$	2	-1	4

Determine the coefficient of x^2 in $P(x)$ if all third-order forward differences are 1.

12. The following data are given for a polynomial $P(x)$ of unknown degree.

x	0	1	2	3
$P(x)$	4	9	15	18

Determine the coefficient of x^3 in $P(x)$ if all fourth-order forward differences are 1.

13. The Newton forward-difference formula is used to approximate $f(0.3)$ given the following data.

x	0.0	0.2	0.4	0.6
$f(x)$	15.0	21.0	30.0	51.0

Suppose it is discovered that $f(0.4)$ was understated by 10 and $f(0.6)$ was overstated by 5. By what amount should the approximation to $f(0.3)$ be changed?

14. For a function f , the Newton divided-difference formula gives the interpolating polynomial

$$P_3(x) = 1 + 4x + 4x(x - 0.25) + \frac{16}{3}x(x - 0.25)(x - 0.5),$$

on the nodes $x_0 = 0$, $x_1 = 0.25$, $x_2 = 0.5$ and $x_3 = 0.75$. Find $f(0.75)$.

15. A fourth-degree polynomial $P(x)$ satisfies $\Delta^4 P(0) = 24$, $\Delta^3 P(0) = 6$, and $\Delta^2 P(0) = 0$, where $\Delta P(x) = P(x+1) - P(x)$. Compute $\Delta^2 P(10)$.
16. For a function f , the forward-divided differences are given by

$x_0 = 0.0$	$f[x_0]$		
		$f[x_0, x_1]$	
$x_1 = 0.4$	$f[x_1]$		$f[x_0, x_1, x_2] = \frac{50}{7}$
		$f[x_1, x_2] = 10$	
$x_2 = 0.7$	$f[x_2] = 6$		

Determine the missing entries in the table.

APPLIED EXERCISES

17. a. The introduction to this chapter included a table listing the population of the United States from 1960 to 2010. Use appropriate divided differences to approximate the population in the years 1950, 1975, 2014, and 2020.
- b. The population in 1950 was approximately 150,697,360, and in 2014 the population was estimated to be 317,298,000. How accurate do you think your 1975 and 2020 figures are?
18. The fastest time ever recorded in the Kentucky Derby was by a horse named Secretariat in 1973. He covered the $1\frac{1}{4}$ mile track in $1:59\frac{2}{5}$ (1 minute and 59.4 seconds). Times at the quarter-mile, half-mile, and mile poles were $0:25\frac{1}{5}$, $0:49\frac{1}{5}$, and $1:36\frac{2}{5}$.
- a. Use interpolation to predict the time at the three-quarter mile pole and compare this to the actual time of 1:13.
- b. Use the derivative of the interpolating polynomial to estimate the speed of Secretariat at the end of the race.

THEORETICAL EXERCISES

19. Show that the polynomial interpolating the following data has degree three.

x	-2	-1	0	1	2	3
$f(x)$	1	4	11	16	13	-4

20. a. Show that the cubic polynomials

$$P(x) = 3 - 2(x + 1) + 0(x + 1)(x) + (x + 1)(x)(x - 1)$$

and

$$Q(x) = -1 + 4(x + 2) - 3(x + 2)(x + 1) + (x + 2)(x + 1)(x)$$

both interpolate the data

x	-2	-1	0	1	2
$f(x)$	-1	3	1	-1	3

- b. Why does part (a) not violate the uniqueness property of interpolating polynomials?

21. Given

$$\begin{aligned} P_n(x) = & f[x_0] + f[x_0, x_1](x - x_0) + a_2(x - x_0)(x - x_1) \\ & + a_3(x - x_0)(x - x_1)(x - x_2) + \cdots \\ & + a_n(x - x_0)(x - x_1) \cdots (x - x_{n-1}), \end{aligned}$$

use $P_n(x_2)$ to show that $a_2 = f[x_0, x_1, x_2]$.

22. Show that

$$f[x_0, x_1, \dots, x_n, x] = \frac{f^{(n+1)}(\xi(x))}{(n+1)!},$$

for some $\xi(x)$. [Hint: From Eq. (3.3),

$$f(x) = P_n(x) + \frac{f^{(n+1)}(\xi(x))}{(n+1)!}(x - x_0) \cdots (x - x_n).$$

Considering the interpolation polynomial of degree $n + 1$ on $x_0, x_1, \dots, x_n, x$, we have

$$f(x) = P_{n+1}(x) = P_n(x) + f[x_0, x_1, \dots, x_n, x](x - x_0) \cdots (x - x_n).$$

23. Let $i_0, i_1, \dots, i_n$ be a rearrangement of the integers $0, 1, \dots, n$. Show that $f[x_{i_0}, x_{i_1}, \dots, x_{i_n}] = f[x_0, x_1, \dots, x_n]$. [Hint: Consider the leading coefficient of the n th Lagrange polynomial on the data $\{x_0, x_1, \dots, x_n\} = \{x_{i_0}, x_{i_1}, \dots, x_{i_n}\}$.]

DISCUSSION QUESTIONS

1. Compare and contrast the various divided-difference methods you read about in this chapter.
2. Is it easier to add a new data pair using divided-difference methods or the Lagrange polynomial in order to obtain a higher-degree polynomial?
3. The Lagrange polynomial was used to derive the error formula for polynomial interpolation. Could any of the divided-difference formulas be used to derive that error? why or Why not?



3.4 Hermite Interpolation

The Latin word *osculum*, literally a “small mouth” or “kiss,” when applied to a curve, indicates that it just touches and has the same shape. Hermite interpolation has this osculating property. It matches a given curve, and its derivative forces the interpolating curve to “kiss” the given curve.

Osculating polynomials generalize both the Taylor polynomials and the Lagrange polynomials. Suppose that we are given $n + 1$ distinct numbers $x_0, x_1, \dots, x_n$ in $[a, b]$ and nonnegative integers $m_0, m_1, \dots, m_n$, and $m = \max\{m_0, m_1, \dots, m_n\}$. The osculating polynomial approximating a function $f \in C^m[a, b]$ at x_i , for each $i = 0, \dots, n$, is the polynomial of least degree that has the same values as the function f and all its derivatives of order less than or equal to m_i at each x_i . The degree of this osculating polynomial is at most

$$M = \sum_{i=0}^n m_i + n$$

because the number of conditions to be satisfied is $\sum_{i=0}^n m_i + (n + 1)$, and a polynomial of degree M has $M + 1$ coefficients that can be used to satisfy these conditions.

Definition 3.8

Let $x_0, x_1, \dots, x_n$ be $n + 1$ distinct numbers in $[a, b]$, and for $i = 0, 1, \dots, n$, let m_i be a nonnegative integer. Suppose that $f \in C^m[a, b]$, where $m = \max_{0 \leq i \leq n} m_i$.

The **osculating polynomial** approximating f is the polynomial $P(x)$ of least degree such that

$$\frac{d^k P(x_i)}{dx^k} = \frac{d^k f(x_i)}{dx^k}, \quad \text{for each } i = 0, 1, \dots, n \quad \text{and} \quad k = 0, 1, \dots, m_i. \quad \blacksquare$$

Charles Hermite (1822–1901) made significant mathematical discoveries throughout his life in areas such as complex analysis and number theory, particularly involving the theory of equations. He is perhaps best known for proving in 1873 that e is transcendental; that is, it is not the solution to any algebraic equation having integer coefficients. This led in 1882 to Lindemann’s proof that π is also transcendental, which demonstrated that it is impossible to use the standard geometry tools of Euclid to construct a square that has the same area as a unit circle.

Note that when $n = 0$, the osculating polynomial approximating f is the m_0 th Taylor polynomial for f at x_0 . When $m_i = 0$ for each i , the osculating polynomial is the n th Lagrange polynomial interpolating f on $x_0, x_1, \dots, x_n$.

Hermite Polynomials

The case when $m_i = 1$, for each $i = 0, 1, \dots, n$, gives the **Hermite polynomials**. For a given function f , these polynomials agree with f at $x_0, x_1, \dots, x_n$. In addition, since their first derivatives agree with those of f , they have the same “shape” as the function at $(x_i, f(x_i))$ in the sense that the *tangent lines* to the polynomial and the function agree. We will restrict our study of osculating polynomials to this situation and consider first a theorem that describes precisely the form of the Hermite polynomials.

Theorem 3.9

If $f \in C^1[a, b]$ and $x_0, \dots, x_n \in [a, b]$ are distinct, the unique polynomial of least degree agreeing with f and f' at $x_0, \dots, x_n$ is the Hermite polynomial of degree at most $2n + 1$ given by

$$H_{2n+1}(x) = \sum_{j=0}^n f(x_j) H_{n,j}(x) + \sum_{j=0}^n f'(x_j) \hat{H}_{n,j}(x),$$

where, for $L_{n,j}(x)$ denoting the j th Lagrange coefficient polynomial of degree n , we have

$$H_{n,j}(x) = [1 - 2(x - x_j)L'_{n,j}(x_j)]L_{n,j}^2(x) \quad \text{and} \quad \hat{H}_{n,j}(x) = (x - x_j)L_{n,j}^2(x).$$

Moreover, if $f \in C^{2n+2}[a, b]$, then

$$f(x) = H_{2n+1}(x) + \frac{(x - x_0)^2 \dots (x - x_n)^2}{(2n + 2)!} f^{(2n+2)}(\xi(x)),$$

for some (generally unknown) $\xi(x)$ in the interval (a, b) . ■

In 1878, Hermite gave a description of a general osculatory polynomial in a letter to Carl W. Borchardt, to whom he regularly sent his new results. His demonstration is an interesting application of the use of complex integration techniques to solve a real-valued problem.

Proof First, recall that

$$L_{n,j}(x_i) = \begin{cases} 0, & \text{if } i \neq j, \\ 1, & \text{if } i = j. \end{cases}$$

Hence, when $i \neq j$,

$$H_{n,j}(x_i) = 0 \quad \text{and} \quad \hat{H}_{n,j}(x_i) = 0,$$

whereas, for each i ,

$$H_{n,i}(x_i) = [1 - 2(x_i - x_i)L'_{n,i}(x_i)] \cdot 1 = 1 \quad \text{and} \quad \hat{H}_{n,i}(x_i) = (x_i - x_i) \cdot 1^2 = 0.$$

As a consequence,

$$H_{2n+1}(x_i) = \sum_{\substack{j=0 \\ j \neq i}}^n f(x_j) \cdot 0 + f(x_i) \cdot 1 + \sum_{j=0}^n f'(x_j) \cdot 0 = f(x_i),$$

so H_{2n+1} agrees with f at $x_0, x_1, \dots, x_n$.

To show the agreement of H'_{2n+1} with f' at the nodes, first note that $L_{n,j}(x)$ is a factor of $H'_{n,j}(x)$, so $H'_{n,j}(x_i) = 0$ when $i \neq j$. In addition, when $i = j$, we have $L_{n,i}(x_i) = 1$, so

$$\begin{aligned} H'_{n,i}(x_i) &= -2L'_{n,i}(x_i) \cdot L_{n,i}^2(x_i) + [1 - 2(x_i - x_i)L'_{n,i}(x_i)]2L_{n,i}(x_i)L'_{n,i}(x_i) \\ &= -2L'_{n,i}(x_i) + 2L'_{n,i}(x_i) = 0. \end{aligned}$$

Hence, $H'_{n,j}(x_i) = 0$ for all i and j .

Finally,

$$\begin{aligned} \hat{H}'_{n,j}(x_i) &= L_{n,j}^2(x_i) + (x_i - x_j)2L_{n,j}(x_i)L'_{n,j}(x_i) \\ &= L_{n,j}(x_i)[L_{n,j}(x_i) + 2(x_i - x_j)L'_{n,j}(x_i)], \end{aligned}$$

so $\hat{H}'_{n,j}(x_i) = 0$ if $i \neq j$ and $\hat{H}'_{n,i}(x_i) = 1$. Combining these facts, we have

$$H'_{2n+1}(x_i) = \sum_{j=0}^n f(x_j) \cdot 0 + \sum_{\substack{j=0 \\ j \neq i}}^n f'(x_j) \cdot 0 + f'(x_i) \cdot 1 = f'(x_i).$$

Therefore, H_{2n+1} agrees with f and H'_{2n+1} with f' at $x_0, x_1, \dots, x_n$.

The uniqueness of this polynomial and the derivation of the error formula are considered in Exercise 11. ■

Example 1 Use the Hermite polynomial that agrees with the data listed in Table 3.15 to find an approximation of $f(1.5)$.

Table 3.15

k	x_k	$f(x_k)$	$f'(x_k)$
0	1.3	0.6200860	-0.5220232
1	1.6	0.4554022	-0.5698959
2	1.9	0.2818186	-0.5811571

Solution We first compute the Lagrange polynomials and their derivatives. This gives

$$\begin{aligned} L_{2,0}(x) &= \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)} = \frac{50}{9}x^2 - \frac{175}{9}x + \frac{152}{9}, & L'_{2,0}(x) &= \frac{100}{9}x - \frac{175}{9}; \\ L_{2,1}(x) &= \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)} = \frac{-100}{9}x^2 + \frac{320}{9}x - \frac{247}{9}, & L'_{2,1}(x) &= \frac{-200}{9}x + \frac{320}{9}; \end{aligned}$$

and

$$L_{2,2}(x) = \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)} = \frac{50}{9}x^2 - \frac{145}{9}x + \frac{104}{9}, \quad L'_{2,2}(x) = \frac{100}{9}x - \frac{145}{9}.$$

The polynomials $H_{2,j}(x)$ and $\hat{H}_{2,j}(x)$ are then

$$\begin{aligned} H_{2,0}(x) &= [1 - 2(x-1.3)(-5)] \left(\frac{50}{9}x^2 - \frac{175}{9}x + \frac{152}{9} \right)^2 \\ &= (10x - 12) \left(\frac{50}{9}x^2 - \frac{175}{9}x + \frac{152}{9} \right)^2, \\ H_{2,1}(x) &= 1 \cdot \left(\frac{-100}{9}x^2 + \frac{320}{9}x - \frac{247}{9} \right)^2, \\ H_{2,2}(x) &= 10(2-x) \left(\frac{50}{9}x^2 - \frac{145}{9}x + \frac{104}{9} \right)^2, \\ \hat{H}_{2,0}(x) &= (x-1.3) \left(\frac{50}{9}x^2 - \frac{175}{9}x + \frac{152}{9} \right)^2, \\ \hat{H}_{2,1}(x) &= (x-1.6) \left(\frac{-100}{9}x^2 + \frac{320}{9}x - \frac{247}{9} \right)^2, \end{aligned}$$

and

$$\hat{H}_{2,2}(x) = (x-1.9) \left(\frac{50}{9}x^2 - \frac{145}{9}x + \frac{104}{9} \right)^2.$$

Finally,

$$\begin{aligned} H_5(x) &= 0.6200860H_{2,0}(x) + 0.4554022H_{2,1}(x) + 0.2818186H_{2,2}(x) \\ &\quad - 0.5220232\hat{H}_{2,0}(x) - 0.5698959\hat{H}_{2,1}(x) - 0.5811571\hat{H}_{2,2}(x) \end{aligned}$$

and

$$\begin{aligned} H_5(1.5) &= 0.6200860 \left(\frac{4}{27} \right) + 0.4554022 \left(\frac{64}{81} \right) + 0.2818186 \left(\frac{5}{81} \right) \\ &\quad - 0.5220232 \left(\frac{4}{405} \right) - 0.5698959 \left(\frac{-32}{405} \right) - 0.5811571 \left(\frac{-2}{405} \right) = 0.5118277, \end{aligned}$$

a result that is accurate to the places listed. ■

Although Theorem 3.9 provides a complete description of the Hermite polynomials, it is clear from Example 1 that the need to determine and evaluate the Lagrange polynomials and their derivatives makes the procedure tedious even for small values of n .

Hermite Polynomials Using Divided Differences

There is an alternative method for generating Hermite approximations that has as its basis the Newton interpolatory divided-difference formula (3.10) at $x_0, x_1, \dots, x_n$; that is,

$$P_n(x) = f[x_0] + \sum_{k=1}^n f[x_0, x_1, \dots, x_k](x - x_0) \cdots (x - x_{k-1}).$$

The alternative method uses the connection between the n th divided difference and the n th derivative of f , as outlined in Theorem 3.6 in Section 3.3.

Suppose that the distinct numbers $x_0, x_1, \dots, x_n$ are given together with the values of f and f' at these numbers. Define a new sequence $z_0, z_1, \dots, z_{2n+1}$ by

$$z_{2i} = z_{2i+1} = x_i, \quad \text{for each } i = 0, 1, \dots, n,$$

and construct the divided difference table in the form of Table 3.9 that uses $z_0, z_1, \dots, z_{2n+1}$.

Since $z_{2i} = z_{2i+1} = x_i$ for each i , we cannot define $f[z_{2i}, z_{2i+1}]$ by the divided-difference formula. However, if we assume, based on Theorem 3.6, that the reasonable substitution in this situation is $f[z_{2i}, z_{2i+1}] = f'(z_{2i}) = f'(x_i)$, we can use the entries

$$f'(x_0), f'(x_1), \dots, f'(x_n)$$

in place of the undefined first divided differences

$$f[z_0, z_1], f[z_2, z_3], \dots, f[z_{2n}, z_{2n+1}].$$

The remaining divided differences are produced as usual, and the appropriate divided differences are employed in Newton's interpolatory divided-difference formula. Table 3.16 shows the entries that are used for the first three divided-difference columns when determining the Hermite polynomial $H_5(x)$ for x_0, x_1 , and x_2 . The remaining entries are generated in the same manner as in Table 3.9. The Hermite polynomial is then given by

$$H_{2n+1}(x) = f[z_0] + \sum_{k=1}^{2n+1} f[z_0, \dots, z_k](x - z_0)(x - z_1) \cdots (x - z_{k-1}).$$

A proof of this fact can be found in [Pow], p. 56.

Table 3.16

z	$f(z)$	First divided differences	Second divided differences
$z_0 = x_0$	$f[z_0] = f(x_0)$		
		$f[z_0, z_1] = f'(x_0)$	
$z_1 = x_0$	$f[z_1] = f(x_0)$		$f[z_0, z_1, z_2] = \frac{f[z_1, z_2] - f[z_0, z_1]}{z_2 - z_0}$
		$f[z_1, z_2] = \frac{f[z_2] - f[z_1]}{z_2 - z_1}$	
$z_2 = x_1$	$f[z_2] = f(x_1)$		$f[z_1, z_2, z_3] = \frac{f[z_2, z_3] - f[z_1, z_2]}{z_3 - z_1}$
		$f[z_2, z_3] = f'(x_1)$	
$z_3 = x_1$	$f[z_3] = f(x_1)$		$f[z_2, z_3, z_4] = \frac{f[z_3, z_4] - f[z_2, z_3]}{z_4 - z_2}$
		$f[z_3, z_4] = \frac{f[z_4] - f[z_3]}{z_4 - z_3}$	
$z_4 = x_2$	$f[z_4] = f(x_2)$		$f[z_3, z_4, z_5] = \frac{f[z_4, z_5] - f[z_3, z_4]}{z_5 - z_3}$
		$f[z_4, z_5] = f'(x_2)$	
$z_5 = x_2$	$f[z_5] = f(x_2)$		

Example 2 Use the data given in Example 1 and the divided difference method to determine the Hermite polynomial approximation at $x = 1.5$.

Solution The underlined entries in the first three columns of Table 3.17 are the data given in Example 1. The remaining entries in this table are generated by the standard divided-difference formula (3.9).

For example, for the second entry in the third column we use the second 1.3 entry in the second column and the first 1.6 entry in that column to obtain

$$\frac{0.4554022 - 0.6200860}{1.6 - 1.3} = -0.5489460.$$

For the first entry in the fourth column, we use the first 1.3 entry in the third column and the first 1.6 entry in that column to obtain

$$\frac{-0.5489460 - (-0.5220232)}{1.6 - 1.3} = -0.0897427.$$

The value of the Hermite polynomial at 1.5 is

$$\begin{aligned} H_5(1.5) &= f[1.3] + f'(1.3)(1.5 - 1.3) + f[1.3, 1.3, 1.6](1.5 - 1.3)^2 \\ &\quad + f[1.3, 1.3, 1.6, 1.6](1.5 - 1.3)^2(1.5 - 1.6) \\ &\quad + f[1.3, 1.3, 1.6, 1.6, 1.9](1.5 - 1.3)^2(1.5 - 1.6)^2 \\ &\quad + f[1.3, 1.3, 1.6, 1.6, 1.9, 1.9](1.5 - 1.3)^2(1.5 - 1.6)^2(1.5 - 1.9) \\ &= 0.6200860 + (-0.5220232)(0.2) + (-0.0897427)(0.2)^2 \\ &\quad + 0.0663657(0.2)^2(-0.1) + 0.0026663(0.2)^2(-0.1)^2 \\ &\quad + (-0.0027738)(0.2)^2(-0.1)^2(-0.4) \\ &= 0.5118277. \end{aligned}$$

Table 3.17

<u>1.3</u>	<u>0.6200860</u>					
		<u>-0.5220232</u>				
<u>1.3</u>	<u>0.6200860</u>		-0.0897427			
		-0.5489460		0.0663657		
<u>1.6</u>	<u>0.4554022</u>		-0.0698330		0.0026663	
		<u>-0.5698959</u>		0.0679655		-0.0027738
<u>1.6</u>	<u>0.4554022</u>		-0.0290537		0.0010020	
		-0.5786120		0.0685667		
<u>1.9</u>	<u>0.2818186</u>		-0.0084837			
		<u>-0.5811571</u>				
<u>1.9</u>	<u>0.2818186</u>					

The technique used in Algorithm 3.3 can be extended for use in determining other osculating polynomials. A concise discussion of the procedures can be found in [Pow], pp. 53–57.

ALGORITHM
3.3
Hermite Interpolation

To obtain the coefficients of the Hermite interpolating polynomial $H(x)$ on the $(n + 1)$ distinct numbers $x_0, \dots, x_n$ for the function f :

INPUT numbers $x_0, x_1, \dots, x_n$; values $f(x_0), \dots, f(x_n)$ and $f'(x_0), \dots, f'(x_n)$.

OUTPUT the numbers $Q_{0,0}, Q_{1,1}, \dots, Q_{2n+1,2n+1}$ where

$$\begin{aligned} H(x) = & Q_{0,0} + Q_{1,1}(x - x_0) + Q_{2,2}(x - x_0)^2 + Q_{3,3}(x - x_0)^2(x - x_1) \\ & + Q_{4,4}(x - x_0)^2(x - x_1)^2 + \dots \\ & + Q_{2n+1,2n+1}(x - x_0)^2(x - x_1)^2 \dots (x - x_{n-1})^2(x - x_n). \end{aligned}$$

Step 1 For $i = 0, 1, \dots, n$ do Steps 2 and 3.

Step 2 Set $z_{2i} = x_i$;

$$z_{2i+1} = x_i;$$

$$Q_{2i,0} = f(x_i);$$

$$Q_{2i+1,0} = f(x_i);$$

$$Q_{2i+1,1} = f'(x_i).$$

Step 3 If $i \neq 0$ then set

$$Q_{2i,1} = \frac{Q_{2i,0} - Q_{2i-1,0}}{z_{2i} - z_{2i-1}}.$$

Step 4 For $i = 2, 3, \dots, 2n + 1$

$$\text{for } j = 2, 3, \dots, i \text{ set } Q_{i,j} = \frac{Q_{i,j-1} - Q_{i-1,j-1}}{z_i - z_{i-j}}.$$

Step 5 OUTPUT $(Q_{0,0}, Q_{1,1}, \dots, Q_{2n+1,2n+1})$;
STOP.

EXERCISE SET 3.4

1. Use Theorem 3.9 or Algorithm 3.3 to construct an approximating polynomial for the following data.

a.	x	$f(x)$	$f'(x)$
	8.3	17.56492	3.116256
	8.6	18.50515	3.151762

b.	x	$f(x)$	$f'(x)$
	0.8	0.22363362	2.1691753
	1.0	0.65809197	2.0466965

c.	x	$f(x)$	$f'(x)$
	-0.5	-0.0247500	0.7510000
	-0.25	0.3349375	2.1890000
	0	1.1010000	4.0020000

d.	x	$f(x)$	$f'(x)$
	0.1	-0.62049958	3.58502082
	0.2	-0.28398668	3.14033271
	0.3	0.00660095	2.66668043
	0.4	0.24842440	2.16529366

2. Use Theorem 3.9 or Algorithm 3.3 to construct an approximating polynomial for the following data.

a.	x	$f(x)$	$f'(x)$
	0	1.00000	2.00000
	0.5	2.71828	5.43656

b.	x	$f(x)$	$f'(x)$
	-0.25	1.33203	0.437500
	0.25	0.800781	-0.625000

c.	x	$f(x)$	$f'(x)$
	0.1	-0.29004996	-2.8019975
	0.2	-0.56079734	-2.6159201
	0.3	-0.81401972	-2.9734038

d.	x	$f(x)$	$f'(x)$
	-1	0.86199480	0.15536240
	-0.5	0.95802009	0.23269654
	0	1.0986123	0.33333333
	0.5	1.2943767	0.45186776

3. The data in Exercise 1 were generated using the following functions. Use the polynomials constructed in Exercise 1 for the given value of x to approximate $f(x)$ and calculate the absolute error.
- $f(x) = x \ln x$; approximate $f(8.4)$.
 - $f(x) = \sin(e^x - 2)$; approximate $f(0.9)$.
 - $f(x) = x^3 + 4.001x^2 + 4.002x + 1.101$; approximate $f(-1/3)$.
 - $f(x) = x \cos x - 2x^2 + 3x - 1$; approximate $f(0.25)$.
4. The data in Exercise 2 were generated using the following functions. Use the polynomials constructed in Exercise 2 for the given value of x to approximate $f(x)$ and calculate the absolute error.
- $f(x) = e^{2x}$; approximate $f(0.43)$.
 - $f(x) = x^4 - x^3 + x^2 - x + 1$; approximate $f(0)$.
 - $f(x) = x^2 \cos x - 3x$; approximate $f(0.18)$.
 - $f(x) = \ln(e^x + 2)$; approximate $f(0.25)$.
5. a. Use the following values and five-digit rounding arithmetic to construct the Hermite interpolating polynomial to approximate $\sin 0.34$.

x	$\sin x$	$D_x \sin x = \cos x$
0.30	0.29552	0.95534
0.32	0.31457	0.94924
0.35	0.34290	0.93937

- Determine an error bound for the approximation in part (a) and compare it to the actual error.
 - Add $\sin 0.33 = 0.32404$ and $\cos 0.33 = 0.94604$ to the data and redo the calculations.
6. Let $f(x) = 3xe^x - e^{2x}$.
- Approximate $f(1.03)$ by the Hermite interpolating polynomial of degree at most three using $x_0 = 1$ and $x_1 = 1.05$. Compare the actual error to the error bound.
 - Repeat (a) with the Hermite interpolating polynomial of degree at most five using $x_0 = 1$, $x_1 = 1.05$, and $x_2 = 1.07$.
7. The following table lists data for the function described by $f(x) = e^{0.1x^2}$. Approximate $f(1.25)$ by using $H_5(1.25)$ and $H_3(1.25)$, where H_5 uses the nodes $x_0 = 1$, $x_1 = 2$, and $x_2 = 3$ and H_3 uses the nodes $\bar{x}_0 = 1$ and $\bar{x}_1 = 1.5$. Find error bounds for these approximations.

x	$f(x) = e^{0.1x^2}$	$f'(x) = 0.2xe^{0.1x^2}$
$x_0 = \bar{x}_0 = 1$	1.105170918	0.2210341836
$\bar{x}_1 = 1.5$	1.252322716	0.3756968148
$x_1 = 2$	1.491824698	0.5967298792
$x_2 = 3$	2.459603111	1.475761867

APPLIED EXERCISES

8. A baseball pitcher throws a fastball from the pitcher's mound to the catcher. Although the distance from the mound to home plate is 60 feet 6 inches, the ball typically travels about 55 feet 5 inches. Suppose the initial velocity of the ball is 95 miles per hour and that the terminal velocity at home plate is 92 miles per hour. Construct a Hermite interpolating polynomial for the data

Time t (in seconds)	0	0.4
Distance d (in feet)	0	55.5
Speed (miles per hour)	95	92

- a. Use the derivative of the Hermite polynomial to estimate the speed of the baseball in miles per hour at $t = 0.2$ seconds.
- b. Does the maximum velocity of the ball occur at $t = 0$, or does the derivative of the Hermite polynomial have a maximum exceeding 95 miles per hour? If so, does this seem reasonable? [Hint: Convert miles per hour to feet per second to solve the problem and then convert back to miles per hour to give the answers.]
9. A car traveling along a straight road is clocked at a number of points. The data from the observations are given in the following table, where the time is in seconds, the distance is in feet, and the speed is in feet per second.

Time	0	3	5	8	13
Distance	0	225	383	623	993
Speed	75	77	80	74	72

- a. Use a Hermite polynomial to predict the position of the car and its speed when $t = 10$ seconds.
- b. Use the derivative of the Hermite polynomial to determine whether the car ever exceeds a 55-mile-per-hour speed limit on the road. If so, what is the first time the car exceeds this speed?
- c. What is the predicted maximum speed for the car?

THEORETICAL EXERCISES

10. Let $z_0 = x_0$, $z_1 = x_0$, $z_2 = x_1$, and $z_3 = x_1$. Form the following divided-difference table.

$z_0 = x_0$	$f[z_0] = f(x_0)$			
		$f[z_0, z_1] = f'(x_0)$		
$z_1 = x_0$	$f[z_1] = f(x_0)$		$f[z_0, z_1, z_2]$	
		$f[z_1, z_2]$		$f[z_0, z_1, z_2, z_3]$
$z_2 = x_1$	$f[z_2] = f(x_1)$		$f[z_1, z_2, z_3]$	
		$f[z_2, z_3] = f'(x_1)$		
$z_3 = x_1$	$f[z_3] = f(x_1)$			

Show that the cubic Hermite polynomial $H_3(x)$ can also be written as $f[z_0] + f[z_0, z_1](x - x_0) + f[z_0, z_1, z_2](x - x_0)^2 + f[z_0, z_1, z_2, z_3](x - x_0)^2(x - x_1)$.

11. a. Show that $H_{2n+1}(x)$ is the unique polynomial of least degree agreeing with f and f' at $x_0, \dots, x_n$. [Hint: Assume that $P(x)$ is another such polynomial and consider $D = H_{2n+1} - P$ and D' at $x_0, x_1, \dots, x_n$.]
- b. Derive the error term in Theorem 3.9. [Hint: Use the same method as in the Lagrange error derivation, Theorem 3.3, defining

$$g(t) = f(t) - H_{2n+1}(t) - \frac{(t - x_0)^2 \cdots (t - x_n)^2}{(x - x_0)^2 \cdots (x - x_n)^2} [f(x) - H_{2n+1}(x)]$$

and using the fact that $g'(t)$ has $(2n + 2)$ distinct zeros in $[a, b]$.

DISCUSSION QUESTIONS

1. One of the problems with polynomial interpolation is that although it fits the points, the shape of the curve doesn't always match very well. One approach is to use interpolating polynomials and match the derivatives as well as the points. Describe in your own words how this is accomplished.
2. In this section, two methods for finding the Hermite polynomial were presented. Explain the usefulness for each of the two methods.
3. Investigate the derivation of the divided difference method to compute the Hermite interpolating polynomial. [Hint: Look at the Powell reference].

3.5 Cubic Spline Interpolation¹

The previous sections concerned the approximation of arbitrary functions on closed intervals using a single polynomial. However, high-degree polynomials can oscillate erratically; that is, a minor fluctuation over a small portion of the interval can induce large fluctuations over the entire range. We will see a good example of this in Figure 3.14 at the end of this section.

An alternative approach is to divide the approximation interval into a collection of subintervals and construct a (generally) different approximating polynomial on each subinterval. This is called **piecewise-polynomial approximation**.

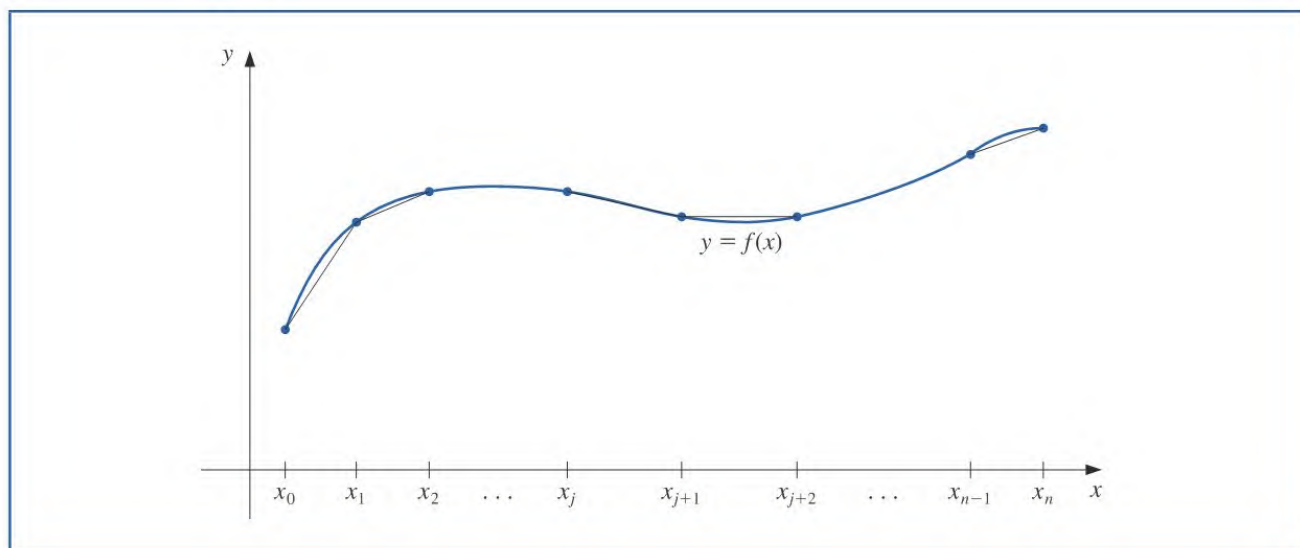
Piecewise-Polynomial Approximation

The simplest piecewise-polynomial approximation is **piecewise-linear** interpolation, which consists of joining a set of data points

$$\{(x_0, f(x_0)), (x_1, f(x_1)), \dots, (x_n, f(x_n))\}$$

by a series of straight lines, as shown in Figure 3.7.

Figure 3.7



A disadvantage of linear function approximation is that there is likely no differentiability at the endpoints of the subintervals, which, in a geometrical context, means that the interpolating function is not “smooth.” Often it is clear from physical conditions that smoothness is required, so the approximating function must be continuously differentiable.

An alternative procedure is to use a piecewise polynomial of Hermite type. For example, if the values of f and of f' are known at each of the points $x_0 < x_1 < \dots < x_n$, a cubic Hermite polynomial can be used on each of the subintervals $[x_0, x_1]$, $[x_1, x_2]$, $\dots$, $[x_{n-1}, x_n]$ to obtain a function that has a continuous derivative on the interval $[x_0, x_n]$.

Determining the appropriate Hermite cubic polynomial on a given interval is simply a matter of computing $H_3(x)$ for that interval. The Lagrange interpolating polynomials needed

¹The proofs of the theorems in this section rely on results in Chapter 6.

Isaac Jacob Schoenberg (1903–1990) developed his work on splines during World War II while on leave from the University of Pennsylvania to work at the Army's Ballistic Research Laboratory in Aberdeen, Maryland. His original work involved numerical procedures for solving differential equations. The much broader application of splines to the areas of data fitting and computer-aided geometric design became evident with the widespread availability of computers in the 1960s.

The root of the word “spline” is the same as that of “splint.” It was originally a small strip of wood that could be used to join two boards. Later, the word was used to refer to a long, flexible strip, generally of metal, that could be used to draw continuous, smooth curves by forcing the strip to pass through specified points and tracing along the curve.

to determine H_3 are of first degree, so this can be accomplished without great difficulty. However, to use Hermite piecewise polynomials for general interpolation, we need to know the derivative of the function being approximated, and this is frequently unavailable.

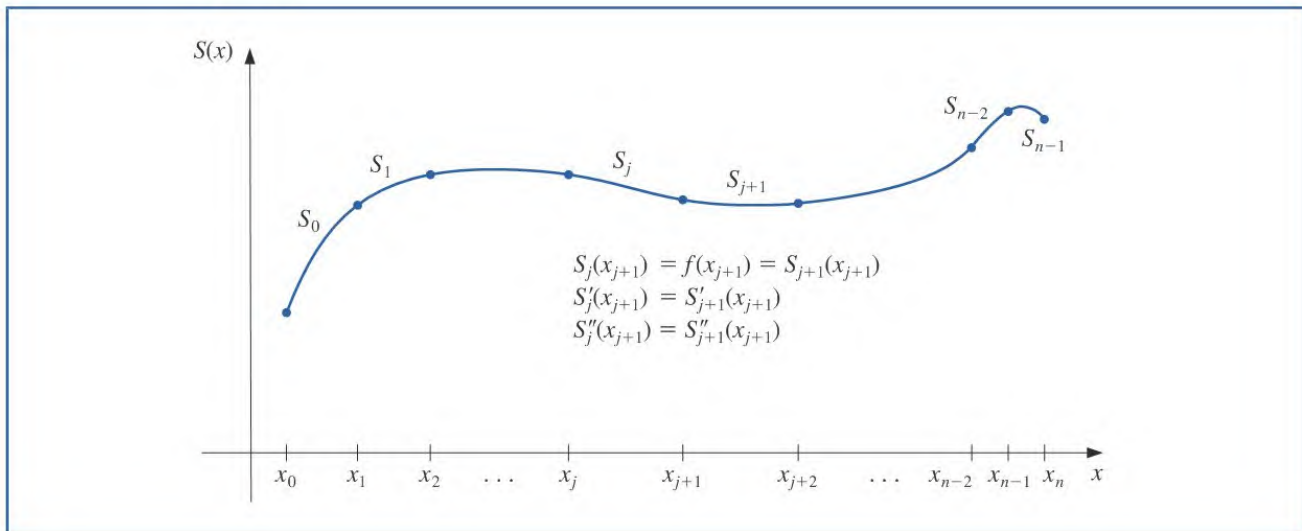
The remainder of this section considers approximation using piecewise polynomials that require no specific derivative information, except perhaps at the endpoints of the interval on which the function is being approximated.

The simplest type of differentiable piecewise-polynomial function on an entire interval $[x_0, x_n]$ is the function obtained by fitting one quadratic polynomial between each successive pair of nodes. This is done by constructing a quadratic on $[x_0, x_1]$ agreeing with the function at x_0 and x_1 , another quadratic on $[x_1, x_2]$ agreeing with the function at x_1 and x_2 , and so on. A general quadratic polynomial has three arbitrary constants—the constant term, the coefficient of x , and the coefficient of x^2 —and only two conditions are required to fit the data at the endpoints of each subinterval. So, flexibility exists that permits the quadratics to be chosen so that the interpolant has a continuous derivative on $[x_0, x_n]$. The difficulty arises because we generally need to specify conditions about the derivative of the interpolant at the endpoints x_0 and x_n . There is not a sufficient number of constants to ensure that the conditions will be satisfied. (See Exercise 34.)

Cubic Splines

The most common piecewise-polynomial approximation uses cubic polynomials between each successive pair of nodes and is called **cubic spline interpolation**. A general cubic polynomial involves four constants, so there is sufficient flexibility in the cubic spline procedure to ensure that the interpolant not only is continuously differentiable on the interval but also has a continuous second derivative. The construction of the cubic spline does not, however, assume that the derivatives of the interpolant agree with those of the function it is approximating, even at the nodes. (See Figure 3.8.)

Figure 3.8



Definition 3.10 Given a function f defined on $[a, b]$ and a set of nodes $a = x_0 < x_1 < \cdots < x_n = b$, a **cubic spline interpolant** S for f is a function that satisfies the following conditions:

A natural spline has no conditions imposed for the direction at its endpoints, so the curve takes the shape of a straight line after it passes through the interpolation points nearest its endpoints. The name derives from the fact that this is the natural shape a flexible strip assumes if forced to pass through specified interpolation points with no additional constraints. (See Figure 3.9.)



Figure 3.9

- (a) $S(x)$ is a cubic polynomial, denoted $S_j(x)$, on the subinterval $[x_j, x_{j+1}]$ for each $j = 0, 1, \dots, n-1$;
- (b) $S_j(x_j) = f(x_j)$ and $S_j(x_{j+1}) = f(x_{j+1})$ for each $j = 0, 1, \dots, n-1$;
- (c) $S_{j+1}(x_{j+1}) = S_j(x_{j+1})$ for each $j = 0, 1, \dots, n-2$; (Implied by (b).)
- (d) $S'_{j+1}(x_{j+1}) = S'_j(x_{j+1})$ for each $j = 0, 1, \dots, n-2$;
- (e) $S''_{j+1}(x_{j+1}) = S''_j(x_{j+1})$ for each $j = 0, 1, \dots, n-2$;
- (f) One of the following sets of boundary conditions is satisfied:
 - (i) $S''(x_0) = S''(x_n) = 0$ (**natural (or free) boundary**);
 - (ii) $S'(x_0) = f'(x_0)$ and $S'(x_n) = f'(x_n)$ (**clamped boundary**).

Although cubic splines are defined with other boundary conditions, the conditions given in part (f) are sufficient for our purposes. When the free boundary conditions occur, the spline is called a **natural spline**, and its graph approximates the shape that a long, flexible rod would assume if forced to go through the data points $\{(x_0, f(x_0)), (x_1, f(x_1)), \dots, (x_n, f(x_n))\}$.

In general, clamped boundary conditions lead to more accurate approximations because they include more information about the function. However, for this type of boundary condition to hold, it is necessary to have either the values of the derivative at the endpoints or an accurate approximation to those values.

Example 1 Construct a natural cubic spline that passes through the points $(1, 2)$, $(2, 3)$, and $(3, 5)$.

Solution This spline consists of two cubics. The first for the interval $[1, 2]$, denoted

$$S_0(x) = a_0 + b_0(x-1) + c_0(x-1)^2 + d_0(x-1)^3,$$

and the other for $[2, 3]$, denoted

$$S_1(x) = a_1 + b_1(x-2) + c_1(x-2)^2 + d_1(x-2)^3.$$

There are eight constants to be determined, and this requires eight conditions. Four conditions come from the fact that the splines must agree with the data at the nodes. Hence,

$$\begin{aligned} 2 &= f(1) = a_0, & 3 &= f(2) = a_0 + b_0 + c_0 + d_0, & 3 &= f(2) = a_1, & \text{and} \\ 5 &= f(3) = a_1 + b_1 + c_1 + d_1. \end{aligned}$$

Two more come from the fact that $S'_0(2) = S'_1(2)$ and $S''_0(2) = S''_1(2)$. These are

$$S'_0(2) = S'_1(2) : \quad b_0 + 2c_0 + 3d_0 = b_1 \quad \text{and} \quad S''_0(2) = S''_1(2) : \quad 2c_0 + 6d_0 = 2c_1.$$

The final two come from the natural boundary conditions:

$$S''_0(1) = 0 : \quad 2c_0 = 0 \quad \text{and} \quad S''_1(3) = 0 : \quad 2c_1 + 6d_1 = 0.$$

Solving this system of equations gives the spline

$$S(x) = \begin{cases} 2 + \frac{3}{4}(x-1) + \frac{1}{4}(x-1)^3, & \text{for } x \in [1, 2] \\ 3 + \frac{3}{2}(x-2) + \frac{3}{4}(x-2)^2 - \frac{1}{4}(x-2)^3, & \text{for } x \in [2, 3]. \end{cases}$$

Construction of a Cubic Spline

As the preceding example demonstrates, a spline defined on an interval that is divided into n subintervals will require determining $4n$ constants. To construct the cubic spline interpolant for a given function f , the conditions in the definition are applied to the cubic polynomials

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3,$$

for each $j = 0, 1, \dots, n-1$. Since $S_j(x_j) = a_j = f(x_j)$, condition (c) can be applied to obtain

$$a_{j+1} = S_{j+1}(x_{j+1}) = S_j(x_{j+1}) = a_j + b_j(x_{j+1} - x_j) + c_j(x_{j+1} - x_j)^2 + d_j(x_{j+1} - x_j)^3,$$

for each $j = 0, 1, \dots, n-2$.

The terms $x_{j+1} - x_j$ are used repeatedly in this development, so it is convenient to introduce the simpler notation

$$h_j = x_{j+1} - x_j,$$

for each $j = 0, 1, \dots, n-1$. If we also define $a_n = f(x_n)$, then the equation

$$a_{j+1} = a_j + b_j h_j + c_j h_j^2 + d_j h_j^3 \quad (3.15)$$

holds for each $j = 0, 1, \dots, n-1$.

In a similar manner, define $b_n = S'(x_n)$ and observe that

$$S'_j(x) = b_j + 2c_j(x - x_j) + 3d_j(x - x_j)^2$$

implies $S'_j(x_j) = b_j$, for each $j = 0, 1, \dots, n-1$. Applying condition in part (d) gives

$$b_{j+1} = b_j + 2c_j h_j + 3d_j h_j^2, \quad (3.16)$$

for each $j = 0, 1, \dots, n-1$.

Another relationship between the coefficients of S_j is obtained by defining $c_n = S''(x_n)/2$ and applying condition in part (e). Then, for each $j = 0, 1, \dots, n-1$,

$$c_{j+1} = c_j + 3d_j h_j. \quad (3.17)$$

Solving for d_j in Eq. (3.17) and substituting this value into Eqs. (3.15) and (3.16) gives, for each $j = 0, 1, \dots, n-1$, the new equations

$$a_{j+1} = a_j + b_j h_j + \frac{h_j^2}{3}(2c_j + c_{j+1}) \quad (3.18)$$

and

$$b_{j+1} = b_j + h_j(c_j + c_{j+1}). \quad (3.19)$$

The final relationship involving the coefficients is obtained by solving the appropriate equation in the form of Eq. (3.18), first for b_j ,

$$b_j = \frac{1}{h_j}(a_{j+1} - a_j) - \frac{h_j}{3}(2c_j + c_{j+1}), \quad (3.20)$$

Clamping a spline indicates that the ends of the flexible strip are fixed so that it is forced to take a specific direction at each of its endpoints. This is important, for example, when two spline functions should match at their endpoints. This is done mathematically by specifying the values of the derivative of the curve at the endpoints of the spline.

and then, with a reduction of the index, for b_{j-1} . This gives

$$b_{j-1} = \frac{1}{h_{j-1}}(a_j - a_{j-1}) - \frac{h_{j-1}}{3}(2c_{j-1} + c_j).$$

Substituting these values into the equation derived from Eq. (3.19), with the index reduced by one, gives the linear system of equations

$$h_{j-1}c_{j-1} + 2(h_{j-1} + h_j)c_j + h_jc_{j+1} = \frac{3}{h_j}(a_{j+1} - a_j) - \frac{3}{h_{j-1}}(a_j - a_{j-1}), \quad (3.21)$$

for each $j = 1, 2, \dots, n-1$. This system involves only the $\{c_j\}_{j=0}^n$ as unknowns. The values of $\{h_j\}_{j=0}^{n-1}$ and $\{a_j\}_{j=0}^n$ are given, respectively, by the spacing of the nodes $\{x_j\}_{j=0}^n$ and the values of f at the nodes. So, once the values of $\{c_j\}_{j=0}^n$ are determined, it is a simple matter to find the remainder of the constants $\{b_j\}_{j=0}^{n-1}$ from Eq. (3.20) and $\{d_j\}_{j=0}^{n-1}$ from Eq. (3.17). Then we can construct the cubic polynomials $\{S_j(x)\}_{j=0}^{n-1}$.

The major question that arises in connection with this construction is whether the values of $\{c_j\}_{j=0}^n$ can be found using the system of equations given in Eq. (3.21) and, if so, whether these values are unique. The following theorems indicate that this is the case when either of the boundary conditions given in part (f) of the definition are imposed. The proofs of these theorems require material from linear algebra, which is discussed in Chapter 6.

Natural Splines

Theorem 3.11 If f is defined at $a = x_0 < x_1 < \dots < x_n = b$, then f has a unique natural spline interpolant S on the nodes $x_0, x_1, \dots, x_n$; that is, a spline interpolant that satisfies the natural boundary conditions $S''(a) = 0$ and $S''(b) = 0$.

Proof The boundary conditions in this case imply that $c_n = S''(x_n)/2 = 0$ and that

$$0 = S''(x_0) = 2c_0 + 6d_0(x_0 - x_0),$$

so $c_0 = 0$. The two equations $c_0 = 0$ and $c_n = 0$ together with the equations in (3.21) produce a linear system described by the vector equation $A\mathbf{x} = \mathbf{b}$, where A is the $(n+1) \times (n+1)$ matrix

$$A = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ h_0 & 2(h_0 + h_1) & h_1 & \dots & 0 \\ 0 & h_1 & 2(h_1 + h_2) & \dots & h_{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & \dots & \dots & 1 \end{bmatrix},$$

and $\mathbf{b}$ and $\mathbf{x}$ are the vectors

$$\mathbf{b} = \begin{bmatrix} 0 \\ \frac{3}{h_1}(a_2 - a_1) - \frac{3}{h_0}(a_1 - a_0) \\ \vdots \\ \frac{3}{h_{n-1}}(a_n - a_{n-1}) - \frac{3}{h_{n-2}}(a_{n-1} - a_{n-2}) \\ 0 \end{bmatrix} \quad \text{and} \quad \mathbf{x} = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix}.$$

The matrix A is strictly diagonally dominant; that is, in each row, the magnitude of the diagonal entry exceeds the sum of the magnitudes of all the other entries in the row. A linear system with a matrix of this form will be shown by Theorem 6.21 in Section 6.6 to have a unique solution for $c_0, c_1, \dots, c_n$. ■

The solution to the cubic spline problem with the boundary conditions $S''(x_0) = S''(x_n) = 0$ can be obtained by applying Algorithm 3.4.

ALGORITHM 3.4

Natural Cubic Spline

To construct the cubic spline interpolant S for the function f , defined at the numbers $x_0 < x_1 < \dots < x_n$, satisfying $S''(x_0) = S''(x_n) = 0$:

INPUT $n; x_0, x_1, \dots, x_n; a_0 = f(x_0), a_1 = f(x_1), \dots, a_n = f(x_n)$.

OUTPUT a_j, b_j, c_j, d_j for $j = 0, 1, \dots, n-1$.

(Note: $S(x) = S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3$ for $x_j \leq x \leq x_{j+1}$.)

Step 1 For $i = 0, 1, \dots, n-1$ set $h_i = x_{i+1} - x_i$.

Step 2 For $i = 1, 2, \dots, n-1$ set

$$\alpha_i = \frac{3}{h_i}(a_{i+1} - a_i) - \frac{3}{h_{i-1}}(a_i - a_{i-1}).$$

Step 3 Set $l_0 = 1$; (Steps 3, 4, and 5 and part of Step 6 solve a tridiagonal linear system using a method described in Algorithm 6.7.)

$$\mu_0 = 0;$$

$$z_0 = 0.$$

Step 4 For $i = 1, 2, \dots, n-1$

$$\text{set } l_i = 2(x_{i+1} - x_{i-1}) - h_{i-1}\mu_{i-1};$$

$$\mu_i = h_i/l_i;$$

$$z_i = (\alpha_i - h_{i-1}z_{i-1})/l_i.$$

Step 5 Set $l_n = 1$;

$$z_n = 0;$$

$$c_n = 0.$$

Step 6 For $j = n-1, n-2, \dots, 0$

$$\text{set } c_j = z_j - \mu_j c_{j+1};$$

$$b_j = (a_{j+1} - a_j)/h_j - h_j(c_{j+1} + 2c_j)/3;$$

$$d_j = (c_{j+1} - c_j)/(3h_j).$$

Step 7 **OUTPUT** (a_j, b_j, c_j, d_j) for $j = 0, 1, \dots, n-1$;
STOP. ■

Example 2 At the beginning of Chapter 3, we gave some Taylor polynomials to approximate the exponential $f(x) = e^x$. Use the data points $(0, 1)$, $(1, e)$, $(2, e^2)$, and $(3, e^3)$ to form a natural spline $S(x)$ that approximates $f(x) = e^x$.

Solution We have $n = 3$, $h_0 = h_1 = h_2 = 1$, $a_0 = 1$, $a_1 = e$, $a_2 = e^2$, and $a_3 = e^3$. So, the matrix A and the vectors $\mathbf{b}$ and $\mathbf{x}$ given in Theorem 3.11 have the forms

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 0 & 1 & 4 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0 \\ 3(e^2 - 2e + 1) \\ 3(e^3 - 2e^2 + e) \\ 0 \end{bmatrix}, \quad \text{and} \quad \mathbf{x} = \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}.$$

The vector-matrix equation $A\mathbf{x} = \mathbf{b}$ is equivalent to the system of equations

$$\begin{aligned} c_0 &= 0, \\ c_0 + 4c_1 + c_2 &= 3(e^2 - 2e + 1), \\ c_1 + 4c_2 + c_3 &= 3(e^3 - 2e^2 + e), \\ c_3 &= 0. \end{aligned}$$

This system has the solution $c_0 = c_3 = 0$, and to five decimal places,

$$c_1 = \frac{1}{5}(-e^3 + 6e^2 - 9e + 4) \approx 0.75685, \quad \text{and} \quad c_2 = \frac{1}{5}(4e^3 - 9e^2 + 6e - 1) \approx 5.83007.$$

Solving for the remaining constants gives

$$\begin{aligned} b_0 &= \frac{1}{h_0}(a_1 - a_0) - \frac{h_0}{3}(c_1 + 2c_0) \\ &= (e - 1) - \frac{1}{15}(-e^3 + 6e^2 - 9e + 4) \approx 1.46600, \\ b_1 &= \frac{1}{h_1}(a_2 - a_1) - \frac{h_1}{3}(c_2 + 2c_1) \\ &= (e^2 - e) - \frac{1}{15}(2e^3 + 3e^2 - 12e + 7) \approx 2.22285, \\ b_2 &= \frac{1}{h_2}(a_3 - a_2) - \frac{h_2}{3}(c_3 + 2c_2) \\ &= (e^3 - e^2) - \frac{1}{15}(8e^3 - 18e^2 + 12e - 2) \approx 8.80977, \\ d_0 &= \frac{1}{3h_0}(c_1 - c_0) = \frac{1}{15}(-e^3 + 6e^2 - 9e + 4) \approx 0.25228, \\ d_1 &= \frac{1}{3h_1}(c_2 - c_1) = \frac{1}{3}(e^3 - 3e^2 + 3e - 1) \approx 1.69107, \end{aligned}$$

and

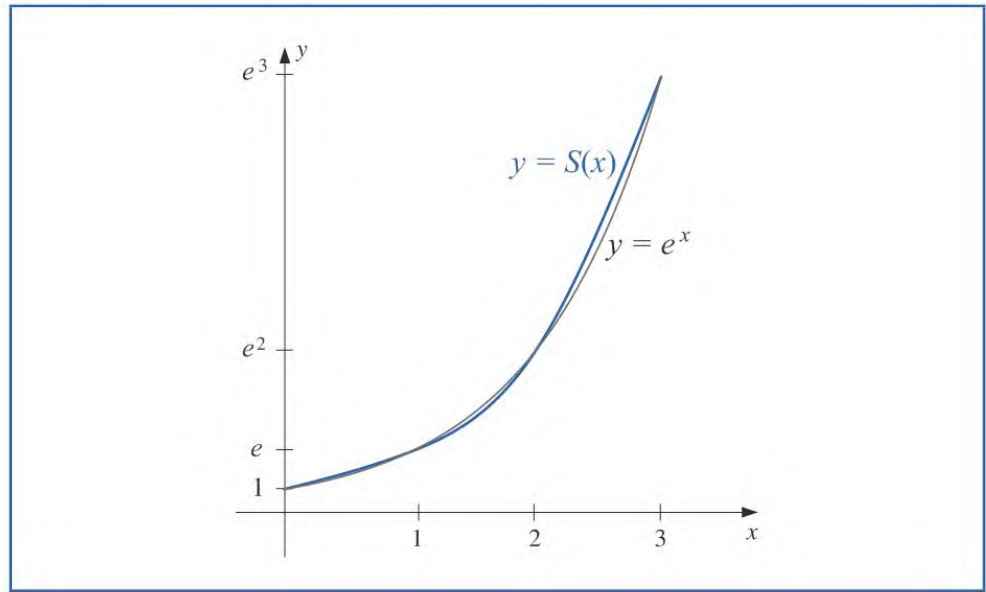
$$d_2 = \frac{1}{3h_2}(c_3 - c_1) = \frac{1}{15}(-4e^3 + 9e^2 - 6e + 1) \approx -1.94336.$$

The natural cubic spline is described piecewise by

$$S(x) = \begin{cases} 1 + 1.46600x + 0.25228x^3, & \text{for } x \in [0, 1], \\ 2.71828 + 2.22285(x-1) + 0.75685(x-1)^2 + 1.69107(x-1)^3, & \text{for } x \in [1, 2], \\ 7.38906 + 8.80977(x-2) + 5.83007(x-2)^2 - 1.94336(x-2)^3, & \text{for } x \in [2, 3]. \end{cases}$$

The spline and its agreement with $f(x) = e^x$ are shown in Figure 3.10. ■

Figure 3.10



Once we have determined a spline approximation for a function, we can use it to approximate other properties of the function. The next illustration involves the integral of the spline we found in the previous example.

Illustration To approximate the integral of $f(x) = e^x$ on $[0, 3]$, which has the value

$$\int_0^3 e^x dx = e^3 - 1 \approx 20.08553692 - 1 = 19.08553692,$$

we can piecewise integrate the spline that approximates f on this interval. This gives

$$\begin{aligned} \int_0^3 S(x) dx &= \int_0^1 (1 + 1.46600x + 0.25228x^3) dx \\ &\quad + \int_1^2 (2.71828 + 2.22285(x-1) + 0.75685(x-1)^2 + 1.69107(x-1)^3) dx \\ &\quad + \int_2^3 (7.38906 + 8.80977(x-2) + 5.83007(x-2)^2 - 1.94336(x-2)^3) dx. \end{aligned}$$

Integrating and collecting values from like powers gives

$$\begin{aligned}
 \int_0^3 S(x) &= \left[x + 1.46600 \frac{x^2}{2} + 0.25228 \frac{x^4}{4} \right]_0^1 \\
 &\quad + \left[2.71828(x-1) + 2.22285 \frac{(x-1)^2}{2} + 0.75685 \frac{(x-1)^3}{3} + 1.69107 \frac{(x-1)^4}{4} \right]_1^2 \\
 &\quad + \left[7.38906(x-2) + 8.80977 \frac{(x-2)^2}{2} + 5.83007 \frac{(x-2)^3}{3} - 1.94336 \frac{(x-2)^4}{4} \right]_2^3 \\
 &= (1 + 2.71828 + 7.38906) + \frac{1}{2} (1.46600 + 2.22285 + 8.80977) \\
 &\quad + \frac{1}{3} (0.75685 + 5.83007) + \frac{1}{4} (0.25228 + 1.69107 - 1.94336) \\
 &= 19.55229.
 \end{aligned}$$

Because the nodes are equally spaced in this example, the integral approximation is simply

$$\int_0^3 S(x) dx = (a_0 + a_1 + a_2) + \frac{1}{2}(b_0 + b_1 + b_2) + \frac{1}{3}(c_0 + c_1 + c_2) + \frac{1}{4}(d_0 + d_1 + d_2). \quad (3.22)$$

■

Clamped Splines

Example 3 In Example 1, we found a natural spline S that passes through the points $(1, 2)$, $(2, 3)$, and $(3, 5)$. Construct a clamped spline s through these points that has $s'(1) = 2$ and $s'(3) = 1$.

Solution Let

$$s_0(x) = a_0 + b_0(x-1) + c_0(x-1)^2 + d_0(x-1)^3$$

be the cubic on $[1, 2]$ and the cubic on $[2, 3]$ be

$$s_1(x) = a_1 + b_1(x-2) + c_1(x-2)^2 + d_1(x-2)^3.$$

Then most of the conditions to determine the eight constants are the same as those in Example 1. That is,

$$\begin{aligned}
 2 &= f(1) = a_0, & 3 &= f(2) = a_0 + b_0 + c_0 + d_0, & 3 &= f(2) = a_1, & \text{and} \\
 5 &= f(3) = a_1 + b_1 + c_1 + d_1.
 \end{aligned}$$

$$s'_0(2) = s'_1(2) : \quad b_0 + 2c_0 + 3d_0 = b_1 \quad \text{and} \quad s''_0(2) = s''_1(2) : \quad 2c_0 + 6d_0 = 2c_1$$

However, the boundary conditions are now

$$s'_0(1) = 2 : \quad b_0 = 2 \quad \text{and} \quad s'_1(3) = 1 : \quad b_1 + 2c_1 + 3d_1 = 1.$$

Solving this system of equations gives the spline as

$$s(x) = \begin{cases} 2 + 2(x-1) - \frac{5}{2}(x-1)^2 + \frac{3}{2}(x-1)^3, & \text{for } x \in [1, 2] \\ 3 + \frac{3}{2}(x-2) + 2(x-2)^2 - \frac{3}{2}(x-2)^3, & \text{for } x \in [2, 3] \end{cases} \quad \blacksquare$$

In the case of general clamped boundary conditions, we have a result that is similar to the theorem for natural boundary conditions described in Theorem 3.11.

Theorem 3.12 If f is defined at $a = x_0 < x_1 < \cdots < x_n = b$ and differentiable at a and b , then f has a unique clamped spline interpolant S on the nodes $x_0, x_1, \dots, x_n$; that is, a spline interpolant that satisfies the clamped boundary conditions $S'(a) = f'(a)$ and $S'(b) = f'(b)$.

Proof Since $f'(a) = S'(a) = S'(x_0) = b_0$, Eq. (3.20) with $j = 0$ implies

$$f'(a) = \frac{1}{h_0}(a_1 - a_0) - \frac{h_0}{3}(2c_0 + c_1).$$

Consequently,

$$2h_0c_0 + h_0c_1 = \frac{3}{h_0}(a_1 - a_0) - 3f'(a).$$

Similarly,

$$f'(b) = b_n = b_{n-1} + h_{n-1}(c_{n-1} + c_n),$$

so Eq. (3.20) with $j = n - 1$ implies that

$$\begin{aligned} f'(b) &= \frac{a_n - a_{n-1}}{h_{n-1}} - \frac{h_{n-1}}{3}(2c_{n-1} + c_n) + h_{n-1}(c_{n-1} + c_n) \\ &= \frac{a_n - a_{n-1}}{h_{n-1}} + \frac{h_{n-1}}{3}(c_{n-1} + 2c_n), \end{aligned}$$

and

$$h_{n-1}c_{n-1} + 2h_{n-1}c_n = 3f'(b) - \frac{3}{h_{n-1}}(a_n - a_{n-1}).$$

Equations (3.21) together with the equations

$$2h_0c_0 + h_0c_1 = \frac{3}{h_0}(a_1 - a_0) - 3f'(a)$$

and

$$h_{n-1}c_{n-1} + 2h_{n-1}c_n = 3f'(b) - \frac{3}{h_{n-1}}(a_n - a_{n-1})$$

determine the linear system $A\mathbf{x} = \mathbf{b}$, where

$$A = \begin{bmatrix} 2h_0 & h_0 & 0 & \cdots & 0 \\ h_0 & 2(h_0 + h_1) & h_1 & \cdots & 0 \\ 0 & h_1 & 2(h_1 + h_2) & h_2 & \cdots \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & h_{n-2} & 2(h_{n-2} + h_{n-1}) & h_{n-1} \\ 0 & \cdots & 0 & h_{n-1} & 2h_{n-1} \end{bmatrix},$$

$$\mathbf{b} = \begin{bmatrix} \frac{3}{h_0}(a_1 - a_0) - 3f'(a) \\ \frac{3}{h_1}(a_2 - a_1) - \frac{3}{h_0}(a_1 - a_0) \\ \vdots \\ \frac{3}{h_{n-1}}(a_n - a_{n-1}) - \frac{3}{h_{n-2}}(a_{n-1} - a_{n-2}) \\ 3f'(b) - \frac{3}{h_{n-1}}(a_n - a_{n-1}) \end{bmatrix}, \quad \text{and} \quad \mathbf{x} = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix}.$$

This matrix A is also strictly diagonally dominant, so it satisfies the conditions of Theorem 6.21 in Section 6.6. Therefore, the linear system has a unique solution for $c_0, c_1, \dots, c_n$. ■

The solution to the cubic spline problem with the boundary conditions $S'(x_0) = f'(x_0)$ and $S'(x_n) = f'(x_n)$ can be obtained by applying Algorithm 3.5.

ALGORITHM 3.5

Clamped Cubic Spline

To construct the cubic spline interpolant S for the function f defined at the numbers $x_0 < x_1 < \cdots < x_n$, satisfying $S'(x_0) = f'(x_0)$ and $S'(x_n) = f'(x_n)$:

INPUT $n; x_0, x_1, \dots, x_n; a_0 = f(x_0), a_1 = f(x_1), \dots, a_n = f(x_n); FPO = f'(x_0); FPN = f'(x_n)$.

OUTPUT a_j, b_j, c_j, d_j for $j = 0, 1, \dots, n-1$.

(Note: $S(x) = S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3$ for $x_j \leq x \leq x_{j+1}$.)

Step 1 For $i = 0, 1, \dots, n-1$ set $h_i = x_{i+1} - x_i$.

Step 2 Set $\alpha_0 = 3(a_1 - a_0)/h_0 - 3FPO$;
 $\alpha_n = 3FPN - 3(a_n - a_{n-1})/h_{n-1}$.

Step 3 For $i = 1, 2, \dots, n-1$

$$\text{set } \alpha_i = \frac{3}{h_i}(a_{i+1} - a_i) - \frac{3}{h_{i-1}}(a_i - a_{i-1}).$$

Step 4 Set $l_0 = 2h_0$; (Steps 4, 5, and 6 and part of Step 7 solve a tridiagonal linear system using a method described in Algorithm 6.7.)

$$\mu_0 = 0.5;$$

$$z_0 = \alpha_0/l_0.$$

Step 5 For $i = 1, 2, \dots, n-1$

$$\text{set } l_i = 2(x_{i+1} - x_{i-1}) - h_{i-1}\mu_{i-1};$$

$$\mu_i = h_i/l_i;$$

$$z_i = (\alpha_i - h_{i-1}z_{i-1})/l_i.$$

Step 6 Set $l_n = h_{n-1}(2 - \mu_{n-1})$;
 $z_n = (\alpha_n - h_{n-1}z_{n-1})/l_n$;
 $c_n = z_n$.

Step 7 For $j = n - 1, n - 2, \dots, 0$
 set $c_j = z_j - \mu_j c_{j+1}$;
 $b_j = (a_{j+1} - a_j)/h_j - h_j(c_{j+1} + 2c_j)/3$;
 $d_j = (c_{j+1} - c_j)/(3h_j)$.

Step 8 OUTPUT $(a_j, b_j, c_j, d_j$ for $j = 0, 1, \dots, n - 1)$;
 STOP.

Example 4 Example 2 used a natural spline and the data points $(0, 1)$, $(1, e)$, $(2, e^2)$, and $(3, e^3)$ to form a new approximating function $S(x)$. Determine the clamped spline $s(x)$ that uses these data and the additional information that, since $f'(x) = e^x$, $f'(0) = 1$ and $f'(3) = e^3$.

Solution As in Example 2, we have $n = 3$, $h_0 = h_1 = h_2 = 1$, $a_0 = 0$, $a_1 = e$, $a_2 = e^2$, and $a_3 = e^3$. This, together with the information that $f'(0) = 1$ and $f'(3) = e^3$, gives the the matrix A and the vectors $\mathbf{b}$ and $\mathbf{x}$ with the forms

$$A = \begin{bmatrix} 2 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 0 & 1 & 4 & 1 \\ 0 & 0 & 1 & 2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 3(e - 2) \\ 3(e^2 - 2e + 1) \\ 3(e^3 - 2e^2 + e) \\ 3e^2 \end{bmatrix}, \quad \text{and} \quad \mathbf{x} = \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}.$$

The vector-matrix equation $A\mathbf{x} = \mathbf{b}$ is equivalent to the system of equations

$$\begin{aligned} 2c_0 + c_1 &= 3(e - 2), \\ c_0 + 4c_1 + c_2 &= 3(e^2 - 2e + 1), \\ c_1 + 4c_2 + c_3 &= 3(e^3 - 2e^2 + e), \\ c_2 + 2c_3 &= 3e^2. \end{aligned}$$

Solving this system simultaneously for c_0 , c_1 , c_2 and c_3 gives, to five decimal places,

$$\begin{aligned} c_0 &= \frac{1}{15}(2e^3 - 12e^2 + 42e - 59) = 0.44468, \\ c_1 &= \frac{1}{15}(-4e^3 + 24e^2 - 39e + 28) = 1.26548, \\ c_2 &= \frac{1}{15}(14e^3 - 39e^2 + 24e - 8) = 3.35087, \\ c_3 &= \frac{1}{15}(-7e^3 + 42e^2 - 12e + 4) = 9.40815. \end{aligned}$$

Solving for the remaining constants in the same manner as Example 2 gives

$$b_0 = 1.00000, \quad b_1 = 2.71016, \quad b_2 = 7.32652,$$

and

$$d_0 = 0.27360, \quad d_1 = 0.69513, \quad d_2 = 2.01909.$$

This gives the clamped cubic spline

$$s(x) = \begin{cases} 1 + x + 0.44468x^2 + 0.27360x^3, & \text{if } 0 \leq x < 1, \\ 2.71828 + 2.71016(x-1) + 1.26548(x-1)^2 + 0.69513(x-1)^3, & \text{if } 1 \leq x < 2, \\ 7.38906 + 7.32652(x-2) + 3.35087(x-2)^2 + 2.01909(x-2)^3, & \text{if } 2 \leq x \leq 3. \end{cases}$$

The graph of the clamped spline and $f(x) = e^x$ are so similar that no difference can be seen. ■

We can also approximate the integral of f on $[0, 3]$ by integrating the clamped spline. The exact value of the integral is

$$\int_0^3 e^x dx = e^3 - 1 \approx 20.08554 - 1 = 19.08554.$$

Because the data are equally spaced, piecewise integrating the clamped spline results in the same formula as in (3.22); that is,

$$\begin{aligned} \int_0^3 s(x) dx &= (a_0 + a_1 + a_2) + \frac{1}{2}(b_0 + b_1 + b_2) \\ &\quad + \frac{1}{3}(c_0 + c_1 + c_2) + \frac{1}{4}(d_0 + d_1 + d_2). \end{aligned}$$

Hence, the integral approximation is

$$\begin{aligned} \int_0^3 s(x) dx &= (1 + 2.71828 + 7.38906) + \frac{1}{2}(1 + 2.71016 + 7.32652) \\ &\quad + \frac{1}{3}(0.44468 + 1.26548 + 3.35087) + \frac{1}{4}(0.27360 + 0.69513 + 2.01909) \\ &= 19.05965. \end{aligned}$$

The absolute error in the integral approximation using the clamped and natural splines are

$$\text{Natural: } |19.08554 - 19.55229| = 0.46675$$

and

$$\text{Clamped: } |19.08554 - 19.05965| = 0.02589.$$

For integration purposes, the clamped spline is vastly superior. This should be no surprise since the boundary conditions for the clamped spline are exact, whereas for the natural spline we are essentially assuming that, since $f''(x) = e^x$,

$$0 = S''(x) \approx f''(0) = e^1 = 1 \quad \text{and} \quad 0 = S''(3) \approx f''(3) = e^3 \approx 20.$$

The next illustration uses a spline to approximate a curve that has no given functional representation.

Illustration Figure 3.11 shows a ruddy duck in flight. To approximate the top profile of the duck, we have chosen points along the curve through which we want the approximating curve to pass. Table 3.18 lists the coordinates of 21 data points relative to the superimposed coordinate system shown in Figure 3.12. Notice that more points are used when the curve is changing rapidly than when it is changing more slowly.

Figure 3.11

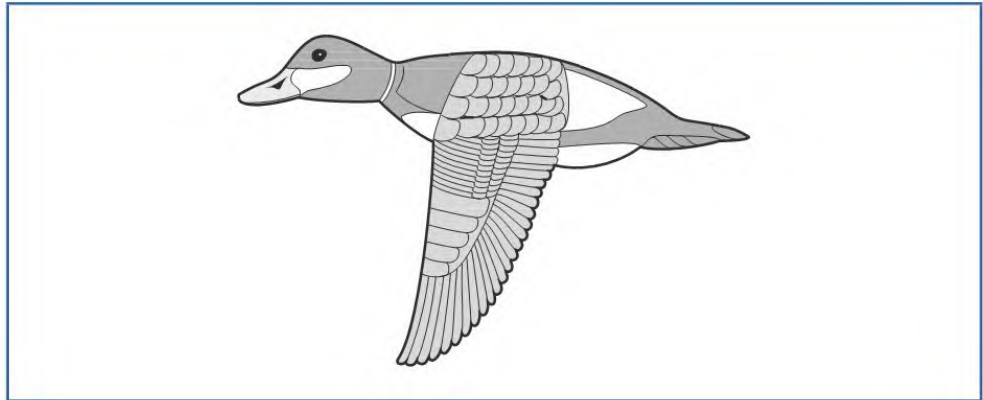
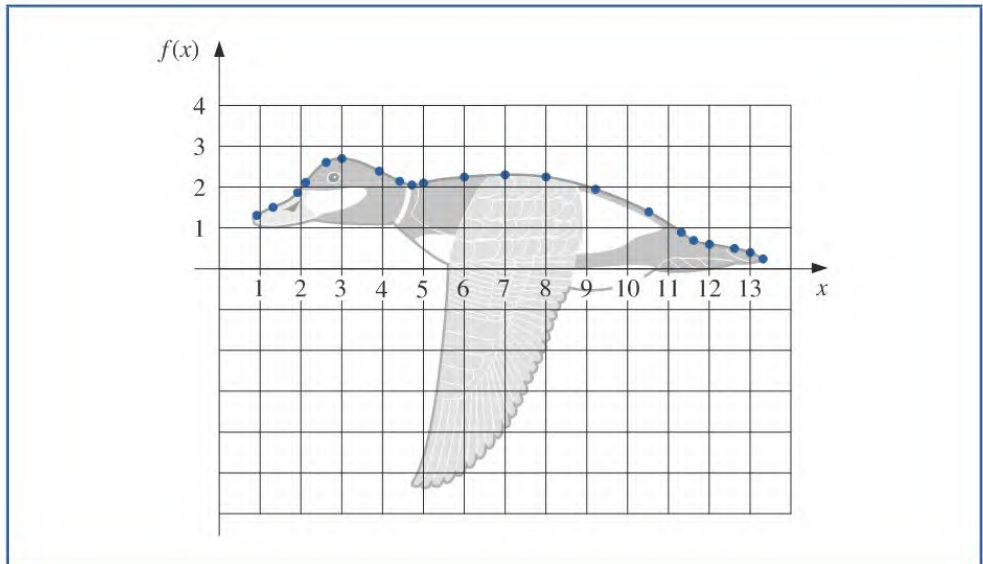


Table 3.18

x	0.9	1.3	1.9	2.1	2.6	3.0	3.9	4.4	4.7	5.0	6.0	7.0	8.0	9.2	10.5	11.3	11.6	12.0	12.6	13.0	13.3
$f(x)$	1.3	1.5	1.85	2.1	2.6	2.7	2.4	2.15	2.05	2.1	2.25	2.3	2.25	1.95	1.4	0.9	0.7	0.6	0.5	0.4	0.25

Figure 3.12

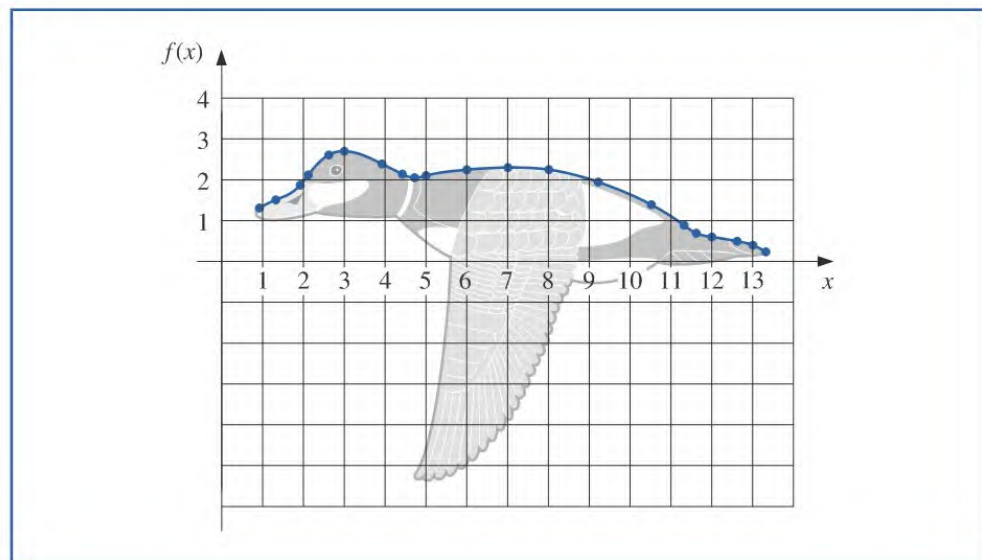


Using Algorithm 3.4 to generate the natural cubic spline for these data produces the coefficients shown in Table 3.19. This spline curve is nearly identical to the profile, as shown in Figure 3.13.

Table 3.19

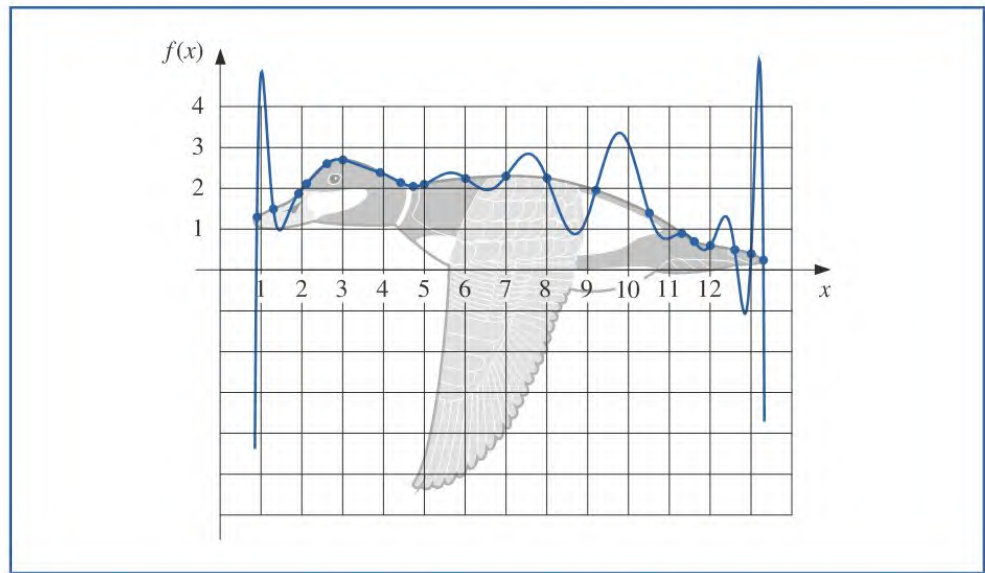
j	x_j	a_j	b_j	c_j	d_j
0	0.9	1.3	0.54	0.00	-0.25
1	1.3	1.5	0.42	-0.30	0.95
2	1.9	1.85	1.09	1.41	-2.96
3	2.1	2.1	1.29	-0.37	-0.45
4	2.6	2.6	0.59	-1.04	0.45
5	3.0	2.7	-0.02	-0.50	0.17
6	3.9	2.4	-0.50	-0.03	0.08
7	4.4	2.15	-0.48	0.08	1.31
8	4.7	2.05	-0.07	1.27	-1.58
9	5.0	2.1	0.26	-0.16	0.04
10	6.0	2.25	0.08	-0.03	0.00
11	7.0	2.3	0.01	-0.04	-0.02
12	8.0	2.25	-0.14	-0.11	0.02
13	9.2	1.95	-0.34	-0.05	-0.01
14	10.5	1.4	-0.53	-0.10	-0.02
15	11.3	0.9	-0.73	-0.15	1.21
16	11.6	0.7	-0.49	0.94	-0.84
17	12.0	0.6	-0.14	-0.06	0.04
18	12.6	0.5	-0.18	0.00	-0.45
19	13.0	0.4	-0.39	-0.54	0.60
20	13.3	0.25			

Figure 3.13



For comparison purposes, Figure 3.14 gives an illustration of the curve that is generated using a Lagrange interpolating polynomial to fit the data given in Table 3.18. The interpolating polynomial in this case is of degree 20 and oscillates wildly. It produces a very strange illustration of the back of a duck, in flight or otherwise.

Figure 3.14



To use a clamped spline to approximate this curve, we would need derivative approximations for the endpoints. Even if these approximations were available, we could expect little improvement because of the close agreement of the natural cubic spline to the curve of the top profile. ■

Constructing a cubic spline to approximate the lower profile of the ruddy duck would be more difficult since the curve for this portion cannot be expressed as a function of x , and at certain points the curve does not appear to be smooth. These problems can be resolved by using separate splines to represent various portions of the curve, but a more effective approach to approximating curves of this type is considered in the next section.

The clamped boundary conditions are generally preferred when approximating functions by cubic splines, so the derivative of the function must be known or approximated at the endpoints of the interval. When the nodes are equally spaced near both endpoints, approximations can be obtained by any of the appropriate formulas given in Sections 4.1 and 4.2. When the nodes are unequally spaced, the problem is considerably more difficult.

To conclude this section, we list an error-bound formula for the cubic spline with clamped boundary conditions. The proof of this result can be found in [Schul], pp. 57–58.

Theorem 3.13 Let $f \in C^4[a, b]$ with $\max_{a \leq x \leq b} |f^{(4)}(x)| = M$. If S is the unique clamped cubic spline interpolant to f with respect to the nodes $a = x_0 < x_1 < \cdots < x_n = b$, then, for all x in $[a, b]$,

$$|f(x) - S(x)| \leq \frac{5M}{384} \max_{0 \leq j \leq n-1} (x_{j+1} - x_j)^4. \quad \blacksquare$$

A fourth-order error-bound result also holds in the case of natural boundary conditions, but it is more difficult to express. (See [BD], pp. 827–835.)

The natural boundary conditions will generally give less accurate results than the clamped conditions near the ends of the interval $[x_0, x_n]$ unless the function f happens to nearly satisfy $f''(x_0) = f''(x_n) = 0$. An alternative to the natural boundary condition that does not require knowledge of the derivative of f is the *not-a-knot* condition (see [Deb2], pp. 55–56). This condition requires that $S'''(x)$ be continuous at x_1 and at x_{n-1} .

EXERCISE SET 3.5

- Determine the natural cubic spline S that interpolates the data $f(0) = 0$, $f(1) = 1$, and $f(2) = 2$.
- Determine the clamped cubic spline s that interpolates the data $f(0) = 0$, $f(1) = 1$ and $f(2) = 2$ and satisfies $s'(0) = s'(2) = 1$.
- Construct the natural cubic spline for the following data.

a.

x	$f(x)$
8.3	17.56492
8.6	18.50515

b.

x	$f(x)$
0.8	0.22363362
1.0	0.65809197

c.

x	$f(x)$
-0.5	-0.0247500
-0.25	0.3349375
0	1.1010000

d.

x	$f(x)$
0.1	-0.62049958
0.2	-0.28398668
0.3	0.00660095
0.4	0.24842440

- Construct the natural cubic spline for the following data.

a.

x	$f(x)$
0	1.00000
0.5	2.71828

b.

x	$f(x)$
-0.25	1.33203
0.25	0.800781

c.

x	$f(x)$
0.1	-0.29004996
0.2	-0.56079734
0.3	-0.81401972

d.

x	$f(x)$
-1	0.86199480
-0.5	0.95802009
0	1.0986123
0.5	1.2943767

- The data in Exercise 3 were generated using the following functions. Use the cubic splines constructed in Exercise 3 for the given value of x to approximate $f(x)$ and $f'(x)$ and calculate the actual error.
 - $f(x) = x \ln x$; approximate $f(8.4)$ and $f'(8.4)$.
 - $f(x) = \sin(e^x - 2)$; approximate $f(0.9)$ and $f'(0.9)$.
 - $f(x) = x^3 + 4.001x^2 + 4.002x + 1.101$; approximate $f(-\frac{1}{3})$ and $f'(-\frac{1}{3})$.
 - $f(x) = x \cos x - 2x^2 + 3x - 1$; approximate $f(0.25)$ and $f'(0.25)$.
- The data in Exercise 4 were generated using the following functions. Use the cubic splines constructed in Exercise 4 for the given value of x to approximate $f(x)$ and $f'(x)$ and calculate the actual error.
 - $f(x) = e^{2x}$; approximate $f(0.43)$ and $f'(0.43)$.
 - $f(x) = x^4 - x^3 + x^2 - x + 1$; approximate $f(0)$ and $f'(0)$.
 - $f(x) = x^2 \cos x - 3x$; approximate $f(0.18)$ and $f'(0.18)$.
 - $f(x) = \ln(e^x + 2)$; approximate $f(0.25)$ and $f'(0.25)$.
- Construct the clamped cubic spline using the data of Exercise 3 and the fact that
 - $f'(8.3) = 1.116256$ and $f'(8.6) = 1.151762$.
 - $f'(0.8) = 2.1691753$ and $f'(1.0) = 2.0466965$.
 - $f'(-0.5) = 0.7510000$ and $f'(0) = 4.0020000$.
 - $f'(0.1) = 3.58502082$ and $f'(0.4) = 2.16529366$.
- Construct the clamped cubic spline using the data of Exercise 3 and the fact that
 - $f'(0) = 2$ and $f'(0.5) = 5.43656$.
 - $f'(-0.25) = 0.437500$ and $f'(0.25) = -0.625000$.
 - $f'(0.1) = -2.8004996$ and $f'(0) = -2.9734038$.
 - $f'(-1) = 0.15536240$ and $f'(0.5) = 0.45186276$.
- Repeat Exercise 5 using the clamped cubic splines constructed in Exercise 7.
- Repeat Exercise 6 using the clamped cubic splines constructed in Exercise 8.

11. A natural cubic spline S on $[0, 2]$ is defined by

$$S(x) = \begin{cases} S_0(x) = 1 + 2x - x^3, & \text{if } 0 \leq x < 1, \\ S_1(x) = 2 + b(x-1) + c(x-1)^2 + d(x-1)^3, & \text{if } 1 \leq x \leq 2. \end{cases}$$

Find b , c , and d .

12. A natural cubic spline S is defined by

$$S(x) = \begin{cases} S_0(x) = 1 + B(x-1) - D(x-1)^3, & \text{if } 1 \leq x < 2, \\ S_1(x) = 1 + b(x-2) - \frac{3}{4}(x-2)^2 + d(x-2)^3, & \text{if } 2 \leq x \leq 3. \end{cases}$$

If S interpolates the data $(1, 1)$, $(2, 1)$, and $(3, 0)$, find B , D , b , and d .

13. A clamped cubic spline s for a function f is defined on $[1, 3]$ by

$$s(x) = \begin{cases} s_0(x) = 3(x-1) + 2(x-1)^2 - (x-1)^3, & \text{if } 1 \leq x < 2, \\ s_1(x) = a + b(x-2) + c(x-2)^2 + d(x-2)^3, & \text{if } 2 \leq x \leq 3. \end{cases}$$

Given $f'(1) = f'(3)$, find a , b , c , and d .

14. A clamped cubic spline s for a function f is defined by

$$s(x) = \begin{cases} s_0(x) = 1 + Bx + 2x^2 - 2x^3, & \text{if } 0 \leq x < 1, \\ s_1(x) = 1 + b(x-1) - 4(x-1)^2 + 7(x-1)^3, & \text{if } 1 \leq x \leq 2. \end{cases}$$

Find $f'(0)$ and $f'(2)$.

15. Given the partition $x_0 = 0$, $x_1 = 0.05$, and $x_2 = 0.1$ of $[0, 0.1]$, find the piecewise linear interpolating function F for $f(x) = e^{2x}$. Approximate $\int_0^{0.1} e^{2x} dx$ with $\int_0^{0.1} F(x) dx$ and compare the results to the actual value.
16. Given the partition $x_0 = 0$, $x_1 = 0.3$, and $x_2 = 0.5$ of $[0, 0.5]$, find the piecewise linear interpolating function F for $f(x) = \sin 3x$. Approximate $\int_0^{0.5} \sin 3x dx$ with $\int_0^{0.5} F(x) dx$ and compare the results to the actual value.
17. Construct a natural cubic spline to approximate $f(x) = \cos \pi x$ by using the values given by $f(x)$ at $x = 0, 0.25, 0.5, 0.75$, and 1.0 . Integrate the spline over $[0, 1]$ and compare the result to $\int_0^1 \cos \pi x dx = 0$. Use the derivatives of the spline to approximate $f'(0.5)$ and $f''(0.5)$. Compare these approximations to the actual values.
18. Construct a natural cubic spline to approximate $f(x) = e^{-x}$ by using the values given by $f(x)$ at $x = 0, 0.25, 0.75$, and 1.0 . Integrate the spline over $[0, 1]$ and compare the result to $\int_0^1 e^{-x} dx = 1 - 1/e$. Use the derivatives of the spline to approximate $f'(0.5)$ and $f''(0.5)$. Compare the approximations to the actual values.
19. Repeat Exercise 17, constructing instead the clamped cubic spline with $f'(0) = f'(1) = 0$.
20. Repeat Exercise 18, constructing instead the clamped cubic spline with $f'(0) = -1$, $f'(1) = -e^{-1}$.
21. Given the partition $x_0 = 0$, $x_1 = 0.05$, $x_2 = 0.1$ of $[0, 0.1]$ and $f(x) = e^{2x}$:
- Find the cubic spline s with clamped boundary conditions that interpolates f .
 - Find an approximation for $\int_0^{0.1} e^{2x} dx$ by evaluating $\int_0^{0.1} s(x) dx$.
 - Use Theorem 3.13 to estimate $\max_{0 \leq x \leq 0.1} |f(x) - s(x)|$ and

$$\left| \int_0^{0.1} f(x) dx - \int_0^{0.1} s(x) dx \right|.$$

- Determine the cubic spline S with natural boundary conditions and compare $S(0.02)$, $s(0.02)$, and $e^{0.04} = 1.04081077$.
22. Given the partition $x_0 = 0$, $x_1 = 0.3$, $x_2 = 0.5$ of $[0, 0.5]$ and $f(x) = \sin 3x$:
- Find the cubic spline s with clamped boundary conditions that interpolates f .
 - Find an approximation for $\int_0^{0.5} \sin 3x dx$ with $\int_0^{0.5} s(x) dx$ and compare the results to the actual value.

APPLIED EXERCISES

23. A car traveling along a straight road is clocked at a number of points. The data from the observations are given in the following table, where the time is in seconds, the distance is in feet, and the speed is in feet per second.

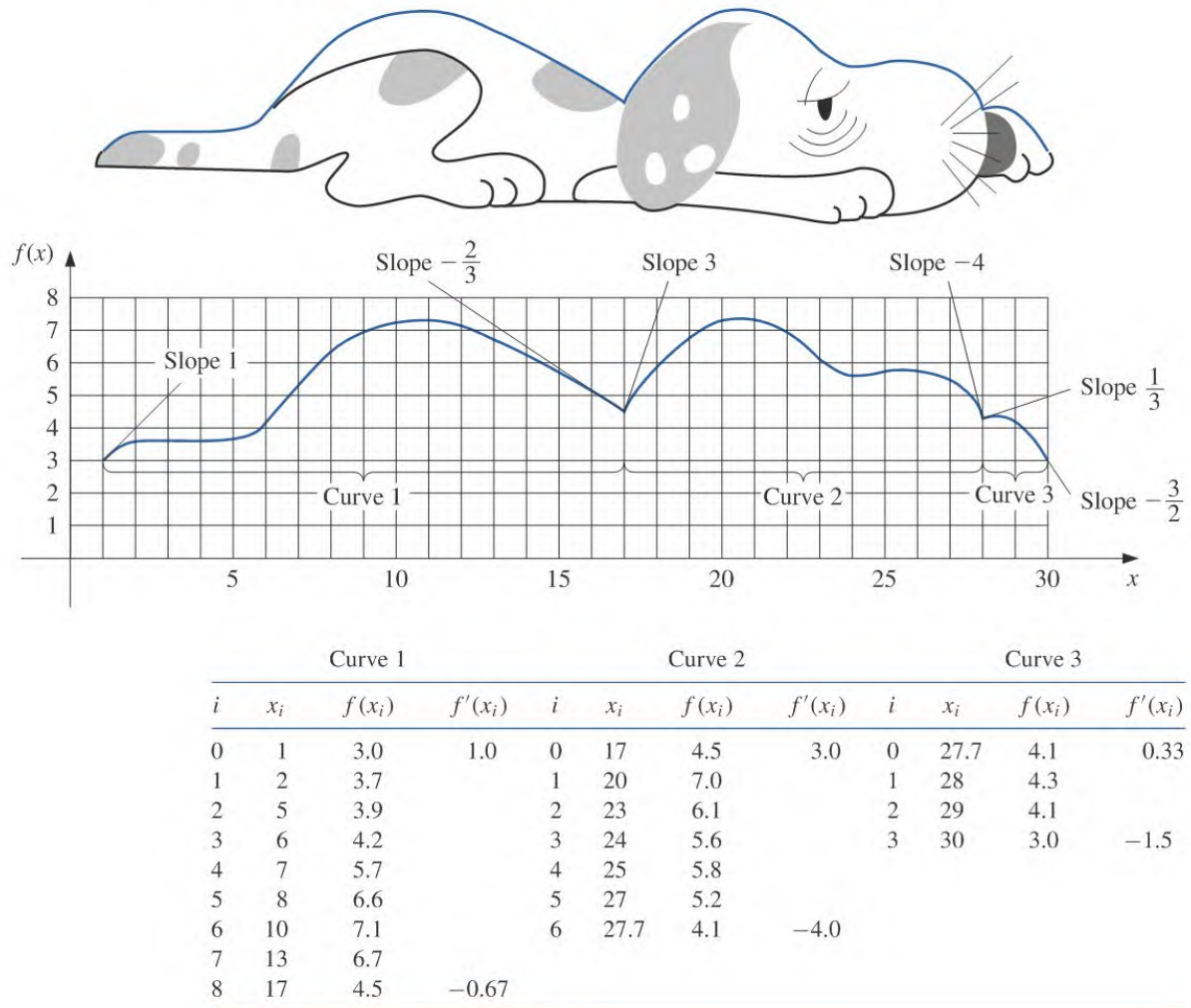
Time	0	3	5	8	13
Distance	0	225	383	623	993
Speed	75	77	80	74	72

- a. Use a clamped cubic spline to predict the position of the car and its speed when $t = 10$ seconds.
 - b. Use the derivative of the spline to determine whether the car ever exceeds a 55-mile-per-hour speed limit on the road; if so, what is the first time the car exceeds this speed?
 - c. What is the predicted maximum speed for the car?
24. a. The introduction to this chapter included a table listing the population of the United States from 1960 to 2010. Use natural cubic spline interpolation to approximate the population in the years 1950, 1975, 2014, and 2020.
- b. The population in 1950 was approximately 150,697,360, and in 2014 the population was estimated to be 317,298,000. How accurate do you think your 1975 and 2020 figures are?
25. It is suspected that the high amounts of tannin in mature oak leaves inhibit the growth of the winter moth (*Operophtera bromata* L., *Geometridae*) larvae that extensively damage these trees in certain years. The following table lists the average weight of two samples of larvae at times in the first 28 days after birth. The first sample was reared on young oak leaves, whereas the second sample was reared on mature leaves from the same tree.
- a. Use a natural cubic spline to approximate the average weight curve for each sample.
 - b. Find an approximate maximum average weight for each sample by determining the maximum of the spline.

Day	0	6	10	13	17	20	28
Sample 1 average weight (mg)	6.67	17.33	42.67	37.33	30.10	29.31	28.74
Sample 2 average weight (mg)	6.67	16.11	18.89	15.00	10.56	9.44	8.89

26. The 2014 Kentucky Derby was won by a horse named California Chrome (5:2 odds favorite) in a time of 2:03.66 (2 minutes and 3.66 seconds) for the $1\frac{1}{4}$ -mile race. Times at the quarter-mile, half-mile, and mile poles were 0:23.04, 0:47.37, and 1:37.45.
- a. Use these values together with the starting time to construct a natural cubic spline for California Chrome's race.
 - b. Use the spline to predict the time at the three-quarter-mile pole and compare this to the actual time of 1:11.80.
 - c. Use the spline to predict California Chrome's starting speed and speed at the finish line.
27. The upper portion of this noble beast is to be approximated using clamped cubic spline interpolants. The curve is drawn on a grid from which the table is constructed. Use Algorithm 3.5 to construct the three clamped cubic splines.

28. Repeat Exercise 27, constructing three natural splines using Algorithm 3.4.



THEORETICAL EXERCISES

29. Suppose that $f(x)$ is a polynomial of degree three. Show that $f(x)$ is its own clamped cubic spline but that it cannot be its own natural cubic spline.
30. Suppose the data $\{x_i, f(x_i)\}_{i=1}^n$ lie on a straight line. What can be said about the natural and clamped cubic splines for the function f ? [Hint: Take a cue from the results of Exercises 1 and 2.]
31. Extend Algorithms 3.4 and 3.5 to include as output the first and second derivatives of the spline at the nodes.
32. Extend Algorithms 3.4 and 3.5 to include as output the integral of the spline over the interval $[x_0, x_n]$.
33. Let $f \in C^2[a, b]$ and let the nodes $a = x_0 < x_1 < \cdots < x_n = b$ be given. Derive an error estimate similar to that in Theorem 3.13 for the piecewise linear interpolating function F . Use this estimate to derive error bounds for Exercise 15.
34. Let f be defined on $[a, b]$ and let the nodes $a = x_0 < x_1 < x_2 = b$ be given. A quadratic spline interpolating function S consists of the quadratic polynomial

$$S_0(x) = a_0 + b_0(x - x_0) + c_0(x - x_0)^2 \quad \text{on } [x_0, x_1]$$

and the quadratic polynomial

$$S_1(x) = a_1 + b_1(x - x_1) + c_1(x - x_1)^2 \quad \text{on } [x_1, x_2],$$

such that

- i. $S(x_0) = f(x_0)$, $S(x_1) = f(x_1)$, and $S(x_2) = f(x_2)$,
- ii. $S \in C^1[x_0, x_2]$.

Show that conditions (i) and (ii) lead to five equations in the six unknowns a_0 , b_0 , c_0 , a_1 , b_1 , and c_1 . The problem is to decide what additional condition to impose to make the solution unique. Does the condition $S \in C^2[x_0, x_2]$ lead to a meaningful solution?

35. Determine a quadratic spline s that interpolates the data $f(0) = 0$, $f(1) = 1$, and $f(2) = 2$ and satisfies $s'(0) = 2$.

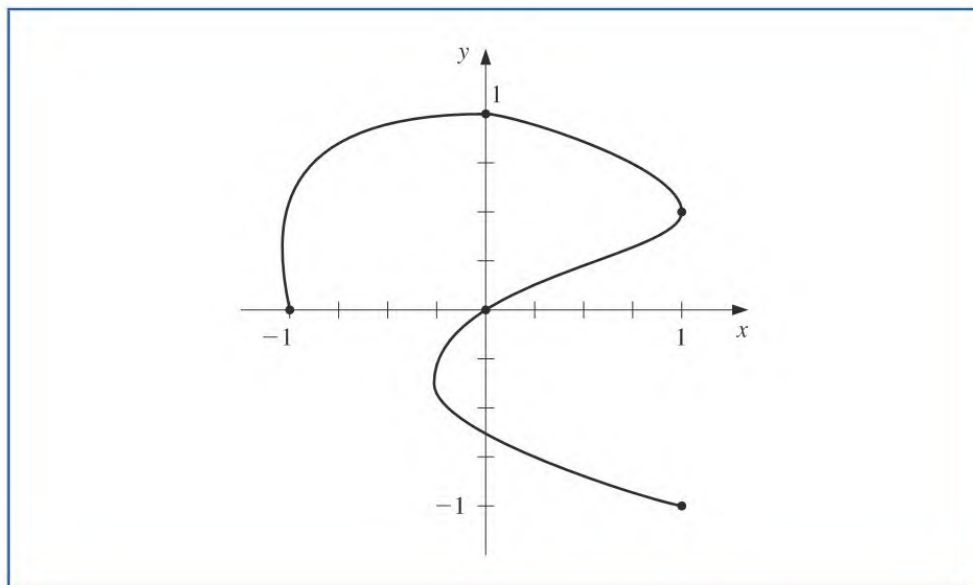
DISCUSSION QUESTIONS

1. Piecewise linear interpolation and cubic spline interpolation were discussed in this section. Quadratic spline interpolation was introduced in an exercise. Higher-degree splines can be computed. Compare the use of quadratic spline interpolation versus cubic spline interpolation.
2. Investigate the so-called not-a-knot interpolation, which is an alternative to natural and clamped cubic spline interpolation.

3.6 Parametric Curves

None of the techniques developed in this chapter can be used to generate curves of the form shown in Figure 3.15, because this curve cannot be expressed as a function of one coordinate variable in terms of the other. In this section, we will see how to represent general curves by using a parameter to express both the x - and the y -coordinate variables. Any good book on computer graphics will show how this technique can be extended to represent general curves and surfaces in space. (See, for example, [FVFH].)

Figure 3.15



A straightforward parametric technique for determining a polynomial or piecewise polynomial to connect the points (x_0, y_0) , (x_1, y_1) , $\dots$, (x_n, y_n) in the order given is to use

a parameter t on an interval $[t_0, t_n]$, with $t_0 < t_1 < \cdots < t_n$, and construct approximation functions with

$$x_i = x(t_i) \quad \text{and} \quad y_i = y(t_i), \quad \text{for each } i = 0, 1, \dots, n.$$

The following example demonstrates the technique in the case where both approximating functions are Lagrange interpolating polynomials.

Example 1 Construct a pair of Lagrange polynomials to approximate the curve shown in Figure 3.15, using the data points shown on the curve.

Solution There is flexibility in choosing the parameter, and we will choose the points $\{t_i\}_{i=0}^4$ equally spaced in $[0, 1]$, which gives the data in Table 3.20.

Table 3.20

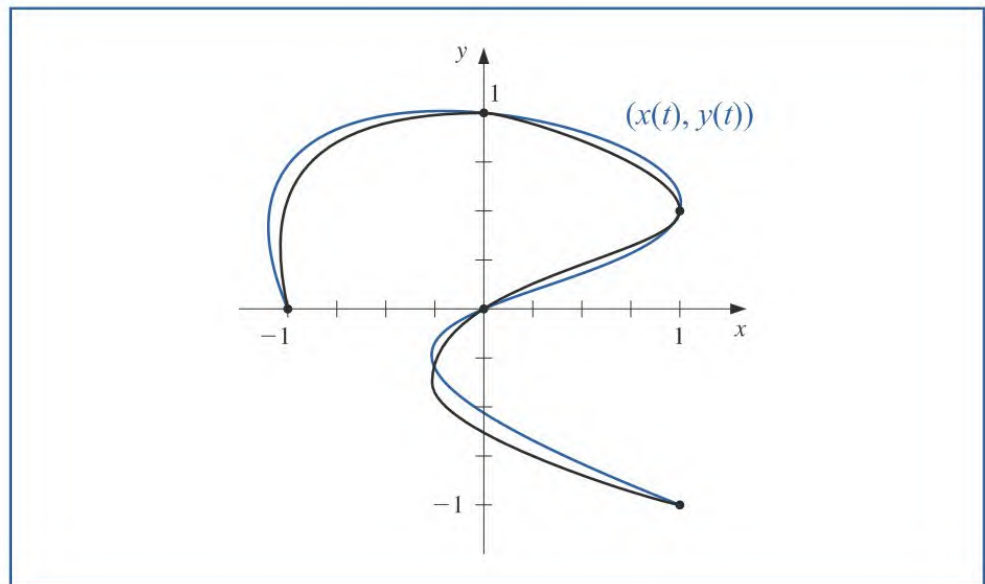
i	0	1	2	3	4
t_i	0	0.25	0.5	0.75	1
x_i	-1	0	1	0	1
y_i	0	1	0.5	0	-1

This produces the interpolating polynomials

$$x(t) = \left(\left((64t - \frac{352}{3})t + 60 \right)t - \frac{14}{3} \right)t - 1 \quad \text{and} \quad y(t) = \left(\left((-\frac{64}{3}t + 48)t - \frac{116}{3} \right)t + 11 \right)t.$$

Plotting this parametric system produces the graph shown in blue in Figure 3.16. Although it passes through the required points and has the same basic shape, it is quite a crude approximation to the original curve. A more accurate approximation would require additional nodes, with the accompanying increase in computation. ■

Figure 3.16



Parametric Hermite and spline curves can be generated in a similar manner, but these also require extensive computational effort.

A successful computer design system needs to be based on a formal mathematical theory so that the results are predictable, but this theory should be performed in the background so that the artist can base the design on aesthetics.

Applications in computer graphics require the rapid generation of smooth curves that can be easily and quickly modified. For both aesthetic and computational reasons, changing one portion of these curves should have little or no effect on other portions of the curves. This eliminates the use of interpolating polynomials and splines since changing one portion of these curves affects the whole curve.

The choice of curve for use in computer graphics is generally a form of the piecewise cubic Hermite polynomial. Each portion of a cubic Hermite polynomial is completely determined by specifying its endpoints and the derivatives at these endpoints. As a consequence, one portion of the curve can be changed while leaving most of the curve the same. Only the adjacent portions need to be modified to ensure smoothness at the endpoints. The computations can be performed quickly, and the curve can be modified a section at a time.

The problem with Hermite interpolation is the need to specify the derivatives at the endpoints of each section of the curve. Suppose the curve has $n + 1$ data points $(x(t_0), y(t_0)), \dots, (x(t_n), y(t_n))$ and we wish to parameterize the cubic to allow complex features. Then we must specify $x'(t_i)$ and $y'(t_i)$, for each $i = 0, 1, \dots, n$. This is not as difficult as it would first appear since each portion is generated independently. We must ensure only that the derivatives at the endpoints of each portion match those in the adjacent portion. Essentially, then, we can simplify the process to one of determining a pair of cubic Hermite polynomials in the parameter t , where $t_0 = 0$ and $t_1 = 1$, given the endpoint data $(x(0), y(0))$ and $(x(1), y(1))$ and the derivatives dy/dx (at $t = 0$) and dy/dx (at $t = 1$).

Notice, however, that we are specifying only six conditions, and the cubic polynomials in $x(t)$ and $y(t)$ each have four parameters, for a total of eight. This provides flexibility in choosing the pair of cubic Hermite polynomials to satisfy the conditions because the natural form for determining $x(t)$ and $y(t)$ requires that we specify $x'(0)$, $x'(1)$, $y'(0)$, and $y'(1)$. The explicit Hermite curve in x and y requires specifying only the quotients

$$\frac{dy}{dx}(t = 0) = \frac{y'(0)}{x'(0)} \quad \text{and} \quad \frac{dy}{dx}(t = 1) = \frac{y'(1)}{x'(1)}.$$

By multiplying $x'(0)$ and $y'(0)$ by a common scaling factor, the tangent line to the curve at $(x(0), y(0))$ remains the same, but the shape of the curve varies. The larger the scaling factor, the closer the curve comes to approximating the tangent line near $(x(0), y(0))$. A similar situation exists at the other endpoint $(x(1), y(1))$.

To further simplify the process in interactive computer graphics, the derivative at an endpoint is specified by using a second point, called a *guidepoint*, on the desired tangent line. The farther the guidepoint is from the node, the more closely the curve approximates the tangent line near the node.

In Figure 3.17, the nodes occur at (x_0, y_0) and (x_1, y_1) , the guidepoint for (x_0, y_0) is $(x_0 + \alpha_0, y_0 + \beta_0)$, and the guidepoint for (x_1, y_1) is $(x_1 - \alpha_1, y_1 - \beta_1)$. The cubic Hermite polynomial $x(t)$ on $[0, 1]$ satisfies

$$x(0) = x_0, \quad x(1) = x_1, \quad x'(0) = \alpha_0, \quad \text{and} \quad x'(1) = \alpha_1.$$

The unique cubic polynomial satisfying these conditions is

$$x(t) = [2(x_0 - x_1) + (\alpha_0 + \alpha_1)]t^3 + [3(x_1 - x_0) - (\alpha_1 + 2\alpha_0)]t^2 + \alpha_0 t + x_0. \quad (3.23)$$

In a similar manner, the unique cubic polynomial satisfying

$$y(0) = y_0, \quad y(1) = y_1, \quad y'(0) = \beta_0, \quad \text{and} \quad y'(1) = \beta_1$$

is

$$y(t) = [2(y_0 - y_1) + (\beta_0 + \beta_1)]t^3 + [3(y_1 - y_0) - (\beta_1 + 2\beta_0)]t^2 + \beta_0 t + y_0. \quad (3.24)$$

Figure 3.17

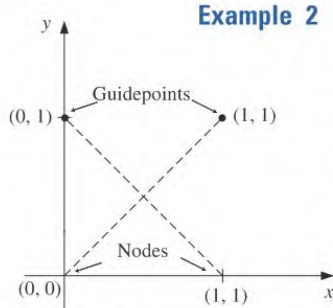
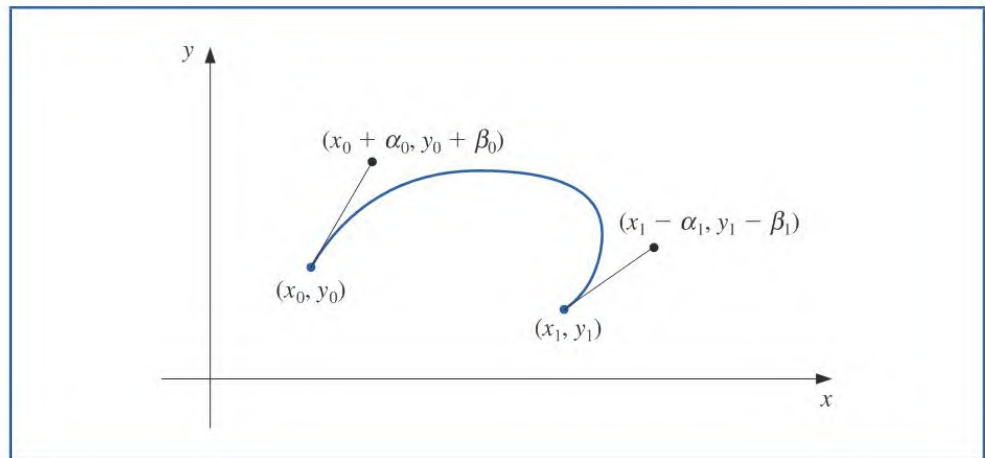


Figure 3.18

Pierre Etienne Bézier (1910–1999) was head of design and production for Renault motorcars for most of his professional life. He began his research into computer-aided design and manufacturing in 1960, developing interactive tools for curve and surface design, and initiated computer-generated milling for automobile modeling. The Bézier curves that bear his name have the advantage of being based on a rigorous mathematical theory that does not need to be explicitly recognized by the practitioner who simply wants to make an aesthetically pleasing curve or surface. These are the curves that are the basis of the powerful Adobe Postscript system and produce the freehand curves that are generated in most sufficiently powerful computer graphics packages.

Example 2

Determine the graph of the parametric curve generated by Eqs. (3.23) and (3.24) when the endpoints are $(x_0, y_0) = (0, 0)$ and $(x_1, y_1) = (1, 0)$ and respective guide points, as shown in Figure 3.18, are $(1, 1)$ and $(0, 1)$.

Solution The endpoint information implies that $x_0 = 0$, $x_1 = 1$, $y_0 = 0$, and $y_1 = 0$, and the guidepoints at $(1, 1)$ and $(0, 1)$ imply that $\alpha_0 = 1$, $\alpha_1 = 1$, $\beta_0 = 1$, and $\beta_1 = -1$. Note that the slopes of the guide lines at $(0, 0)$ and $(1, 0)$ are, respectively,

$$\frac{\beta_0}{\alpha_0} = \frac{1}{1} = 1 \quad \text{and} \quad \frac{\beta_1}{\alpha_1} = \frac{-1}{1} = -1.$$

Equations (3.23) and (3.24) imply that for $t \in [0, 1]$, we have

$$x(t) = [2(0 - 1) + (1 + 1)]t^3 + [3(0 - 0) - (1 + 2 \cdot 1)]t^2 + 1 \cdot t + 0 = t$$

and

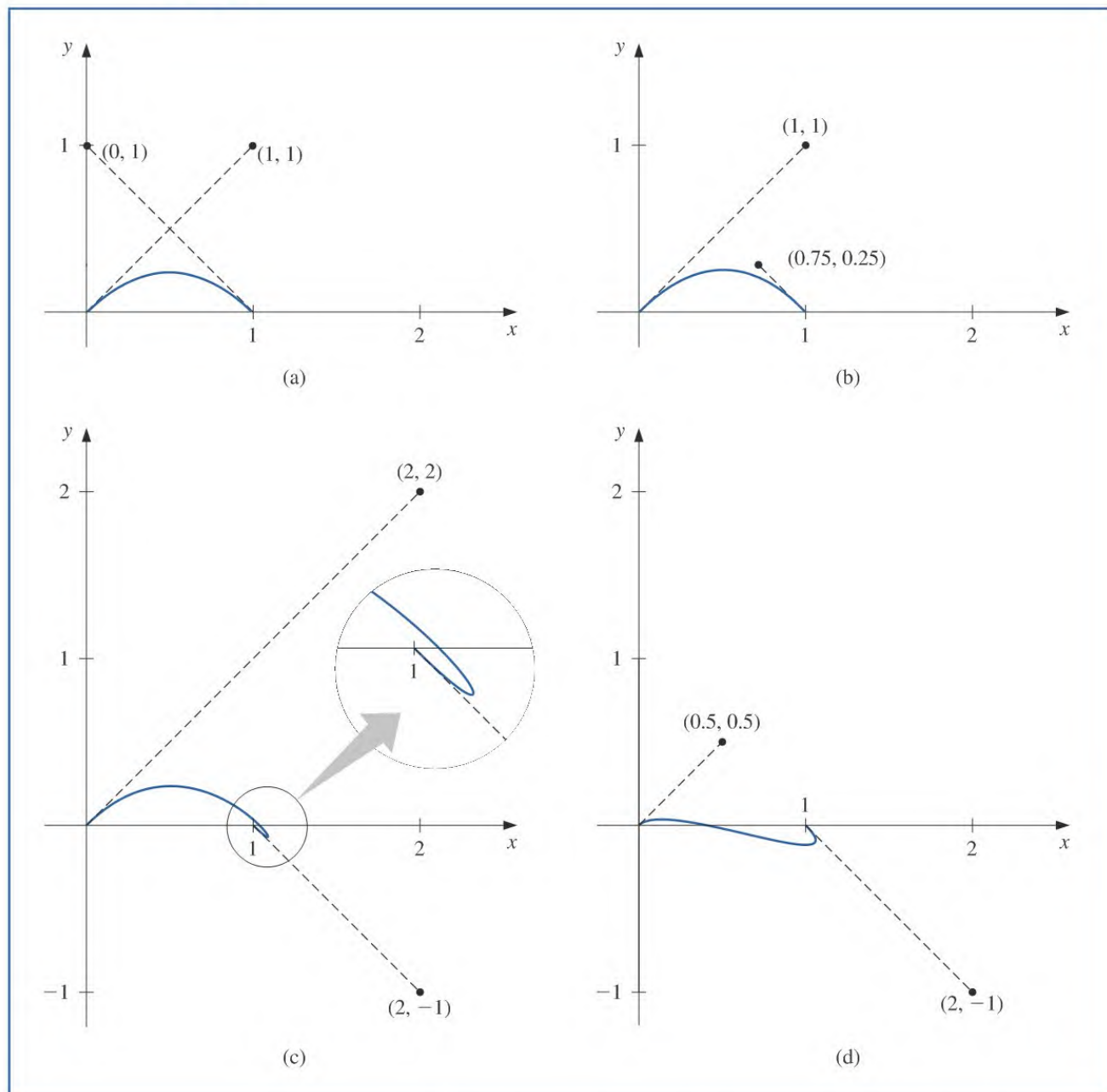
$$y(t) = [2(0 - 0) + (1 + (-1))]t^3 + [3(0 - 0) - (-1 + 2 \cdot 1)]t^2 + 1 \cdot t + 0 = -t^2 + t.$$

This graph is shown as (a) in Figure 3.19, together with some other possibilities of curves produced by Eqs. (3.23) and (3.24) when the nodes are $(0, 0)$ and $(1, 0)$ and the slopes at these nodes are 1 and -1 , respectively. ■

The standard procedure for determining curves in an interactive graphics mode is to first use a mouse or touchpad to set the nodes and guidepoints to generate a first approximation to the curve. These can be set manually, but most graphics systems permit you to use your input device to draw the curve on the screen freehand and will select appropriate nodes and guidepoints for your freehand curve.

The nodes and guidepoints can then be manipulated into a position that produces an aesthetically pleasing curve. Since the computation is minimal, the curve can be determined so quickly that the resulting change is seen immediately. Moreover, all the data needed to compute the curves are embedded in the coordinates of the nodes and guidepoints, so no analytical knowledge is required of the user.

Figure 3.19



Popular graphics programs use this type of system for their freehand graphic representations in a slightly modified form. The Hermite cubics are described as **Bézier polynomials**, which incorporate a scaling factor of three when computing the derivatives at the endpoints. This modifies the parametric equations to

$$x(t) = [2(x_0 - x_1) + 3(\alpha_0 + \alpha_1)]t^3 + [3(x_1 - x_0) - 3(\alpha_1 + 2\alpha_0)]t^2 + 3\alpha_0 t + x_0 \quad (3.25)$$

and

$$y(t) = [2(y_0 - y_1) + 3(\beta_0 + \beta_1)]t^3 + [3(y_1 - y_0) - 3(\beta_1 + 2\beta_0)]t^2 + 3\beta_0 t + y_0, \quad (3.26)$$

for $0 \leq t \leq 1$, but this change is transparent to the user of the system.

Algorithm 3.6 constructs a set of Bézier curves based on the parametric equations in Eqs. (3.25) and (3.26).

ALGORITHM 3.6

Bézier Curve

To construct the cubic Bézier curves $C_0, \dots, C_{n-1}$ in parametric form, where C_i is represented by

$$(x_i(t), y_i(t)) = (a_0^{(i)} + a_1^{(i)}t + a_2^{(i)}t^2 + a_3^{(i)}t^3, b_0^{(i)} + b_1^{(i)}t + b_2^{(i)}t^2 + b_3^{(i)}t^3),$$

for $0 \leq t \leq 1$, as determined by the left endpoint (x_i, y_i) , left guidepoint (x_i^+, y_i^+) , right endpoint (x_{i+1}, y_{i+1}) , and right guidepoint (x_{i+1}^-, y_{i+1}^-) for each $i = 0, 1, \dots, n-1$:

INPUT $n; (x_0, y_0), \dots, (x_n, y_n); (x_0^+, y_0^+), \dots, (x_{n-1}^+, y_{n-1}^+); (x_1^-, y_1^-), \dots, (x_n^-, y_n^-)$.

OUTPUT coefficients $\{a_0^{(i)}, a_1^{(i)}, a_2^{(i)}, a_3^{(i)}, b_0^{(i)}, b_1^{(i)}, b_2^{(i)}, b_3^{(i)}, \text{ for } 0 \leq i \leq n-1\}$.

Step 1 For each $i = 0, 1, \dots, n-1$ do Steps 2 and 3.

Step 2 Set $a_0^{(i)} = x_i$;
 $b_0^{(i)} = y_i$;
 $a_1^{(i)} = 3(x_i^+ - x_i)$;
 $b_1^{(i)} = 3(y_i^+ - y_i)$;
 $a_2^{(i)} = 3(x_i + x_{i+1}^- - 2x_i^+)$;
 $b_2^{(i)} = 3(y_i + y_{i+1}^- - 2y_i^+)$;
 $a_3^{(i)} = x_{i+1} - x_i + 3x_i^+ - 3x_{i+1}^-$;
 $b_3^{(i)} = y_{i+1} - y_i + 3y_i^+ - 3y_{i+1}^-$;

Step 3 OUTPUT $(a_0^{(i)}, a_1^{(i)}, a_2^{(i)}, a_3^{(i)}, b_0^{(i)}, b_1^{(i)}, b_2^{(i)}, b_3^{(i)})$.

Step 4 STOP. ■

Three-dimensional curves are generated in a similar manner by additionally specifying third components z_0 and z_1 for the nodes and $z_0 + \gamma_0$ and $z_1 - \gamma_1$ for the guidepoints. The more difficult problem involving the representation of three-dimensional curves concerns the loss of the third dimension when the curve is projected onto a two-dimensional computer screen. Various projection techniques are used, but this topic lies within the realm of computer graphics. For an introduction to this topic and ways that the technique can be modified for surface representations, see one of the many books on computer graphics methods, such as [FVFH].

EXERCISE SET 3.6

1. Let $(x_0, y_0) = (0, 0)$ and $(x_1, y_1) = (5, 2)$ be the endpoints of a curve. Use the given guidepoints to construct parametric cubic Hermite approximations $(x(t), y(t))$ to the curve and graph the approximations.
 - a. $(1, 1)$ and $(6, 1)$
 - b. $(0.5, 0.5)$ and $(5.5, 1.5)$
 - c. $(1, 1)$ and $(6, 3)$
 - d. $(2, 2)$ and $(7, 0)$

2. Repeat Exercise 1 using cubic Bézier polynomials.
3. Construct and graph the cubic Bézier polynomials given the following points and guidepoints.
 - a. Point (1, 1) with guidepoint (1.5, 1.25) to point (6, 2) with guidepoint (7, 3)
 - b. Point (1, 1) with guidepoint (1.25, 1.5) to point (6, 2) with guidepoint (5, 3)
 - c. Point (0, 0) with guidepoint (0.5, 0.5) to point (4, 6) with entering guidepoint (3.5, 7) and exiting guidepoint (4.5, 5) to point (6, 1) with guidepoint (7, 2)
 - d. Point (0, 0) with guidepoint (0.5, 0.5) to point (2, 1) with entering guidepoint (3, 1) and exiting guidepoint (3, 1) to point (4, 0) with entering guidepoint (5, 1) and exiting guidepoint (3, -1) to point (6, -1) with guidepoint (6.5, -0.25)
4. Use the data in the following table and Algorithm 3.6 to approximate the shape of the letter N.

i	x_i	y_i	α_i	β_i	α'_i	β'_i
0	3	6	3.3	6.5		
1	2	2	2.8	3.0	2.5	2.5
2	6	6	5.8	5.0	5.0	5.8
3	5	2	5.5	2.2	4.5	2.5
4	6.5	3			6.4	2.8

THEORETICAL EXERCISES

5. Suppose a cubic Bézier polynomial is placed through (u_0, v_0) and (u_3, v_3) with guidepoints (u_1, v_1) and (u_2, v_2) , respectively.
 - a. Derive the parametric equations for $u(t)$ and $v(t)$ assuming that

$$u(0) = u_0, \quad u(1) = u_3, \quad u'(0) = u_1 - u_0, \quad u'(1) = u_3 - u_2$$

and

$$v(0) = v_0, \quad v(1) = v_3, \quad v'(0) = v_1 - v_0, \quad v'(1) = v_3 - v_2.$$

- b. Let $f(i/3) = u_i$, for $i = 0, 1, 2, 3$, and $g(i/3) = v_i$, for $i = 0, 1, 2, 3$. Show that the Bernstein polynomial of degree three in t for f is $u(t)$ and the Bernstein polynomial of degree three in t for g is $v(t)$. (See Exercise 23 of Section 3.1.)

DISCUSSION QUESTIONS

1. Investigate the usefulness of the methods in this section to graphics packages.



3.7 Numerical Software and Chapter Review

Interpolation routines included in the IMSL Library are based on the book *A Practical Guide to Splines* by Carl de Boor [Deb] and use interpolation by cubic splines. There are cubic splines to minimize oscillations and to preserve concavity. Methods for two-dimensional interpolation by bicubic splines are also included.

The NAG library contains subroutines for polynomial and Hermite interpolation, for cubic spline interpolation, and for piecewise cubic Hermite interpolation. NAG also contains subroutines for interpolating functions of two variables.

The netlib library contains the subroutines to compute the cubic spline with various endpoint conditions. One package produces the Newton's divided-difference coefficients for a discrete set of data points, and there are various routines for evaluating Hermite piecewise polynomials.

DISCUSSION QUESTIONS

1. Interpolating polynomials are adversely affected by bad data. That is, an error in one data point will affect the entire interpolating polynomial. Splines make it possible to confine the ill effects of an erroneous data point. Discuss how this is accomplished.
2. Cubic splines have the following properties: (a) they interpolate the given data; (b) they have continuity of the zeroth, first, and second derivatives at interior points; and (c) they satisfy certain boundary conditions. Discuss the options for the boundary conditions.

KEY CONCEPTS

Interpolation	Lagrange Polynomial
Weierstrass Approximation Theorem	Neville's Method
Divided Differences	Newton's Divided-Difference Formula
Forward Difference	Backward Difference
Stirling's Formula	Forward-Difference Operator
Backward-Difference Operator	Hermite Interpolation
Osculating Polynomial	Cubic Spline Interpolation
Piecewise Polynomial Approximation	Natural Boundary Condition
Clamped Boundary Condition	Cubic Spline
Natural Cubic Spline	Clamped Cubic Spline
Parametric Curve	Bézier Curve
Error Formulas	

CHAPTER REVIEW

In this chapter, we considered approximating a function using polynomials and piecewise polynomials. We found that the function could be specified by a given defining equation or by providing points in the plane through which the graph of the function passes. A set of nodes $x_0, x_1, \dots, x_n$ was given in each case, and more information, such as the value of various derivatives, may also have been required. We needed to find an approximating function that satisfied the conditions specified by these data.

The interpolating polynomial $P(x)$ was the polynomial of least degree that satisfied, for a function f ,

$$P(x_i) = f(x_i), \quad \text{for each } i = 0, 1, \dots, n.$$

We found that although this interpolating polynomial was unique, it could take many different forms. The Lagrange form was most often used for interpolating tables when n was small and for deriving formulas for approximating derivatives and integrals. Neville's method was used for evaluating several interpolating polynomials at the same value of x . We saw that Newton's forms of the polynomial were more appropriate for computation and were also used extensively for deriving formulas for solving differential equations. However, polynomial interpolation had the inherent weaknesses of oscillation, particularly if the number of nodes was large. In this case, there were other methods that could be better applied.

The Hermite polynomials interpolated a function and its derivative at the nodes. They could be very accurate but required more information about the function being approximated. When there were a large number of nodes, the Hermite polynomials also exhibited oscillation weaknesses.

The most commonly used form of interpolation was piecewise-polynomial interpolation. If function and derivative values were available, piecewise cubic Hermite interpolation was recommended. In fact, this was the preferred method for interpolating values of a function that was the solution to a differential equation. We noted that when only the function values were available, natural cubic spline interpolation could be used. This spline forced the second derivative of the spline to be zero at the endpoints. Other cubic splines required additional data. For example, the clamped cubic spline needed values of the derivative of the function at the endpoints of the interval.

As we stated in Section 3.3, our treatment of divided-difference methods was brief since the results in this section will not be used extensively in subsequent material. Most older texts on numerical analysis have extensive treatments of divided-difference methods. If a more comprehensive treatment is needed, the book by Hildebrand [Hild] is a particularly good reference.

As a final note, there are other methods of interpolation that are commonly used. Trigonometric interpolation, in particular, the Fast Fourier Transform discussed in Chapter 8, is used with large amounts of data when the function is assumed to have a periodic nature. Interpolation by rational functions is also used.

If the data are suspected to be inaccurate, smoothing techniques can be applied, and some form of least squares fit of data is recommended. Polynomials, trigonometric functions, rational functions, and splines can be used in least squares fitting of data. We consider these topics in Chapter 8.

General references to the methods in this chapter are the books by Powell [Pow] and by Davis [Da]. The seminal paper on splines is due to Schoenberg [Scho]. Important books on splines are by Schultz [Schul], De Boor [Deb2], Dierckx [Di], and Schumaker [Schum].